

Mini Claude Code

Python s01 -> s10 逐层复刻完整工程教程

Agent Loop / Tools / Permissions / Context / Memory / Skills / Hooks / Subagents / Sandbox / Multi-Agent

核心方法

每一层只增加一个主要抽象： $sN = sN-1 + \text{一个可解释能力}$ 。重点同时回答 know-why（为什么需要这一层）与 know-how（代码如何落地、如何验证、哪里容易错）。

版本：2026-08-13 | Python 3.11+ | 教学实现

基于 Anthropic/Claude Code 公开文档抽象重建；不包含、复制或依赖未授权专有源码。

0. 如何使用这本教程

这不是“照着 Claude Code UI 抄功能”的教程，而是一条可执行的 Agent Runtime 演化路径。你可以从 s01 直接运行，然后每完成一章就切换到下一目录，观察代码为何需要新增一个层。所有代码均为教学用原创实现。

0.1 成功标准

- 理解最小 tool-use 状态机，知道 Host 为什么是 Agent 的真实执行边界。
- 能把工具能力、权限策略、上下文治理和持久知识拆成互不混淆的层。
- 能实现 progressive disclosure Skill、deterministic Hook 和 context-isolated Subagent。
- 能解释 “Permission != Sandbox” “Subagent != Agent Team” “Memory != Context”。
- 最终能运行一个支持 Git worktree 并行 Worker 的 Mini Multi-Agent Runtime。

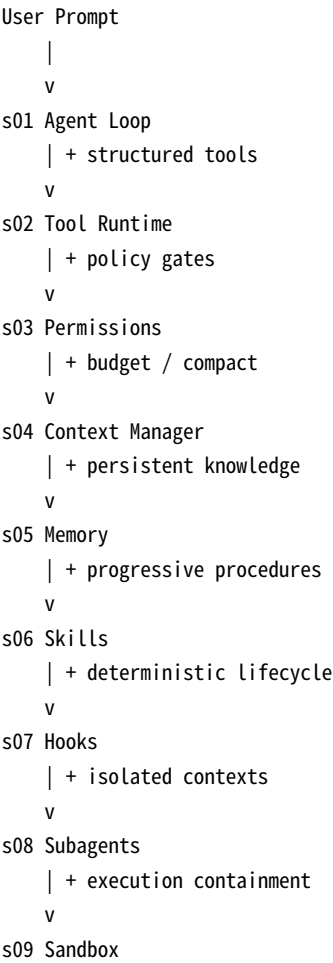
0.2 安全与准确性边界

重要

s09 的 soft sandbox 只提供 cwd、环境变量净化和 timeout，不是强 OS 安全边界。Bubblewrap 例子也只是教学基线。任何真实生产 Coding Agent 都应再加容器/VM、网络代理、密钥隔离、审计、资源限额和组织策略。

另外，本书把公开文档中的 Claude Code 机制映射为一个可学习的 Python Runtime，但不声称代码结构、函数名、目录或内部策略与 Anthropic 私有实现相同。

0.3 全局演化图



| + worktrees / parallel workers / mailbox
v
s10 Multi-Agent Runtime

0.4 十层总览

Stage	新增抽象	核心问题	最终收益
s01	Agent Loop	模型如何连续调用工具？	最小闭环
s02	Tool Runtime	如何让能力可扩展、可约束？	结构化工具 + dispatch
s03	Permission	谁决定副作用能否执行？	Host policy
s04	Context	历史越来越长怎么办？	统计 + compact
s05	Memory	哪些信息要跨会话？	规则 + 持久学习
s06	Skills	重复 SOP 如何按需加载？	progressive disclosure
s07	Hooks	必须发生的动作如何保证？	deterministic lifecycle
s08	Subagents	探索如何不污染主 Context？	context isolation
s09	Sandbox	批准后如何限制 blast radius？	execution containment
s10	Multi-Agent	并行修改如何避免互相踩文件？	worktree + workers

1. 环境、运行方式与学习约定

1.1 依赖

- Python 3.11 或更高版本。
- `anthropic`：Messages API。
- `python-dotenv`：本地开发加载环境变量。
- `pyyaml`：s06 以后解析 Skill/Subagent frontmatter。
- Git：s10 worktree 必需；s01-s09 不强制。

1.2 推荐初始化

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install anthropic python-dotenv pyyaml
export ANTHROPIC_API_KEY=...
export ANTHROPIC_MODEL=<your-model-id>
cd s01_agent_loop
python code.py
```

为什么不在代码里写死 model ID?

模型命名和账号可用范围会变化。教程通过 ANTHROPIC_MODEL 注入具体模型，使 Runtime 的架构与某个时间点的模型名称解耦。

1.3 API 协议必须先理解

Claude Platform 当前公开文档把客户端工具循环明确描述为：发送 messages + tools；模型返回 stop_reason=tool_use 与一个或多个 tool_use block；Host 执行；再发送包含匹配 tool_use_id 的 tool_result；重复直到模型停止请求客户端工具。[R1][R2]

```
messages + tools
  |
  v
Claude response
  | stop_reason == tool_use
  v
scan tool_use blocks
  |
  v
Host executes tools
  |
  v
user content: tool_result(tool_use_id=...)
  |
  +-----> next Messages API call

otherwise -> final answer / other stop reason
```

2. s01 - Agent Loop

从 0 到最小可运行 Agent Loop

观察项	本阶段
阶段目录	s01_agent_loop
文件数量	2
新增文件	README.md, code.py
继承/修改文件	无
Python 行数	150

KNOW-WHY：为什么这一层必须存在

- Agent 的最小闭环不是“模型会调用函数”，而是 Host 必须把 tool_use 结果执行后，以 tool_result 再喂回模型。只要这个闭环正确，后续所有能力都可以在它外面分层叠加。
- 把 stop_reason 当作协议状态机，而不是把一次 API 调用当成一次完整任务。模型可以经历多轮工具调用后才给出最终自然语言答案。
- 工具副作用由 Host 执行，这意味着真正的安全边界、审计、权限和超时必须在 Host 侧实现，而不能只依赖模型提示词。

KNOW-HOW：这一层具体怎么长出来

- 只暴露一个 bash 工具，先让协议足够简单。
- 每次模型响应都原样加入 assistant history；如果 stop_reason == tool_use，则扫描所有 tool_use block。
- Host 执行命令，构造带相同 tool_use_id 的 tool_result，并以 user content block 形式加入 history。
- 循环直到 stop_reason 不再是 tool_use。

数据流 / 控制流

User -> Messages API -> tool_use -> Host/bash -> tool_result -> Messages API -> ... -> final

最容易踩的坑

- 只把工具结果发回模型，却忘了把上一条 assistant tool_use 响应也加入 history。
- 假设 response.content[0] 一定是 tool_use；实际上文本块和多个 tool_use 可以混合出现。
- 把模型输出直接当 shell 命令执行，却没有权限、超时或隔离。这是 s01 故意留下、在后续阶段逐步补齐的缺口。

本阶段验收清单

- 让 Agent 回答当前工作目录，并观察是否经历一次 bash -> tool_result -> end_turn。
- 让 Agent “检查目录后总结项目”，观察是否能连续调用多次工具。
- 制造一个会返回非 0 exit code 的普通命令，确认错误仍作为 tool_result 返回模型而不是让 Host 崩溃。

协议不变量

- 每个 client tool_use 的结果必须通过相同 tool_use_id 对应回去。
- assistant 的 tool_use 响应属于历史的一部分，必须保留。
- 一个响应可能有文本 + 多个 tool_use，因此处理逻辑必须扫描 content blocks。
- Host 应处理 tool execution failure，并让模型看到错误；不要把普通工具失败提升成进程崩溃。

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (150 lines)

```
0001 #!/usr/bin/env python3
0002 """s01 - Minimal agent loop with one Bash tool.
0003
0004 Goal: expose the core client-tool loop:
0005     model -> tool_use -> host executes -> tool_result -> model -> ...
0006
0007 Requirements:
0008     pip install anthropic python-dotenv
0009     ANTHROPIC_API_KEY=...
0010     ANTHROPIC_MODEL=<a model id available to your account>
0011 """
0012 from __future__ import annotations
0013
0014 import json
0015 import os
0016 import subprocess
0017 from typing import Any
0018
0019 from anthropic import Anthropic
0020 from dotenv import load_dotenv
0021
0022 load_dotenv()
0023
0024 MODEL = os.environ.get("ANTHROPIC_MODEL")
0025 if not MODEL:
0026     raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")
0027
0028 client = Anthropic()
0029
0030 TOOLS = [
0031     {
0032         "name": "bash",
0033         "description": "Run a shell command in the current project and return stdout/stderr.",
0034         "input_schema": {
0035             "type": "object",
0036             "properties": {
0037                 "command": {"type": "string", "description": "Shell command to execute"}
0038             },
0039             "required": ["command"],
0040             "additionalProperties": False,
0041         },
0042     },
0043 ]
0044
0045 SYSTEM = """You are Mini Claude Code, a coding agent running in a project directory.
0046 Use the bash tool when you need to inspect files, run tests, or execute commands.
0047 Prefer small, verifiable steps. Explain the final result to the user.
0048 """
0049
0050
0051 def run_bash(command: str) -> str:
0052     """Execute a command and return a compact text result."""
0053     try:
0054         proc = subprocess.run(
0055             command,
0056             shell=True,
0057             text=True,
0058             capture_output=True,
0059             timeout=30,
0060         )
0061     except subprocess.TimeoutExpired:
0062         return "ERROR: command timed out after 30 seconds"
0063
0064     payload = {
0065         "exit_code": proc.returncode,
0066         "stdout": proc.stdout[-12000:],
0067         "stderr": proc.stderr[-12000:],
0068     }
0069     return json.dumps(payload, ensure_ascii=False)
0070
0071
```

```

0072 def block_to_dict(block: Any) -> dict[str, Any]:
0073     """Convert Anthropic SDK content blocks to plain dicts for message history."""
0074     if hasattr(block, "model_dump"):
0075         return block.model_dump(exclude_none=True)
0076     if isinstance(block, dict):
0077         return block
0078     raise TypeError(f"Unsupported block type: {type(block)!r}")
0079
0080
0081 def render_text(blocks: List[Any]) -> None:
0082     for block in blocks:
0083         if getattr(block, "type", None) == "text":
0084             print(block.text)
0085
0086
0087 def agent_loop(messages: List[dict[str, Any]]) -> List[dict[str, Any]]:
0088     """Run until Claude stops requesting client tools."""
0089     while True:
0090         response = client.messages.create(
0091             model=MODEL,
0092             max_tokens=4096,
0093             system=SYSTEM,
0094             tools=TOOLS,
0095             messages=messages,
0096         )
0097
0098         render_text(response.content)
0099         assistant_content = [block_to_dict(b) for b in response.content]
0100         messages.append({"role": "assistant", "content": assistant_content})
0101
0102         if response.stop_reason != "tool_use":
0103             return messages
0104
0105         tool_results: List[dict[str, Any]] = []
0106         for block in response.content:
0107             if getattr(block, "type", None) != "tool_use":
0108                 continue
0109
0110             if block.name == "bash":
0111                 result = run_bash(block.input["command"])
0112             else:
0113                 result = f"ERROR: unknown tool {block.name!r}"
0114
0115             tool_results.append(
0116                 {
0117                     "type": "tool_result",
0118                     "tool_use_id": block.id,
0119                     "content": result,
0120                 }
0121             )
0122
0123         messages.append({"role": "user", "content": tool_results})
0124
0125
0126 def main() -> None:
0127     print("Mini Claude Code s01. Type /exit to quit.")
0128     messages: List[dict[str, Any]] = []
0129
0130     while True:
0131         try:
0132             prompt = input("\nyou> ").strip()
0133         except (EOFError, KeyboardInterrupt):
0134             print()
0135             break
0136
0137         if not prompt:
0138             continue
0139         if prompt in {"/exit", "/quit"}:
0140             break
0141
0142         messages.append({"role": "user", "content": prompt})
0143         try:
0144             messages = agent_loop(messages)
0145         except Exception as exc: # tutorial-friendly top-level boundary
0146             print(f"ERROR: {exc}")
0147
0148
0149 if __name__ == "__main__":
0150     main()

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (10 Lines)

```
0001 # s01 Agent Loop
0002
0003 Run:
0004
0005 ```bash
0006 pip install anthropic python-dotenv
0007 export ANTHROPIC_API_KEY=...
0008 export ANTHROPIC_MODEL=...
0009 python code.py
0010 ```
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

3. s02 - Tool Runtime

加入文件工具、工具分发和工作区路径边界

观察项	本阶段
阶段目录	s02_tools
文件数量	2
新增文件	无（主要修改已有文件）
继承/修改文件	README.md, code.py
Python 行数	227

KNOW-WHY：为什么这一层必须存在

- Coding Agent 不能只有万能 shell。结构化文件工具能缩短参数、减少歧义，并让权限系统能够按“读/写/编辑”表达意图。
- Tool Schema 是给概率模型看的 API affordance：名字、描述、参数边界会直接影响模型是否选对工具。
- 工具实现和 Agent Loop 分离后，新增工具不再修改核心循环；这就是后续 Plugin/MCP/Skill 扩展的基础形态。

KNOW-HOW：这一层具体怎么长出来

5. 加入 read_file / write_file / edit_file / glob，并保留 bash 作为通用逃生舱。
6. 用 TOOL_HANDLERS 做 name -> callable 分发，而不是在 Agent Loop 写一长串 if/elif。
7. 所有文件工具先经过 safe_path：resolve 后必须仍位于 WORKSPACE 之内。
8. 对工具输出做截断，避免一次日志或大文件把 Context Window 灌满。

数据流 / 控制流

```
tool_use -> TOOL_HANDLERS[name] -> safe_path/output-limit -> tool_result
```

最容易踩的坑

- 只检查字符串前缀而不 resolve `..`、符号链接和绝对路径，会产生目录穿越。
- edit_file 如果 old_text 有多个匹配，直接替换第一个会导致不可预测修改；本实现强制唯一匹配。
- 把工具返回完整 stdout/stderr 写入上下文，会很快造成上下文污染。

本阶段验收清单

- 读取一个源码文件。
- 尝试写入工作区内临时文件并再读取。
- 尝试访问 `../` 路径，确认被 safe_path 拒绝。

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (227 lines)

```
0001 #!/usr/bin/env python3
0002 """s02 - Add file tools, handler dispatch, and workspace path safety."""
0003 from __future__ import annotations
0004
0005 import glob as globlib
0006 import json
```

```

0007 import os
0008 import subprocess
0009 from pathlib import Path
0010 from typing import Any, Callable
0011
0012 from anthropic import Anthropic
0013 from dotenv import load_dotenv
0014
0015 load_dotenv()
0016 MODEL = os.environ.get("ANTHROPIC_MODEL")
0017 if not MODEL:
0018     raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")
0019
0020 client = Anthropic()
0021 WORKSPACE = Path.cwd().resolve()
0022 MAX_TOOL_OUTPUT = 12000
0023
0024
0025 def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
0026     return {
0027         "name": name,
0028         "description": description,
0029         "input_schema": {
0030             "type": "object",
0031             "properties": properties,
0032             "required": required,
0033             "additionalProperties": False,
0034         },
0035     }
0036
0037
0038 TOOLS = [
0039     tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
0040     tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
0041     tool(
0042         "write_file",
0043         "Create or replace a UTF-8 text file inside the workspace.",
0044         {"path": {"type": "string"}, "content": {"type": "string"}},
0045         ["path", "content"],
0046     ),
0047     tool(
0048         "edit_file",
0049         "Replace one exact text occurrence in a workspace file.",
0050         {
0051             "path": {"type": "string"},
0052             "old_text": {"type": "string"},
0053             "new_text": {"type": "string"},
0054         },
0055         ["path", "old_text", "new_text"],
0056     ),
0057     tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
0058 ]
0059
0060 SYSTEM = """You are Mini Claude Code, a coding agent.
0061 Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
0062 Make focused changes and verify them before you finish.
0063 """
0064
0065
0066 def safe_path(raw: str) -> Path:
0067     """Resolve a path and reject escapes outside the workspace."""
0068     candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
0069     try:
0070         candidate.relative_to(WORKSPACE)
0071     except ValueError as exc:
0072         raise ValueError(f"Path escapes workspace: {raw!r}") from exc
0073     return candidate
0074
0075
0076 def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
0077     if len(text) <= limit:
0078         return text
0079     half = limit // 2
0080     return text[:half] + "\n...<truncated>...\n" + text[-half:]
0081
0082
0083 def run_bash(command: str) -> str:
0084     try:
0085         proc = subprocess.run(
0086             command,

```

```

0087         shell=True,
0088         text=True,
0089         capture_output=True,
0090         cwd=WORKSPACE,
0091         timeout=30,
0092     )
0093 except subprocess.TimeoutExpired:
0094     return "ERROR: command timed out after 30 seconds"
0095 return json.dumps(
0096     {
0097         "exit_code": proc.returncode,
0098         "stdout": truncate(proc.stdout),
0099         "stderr": truncate(proc.stderr),
0100     },
0101     ensure_ascii=False,
0102 )
0103
0104
0105 def run_read(path: str) -> str:
0106     p = safe_path(path)
0107     if not p.is_file():
0108         return f"ERROR: file not found: {path}"
0109     return truncate(p.read_text(encoding="utf-8", errors="replace"))
0110
0111
0112 def run_write(path: str, content: str) -> str:
0113     p = safe_path(path)
0114     p.parent.mkdir(parents=True, exist_ok=True)
0115     p.write_text(content, encoding="utf-8")
0116     return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"
0117
0118
0119 def run_edit(path: str, old_text: str, new_text: str) -> str:
0120     p = safe_path(path)
0121     if not p.is_file():
0122         return f"ERROR: file not found: {path}"
0123     text = p.read_text(encoding="utf-8", errors="replace")
0124     count = text.count(old_text)
0125     if count == 0:
0126         return "ERROR: old_text not found"
0127     if count > 1:
0128         return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
0129     p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
0130     return f"OK: edited {p.relative_to(WORKSPACE)}"
0131
0132
0133 def run_glob(pattern: str) -> str:
0134     # Resolve matches then apply safe_path to each result to keep output inside the workspace.
0135     raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
0136     matches: List[str] = []
0137     for raw in raw_matches[:500]:
0138         try:
0139             p = safe_path(raw)
0140         except ValueError:
0141             continue
0142         matches.append(str(p.relative_to(WORKSPACE)))
0143     return "\n".join(sorted(matches)) if matches else "(no matches)"
0144
0145
0146 TOOL_HANDLERS: dict[str, Callable[..., str]] = {
0147     "bash": run_bash,
0148     "read_file": run_read,
0149     "write_file": run_write,
0150     "edit_file": run_edit,
0151     "glob": run_glob,
0152 }
0153
0154
0155 def block_to_dict(block: Any) -> dict[str, Any]:
0156     if hasattr(block, "model_dump"):
0157         return block.model_dump(exclude_none=True)
0158     return dict(block)
0159
0160
0161 def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
0162     handler = TOOL_HANDLERS.get(name)
0163     if handler is None:
0164         return f"Unknown tool: {name}", True
0165     try:
0166         return handler(**args), False

```

```

0167     except Exception as exc:
0168         return f"type(exc).__name__: {exc}", True
0169
0170
0171 def agent_loop(messages: List[dict[str, Any]]) -> List[dict[str, Any]]:
0172     while True:
0173         response = client.messages.create(
0174             model=MODEL,
0175             max_tokens=4096,
0176             system=SYSTEM,
0177             tools=TOOLS,
0178             messages=messages,
0179         )
0180
0181         for block in response.content:
0182             if getattr(block, "type", None) == "text":
0183                 print(block.text)
0184
0185         messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0186         if response.stop_reason != "tool_use":
0187             return messages
0188
0189         results = []
0190         for block in response.content:
0191             if getattr(block, "type", None) != "tool_use":
0192                 continue
0193             print(f"[tool] {block.name} {json.dumps(block.input, ensure_ascii=False)[:300]}")
0194             content, is_error = execute_tool(block.name, dict(block.input))
0195             results.append(
0196                 {
0197                     "type": "tool_result",
0198                     "tool_use_id": block.id,
0199                     "content": content,
0200                     "is_error": is_error,
0201                 }
0202             )
0203         messages.append({"role": "user", "content": results})
0204
0205
0206 def main() -> None:
0207     print(f"Mini Claude Code s02 - workspace: {WORKSPACE}")
0208     messages: List[dict[str, Any]] = []
0209     while True:
0210         try:
0211             prompt = input("\nyou> ").strip()
0212         except (EOFError, KeyboardInterrupt):
0213             print()
0214             break
0215         if not prompt:
0216             continue
0217         if prompt in {" /exit", " /quit"}:
0218             break
0219         messages.append({"role": "user", "content": prompt})
0220         try:
0221             messages = agent_loop(messages)
0222         except Exception as exc:
0223             print(f"ERROR: {exc}")
0224
0225
0226 if __name__ == "__main__":
0227     main()

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (10 lines)

```

0001 # s01 Agent Loop
0002
0003 Run:
0004
0005 ```bash
0006 pip install anthropic python-dotenv
0007 export ANTHROPIC_API_KEY=...
0008 export ANTHROPIC_MODEL=...
0009 python code.py
0010 ```

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

4. s03 - Permission System

把副作用控制从工具实现中剥离出来

观察项	本阶段
阶段目录	s03_permissions
文件数量	2
新增文件	无（主要修改已有文件）
继承/修改文件	README.md, code.py
Python 行数	307

KNOW-WHY：为什么这一层必须存在

- “工具存在”不等于“任何时候都允许执行”。Tool capability 与 Permission policy 应是两个独立层。
- 如果每个敏感动作都让模型自己判断风险，安全策略会变成概率性的；Host 侧 Policy Engine 能提供确定性的 deny/ask/allow。
- 审批不是越多越安全。真正目标是把低风险重复操作自动化，把高影响动作升级为显式批准，把明确危险行为直接拒绝。

KNOW-HOW：这一层具体怎么长出来

- Gate 1: hard deny，命中直接拒绝。
- Gate 2: 规则匹配；读取与已知测试命令自动允许，修改和未知 shell 默认 ask。
- Gate 3: 交互式 allow-once。
- 权限函数返回“决定 + 原因”，让日志和后续工具结果都有可解释性。

数据流 / 控制流

tool_use -> hard deny -> rule match -> optional user approval -> execute -> result

最容易踩的坑

- 把 deny list 当完整安全边界。它只能是防御的一层，不能覆盖未知命令组合。
- 把 permission 逻辑写进每个工具处理器，会导致策略散落、难以审计。
- 在非交互环境复用同一审批函数而不定义 fallback，会导致 CI/worker 卡死；s10 会补上 noninteractive policy。

本阶段验收清单

- 只读工具应直接执行。
- 文件写入应请求批准。
- 非 allowlist shell 应请求批准，hard deny 模式应直接拒绝。

为什么 Permission 和 Tool Schema 不能合并

Tool Schema 回答“能力是什么、参数长什么样”；Permission 回答“在当前上下文、当前用户、当前副作用等级下是否允许调用”。同一个 write_file 工具可以在只读审查会话中被 deny，在交互式开发中 ask，在隔离 worktree worker 中 allow。把这两层解耦后，策略可以独立演化。

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (307 lines)

```
0001 #!/usr/bin/env python3
0002 """s03 - Add a three-gate permission system before tool execution."""
0003 from __future__ import annotations
0004
0005 import glob as globlib
0006 import json
0007 import os
0008 import subprocess
0009 import shlex
0010 from enum import Enum
0011 from pathlib import Path
0012 from typing import Any, Callable
0013
0014 from anthropic import Anthropic
0015 from dotenv import load_dotenv
0016
0017 load_dotenv()
0018 MODEL = os.environ.get("ANTHROPIC_MODEL")
0019 if not MODEL:
0020     raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")
0021
0022 client = Anthropic()
0023 WORKSPACE = Path.cwd().resolve()
0024 MAX_TOOL_OUTPUT = 12000
0025
0026
0027 def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
0028     return {
0029         "name": name,
0030         "description": description,
0031         "input_schema": {
0032             "type": "object",
0033             "properties": properties,
0034             "required": required,
0035             "additionalProperties": False,
0036         },
0037     }
0038
0039
0040 TOOLS = [
0041     tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
0042     tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
0043     tool(
0044         "write_file",
0045         "Create or replace a UTF-8 text file inside the workspace.",
0046         {"path": {"type": "string"}, "content": {"type": "string"}},
0047         ["path", "content"],
0048     ),
0049     tool(
0050         "edit_file",
0051         "Replace one exact text occurrence in a workspace file.",
0052         {
0053             "path": {"type": "string"},
0054             "old_text": {"type": "string"},
0055             "new_text": {"type": "string"},
0056         },
0057         ["path", "old_text", "new_text"],
0058     ),
0059     tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
0060 ]
0061
0062 SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
0063 Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
0064 Make focused changes and verify them before you finish.
0065 """
0066
0067
0068 def safe_path(raw: str) -> Path:
0069     """Resolve a path and reject escapes outside the workspace."""
0070     candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
0071     try:
```

```

0072     candidate.relative_to(WORKSPACE)
0073 except ValueError as exc:
0074     raise ValueError(f"Path escapes workspace: {raw!r}") from exc
0075 return candidate
0076
0077
0078 def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
0079     if len(text) <= limit:
0080         return text
0081     half = limit // 2
0082     return text[:half] + "\n...<truncated>...\n" + text[-half:]
0083
0084
0085 def run_bash(command: str) -> str:
0086     try:
0087         proc = subprocess.run(
0088             command,
0089             shell=True,
0090             text=True,
0091             capture_output=True,
0092             cwd=WORKSPACE,
0093             timeout=30,
0094         )
0095     except subprocess.TimeoutExpired:
0096         return "ERROR: command timed out after 30 seconds"
0097     return json.dumps(
0098         {
0099             "exit_code": proc.returncode,
0100             "stdout": truncate(proc.stdout),
0101             "stderr": truncate(proc.stderr),
0102         },
0103         ensure_ascii=False,
0104     )
0105
0106
0107 def run_read(path: str) -> str:
0108     p = safe_path(path)
0109     if not p.is_file():
0110         return f"ERROR: file not found: {path}"
0111     return truncate(p.read_text(encoding="utf-8", errors="replace"))
0112
0113
0114 def run_write(path: str, content: str) -> str:
0115     p = safe_path(path)
0116     p.parent.mkdir(parents=True, exist_ok=True)
0117     p.write_text(content, encoding="utf-8")
0118     return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"
0119
0120
0121 def run_edit(path: str, old_text: str, new_text: str) -> str:
0122     p = safe_path(path)
0123     if not p.is_file():
0124         return f"ERROR: file not found: {path}"
0125     text = p.read_text(encoding="utf-8", errors="replace")
0126     count = text.count(old_text)
0127     if count == 0:
0128         return "ERROR: old_text not found"
0129     if count > 1:
0130         return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
0131     p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
0132     return f"OK: edited {p.relative_to(WORKSPACE)}"
0133
0134
0135 def run_glob(pattern: str) -> str:
0136     # Resolve matches then apply safe_path to each result to keep output inside the workspace.
0137     raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
0138     matches: list[str] = []
0139     for raw in raw_matches[:500]:
0140         try:
0141             p = safe_path(raw)
0142         except ValueError:
0143             continue
0144         matches.append(str(p.relative_to(WORKSPACE)))
0145     return "\n".join(sorted(matches)) if matches else "(no matches)"
0146
0147
0148
0149
0150 class Decision(str, Enum):
0151     ALLOW = "allow"

```

```

0152     ASK = "ask"
0153     DENY = "deny"
0154
0155
0156 # Gate 1: patterns that should never be auto-approved by this tutorial runtime.
0157 HARD_DENY_SUBSTRINGS = (
0158     "sudo ",
0159     "chmod -R 777 /",
0160     "> /dev/sd",
0161     "mkfs.",
0162 )
0163
0164 # Gate 2: commands considered low-risk and repeatable in a coding workspace.
0165 SAFE_COMMAND_PREFIXES = (
0166     "pwd", "ls", "find ", "git status", "git diff", "git log",
0167     "python -m pytest", "pytest", "npm test", "npm run test",
0168     "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
0169 )
0170
0171
0172 def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
0173     """Return a policy decision and human-readable reason.
0174
0175     Order matters: hard deny -> explicit rules -> ask fallback.
0176     """
0177     if name in {"read_file", "glob"}:
0178         return Decision.ALLOW, "read-only workspace operation"
0179
0180     if name in {"write_file", "edit_file"}:
0181         try:
0182             safe_path(args.get("path", ""))
0183         except Exception as exc:
0184             return Decision.DENY, str(exc)
0185         return Decision.ASK, "file modification inside workspace"
0186
0187     if name == "bash":
0188         command = str(args.get("command", "")).strip()
0189         lowered = command.lower()
0190         if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
0191             return Decision.DENY, "matches hard deny policy"
0192         if command.startswith(SAFE_COMMAND_PREFIXES):
0193             return Decision.ALLOW, "matches low-risk command allowlist"
0194         return Decision.ASK, "shell command is not allowlisted"
0195
0196     return Decision.ASK, "unknown or extensible tool"
0197
0198
0199 def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
0200     print(f"\n[permission] {name}: {reason}")
0201     print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
0202     while True:
0203         answer = input("Allow once? [y/N] ").strip().lower()
0204         if answer in {"y", "yes"}:
0205             return True
0206         if answer in {"", "n", "no"}:
0207             return False
0208
0209
0210 def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
0211     decision, reason = classify_permission(name, args)
0212     if decision is Decision.DENY:
0213         return False, f"DENIED: {reason}"
0214     if decision is Decision.ALLOW:
0215         return True, reason
0216     if request_approval(name, args, reason):
0217         return True, "approved by user"
0218     return False, "DENIED: user rejected tool call"
0219
0220
0221 TOOL_HANDLERS: dict[str, Callable[..., str]] = {
0222     "bash": run_bash,
0223     "read_file": run_read,
0224     "write_file": run_write,
0225     "edit_file": run_edit,
0226     "glob": run_glob,
0227 }
0228
0229
0230 def block_to_dict(block: Any) -> dict[str, Any]:
0231     if hasattr(block, "model_dump"):

```



```

0232     return block.model_dump(exclude_none=True)
0233     return dict(block)
0234
0235
0236 def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
0237     handler = TOOL_HANDLERS.get(name)
0238     if handler is None:
0239         return f"Unknown tool: {name}", True
0240     try:
0241         return handler(**args), False
0242     except Exception as exc:
0243         return f"{type(exc).__name__}: {exc}", True
0244
0245
0246 def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
0247     while True:
0248         response = client.messages.create(
0249             model=MODEL,
0250             max_tokens=4096,
0251             system=SYSTEM,
0252             tools=TOOLS,
0253             messages=messages,
0254         )
0255
0256         for block in response.content:
0257             if getattr(block, "type", None) == "text":
0258                 print(block.text)
0259
0260         messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0261         if response.stop_reason != "tool_use":
0262             return messages
0263
0264         results = []
0265         for block in response.content:
0266             if getattr(block, "type", None) != "tool_use":
0267                 continue
0268             args = dict(block.input)
0269             print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
0270             allowed, permission_reason = check_permission(block.name, args)
0271             if allowed:
0272                 content, is_error = execute_tool(block.name, args)
0273             else:
0274                 content, is_error = permission_reason, True
0275             results.append(
0276                 {
0277                     "type": "tool_result",
0278                     "tool_use_id": block.id,
0279                     "content": content,
0280                     "is_error": is_error,
0281                 }
0282             )
0283         messages.append({"role": "user", "content": results})
0284
0285
0286 def main() -> None:
0287     print(f"Mini Claude Code s03 - workspace: {WORKSPACE}")
0288     messages: list[dict[str, Any]] = []
0289     while True:
0290         try:
0291             prompt = input("\nyou> ").strip()
0292         except (EOFError, KeyboardInterrupt):
0293             print()
0294             break
0295         if not prompt:
0296             continue
0297         if prompt in {"exit", "/quit"}:
0298             break
0299         messages.append({"role": "user", "content": prompt})
0300         try:
0301             messages = agent_loop(messages)
0302         except Exception as exc:
0303             print(f"ERROR: {exc}")
0304
0305
0306 if __name__ == "__main__":
0307     main()

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (8 Lines)

```
0001 # s03 Permissions
0002
0003 Adds three gates before execution:
0004 1. hard deny
0005 2. rule matching
0006 3. user approval
0007
0008 Run the same way as s02.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

5. s04 - Context Management

管理有限上下文：统计、清理与压缩

观察项	本阶段
阶段目录	s04_context
文件数量	3
新增文件	context_manager.py
继承/修改文件	README.md, code.py
Python 行数	409

KNOW-WHY：为什么这一层必须存在

- Agent 的主要稀缺资源不是“消息条数”，而是每次推理可见的有限上下文。文件内容、工具输出、历史讨论都争夺同一个窗口。
- 当上下文越来越长，模型并不会自动知道哪些旧细节可以丢弃。需要 Runtime 主动清理或压缩低价值历史。
- Compaction 的目标不是“把所有内容缩短”，而是把原始轨迹转成可继续执行的高价值状态。

KNOW-HOW：这一层具体怎么长出来

- 13. 使用字符数/4 做教学版 approximate token 估算，并提供 /context。
- 14. 当超过预算时，保留最近若干消息，把更早历史交给独立 summarizer 压缩。
- 15. 摘要只保留目标、约束、发现、修改、测试、未解决问题；然后作为 [COMPACTED_CONTEXT] 重新进入历史。
- 16. 提供 /clear 和 /compact，让用户可以主动管理上下文。

数据流 / 控制流

history -> estimate -> (over budget?) -> summarize old + keep recent -> next model call

最容易踩的坑

- 摘要模型如果被要求“总结一切”，容易把大量无关日志也保留下来。
- 把 persistent instruction 只放在对话里，压缩后可能丢失；这正是 s05 引入 CLAUDE.md/Memory 的原因。
- 字符/4 不是精确 tokenizer；它只是一个触发阈值教学实现。

本阶段验收清单

- 降低 MINICLAUDE_CONTEXT_BUDGET，制造自动 compaction。
- 执行 /compact，确认最近消息仍保留。
- 执行 /clear，确认历史归零。

与 Claude Code 公开机制的对应

Claude Code 当前公开文档说明：上下文接近上限时会先清理较老工具输出，再在需要时压缩对话；/context 可以查看占用，/compact 可主动压缩。[R3] 本教程不复制其内部算法，而是用“预算触发 + 结构化摘要 + 保留最近消息”复现同一类工程问题。

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (350 lines)

```
0001 #!/usr/bin/env python3
0002 """s04 - Add context inspection and model-assisted compaction."""
0003 from __future__ import annotations
0004
0005 import glob as globlib
0006 import json
0007 import os
0008 import subprocess
0009 import shlex
0010 from enum import Enum
0011 from pathlib import Path
0012 from typing import Any, Callable
0013
0014 from anthropic import Anthropic
0015 from dotenv import load_dotenv
0016 from context_manager import ContextManager
0017
0018 load_dotenv()
0019 MODEL = os.environ.get("ANTHROPIC_MODEL")
0020 if not MODEL:
0021     raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")
0022
0023 client = Anthropic()
0024 WORKSPACE = Path.cwd().resolve()
0025 MAX_TOOL_OUTPUT = 12000
0026 CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
0027
0028
0029 def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
0030     return {
0031         "name": name,
0032         "description": description,
0033         "input_schema": {
0034             "type": "object",
0035             "properties": properties,
0036             "required": required,
0037             "additionalProperties": False,
0038         },
0039     }
0040
0041
0042 TOOLS = [
0043     tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
0044     tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
0045     tool(
0046         "write_file",
0047         "Create or replace a UTF-8 text file inside the workspace.",
0048         {"path": {"type": "string"}, "content": {"type": "string"}},
0049         ["path", "content"],
0050     ),
0051     tool(
0052         "edit_file",
0053         "Replace one exact text occurrence in a workspace file.",
0054         {
0055             "path": {"type": "string"},
0056             "old_text": {"type": "string"},
0057             "new_text": {"type": "string"},
0058         },
0059         ["path", "old_text", "new_text"],
0060     ),
0061     tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
0062 ]
0063
0064 SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
0065 Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
0066 Make focused changes and verify them before you finish.
0067 """
0068
0069
0070 def safe_path(raw: str) -> Path:
0071     """Resolve a path and reject escapes outside the workspace."""
```

```

0072 candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
0073 try:
0074     candidate.relative_to(WORKSPACE)
0075 except ValueError as exc:
0076     raise ValueError(f"Path escapes workspace: {raw!r}") from exc
0077 return candidate
0078
0079
0080 def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
0081     if len(text) <= limit:
0082         return text
0083     half = limit // 2
0084     return text[:half] + "\n...<truncated>...\n" + text[-half:]
0085
0086
0087 def run_bash(command: str) -> str:
0088     try:
0089         proc = subprocess.run(
0090             command,
0091             shell=True,
0092             text=True,
0093             capture_output=True,
0094             cwd=WORKSPACE,
0095             timeout=30,
0096         )
0097     except subprocess.TimeoutExpired:
0098         return "ERROR: command timed out after 30 seconds"
0099     return json.dumps(
0100         {
0101             "exit_code": proc.returncode,
0102             "stdout": truncate(proc.stdout),
0103             "stderr": truncate(proc.stderr),
0104         },
0105         ensure_ascii=False,
0106     )
0107
0108
0109 def run_read(path: str) -> str:
0110     p = safe_path(path)
0111     if not p.is_file():
0112         return f"ERROR: file not found: {path}"
0113     return truncate(p.read_text(encoding="utf-8", errors="replace"))
0114
0115
0116 def run_write(path: str, content: str) -> str:
0117     p = safe_path(path)
0118     p.parent.mkdir(parents=True, exist_ok=True)
0119     p.write_text(content, encoding="utf-8")
0120     return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"
0121
0122
0123 def run_edit(path: str, old_text: str, new_text: str) -> str:
0124     p = safe_path(path)
0125     if not p.is_file():
0126         return f"ERROR: file not found: {path}"
0127     text = p.read_text(encoding="utf-8", errors="replace")
0128     count = text.count(old_text)
0129     if count == 0:
0130         return "ERROR: old_text not found"
0131     if count > 1:
0132         return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
0133     p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
0134     return f"OK: edited {p.relative_to(WORKSPACE)}"
0135
0136
0137 def run_glob(pattern: str) -> str:
0138     # Resolve matches then apply safe_path to each result to keep output inside the workspace.
0139     raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
0140     matches: list[str] = []
0141     for raw in raw_matches[:500]:
0142         try:
0143             p = safe_path(raw)
0144         except ValueError:
0145             continue
0146         matches.append(str(p.relative_to(WORKSPACE)))
0147     return "\n".join(sorted(matches)) if matches else "(no matches)"
0148
0149
0150
0151

```

```

0152 class Decision(str, Enum):
0153     ALLOW = "allow"
0154     ASK = "ask"
0155     DENY = "deny"
0156
0157
0158 # Gate 1: patterns that should never be auto-approved by this tutorial runtime.
0159 HARD_DENY_SUBSTRINGS = (
0160     "sudo ",
0161     "chmod -R 777 /",
0162     "> /dev/sd",
0163     "mkfs.",
0164 )
0165
0166 # Gate 2: commands considered low-risk and repeatable in a coding workspace.
0167 SAFE_COMMAND_PREFIXES = (
0168     "pwd", "ls", "find ", "git status", "git diff", "git log",
0169     "python -m pytest", "pytest", "npm test", "npm run test",
0170     "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
0171 )
0172
0173
0174 def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
0175     """Return a policy decision and human-readable reason.
0176
0177     Order matters: hard deny -> explicit rules -> ask fallback.
0178     """
0179     if name in {"read_file", "glob":
0180         return Decision.ALLOW, "read-only workspace operation"
0181
0182     if name in {"write_file", "edit_file":
0183         try:
0184             safe_path(args.get("path", ""))
0185         except Exception as exc:
0186             return Decision.DENY, str(exc)
0187         return Decision.ASK, "file modification inside workspace"
0188
0189     if name == "bash":
0190         command = str(args.get("command", "")).strip()
0191         lowered = command.lower()
0192         if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
0193             return Decision.DENY, "matches hard deny policy"
0194         if command.startswith(SAFE_COMMAND_PREFIXES):
0195             return Decision.ALLOW, "matches low-risk command allowlist"
0196         return Decision.ASK, "shell command is not allowlisted"
0197
0198     return Decision.ASK, "unknown or extensible tool"
0199
0200
0201 def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
0202     print(f"\n[permission] {name}: {reason}")
0203     print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
0204     while True:
0205         answer = input("Allow once? [y/N] ").strip().lower()
0206         if answer in {"y", "yes":
0207             return True
0208         if answer in {"", "n", "no":
0209             return False
0210
0211
0212 def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
0213     decision, reason = classify_permission(name, args)
0214     if decision is Decision.DENY:
0215         return False, f"DENIED: {reason}"
0216     if decision is Decision.ALLOW:
0217         return True, reason
0218     if request_approval(name, args, reason):
0219         return True, "approved by user"
0220     return False, "DENIED: user rejected tool call"
0221
0222
0223 TOOL_HANDLERS: dict[str, Callable[..., str]] = {
0224     "bash": run_bash,
0225     "read_file": run_read,
0226     "write_file": run_write,
0227     "edit_file": run_edit,
0228     "glob": run_glob,
0229 }
0230
0231

```

```

0232 def block_to_dict(block: Any) -> dict[str, Any]:
0233     if hasattr(block, "model_dump"):
0234         return block.model_dump(exclude_none=True)
0235     return dict(block)
0236
0237
0238 def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
0239     handler = TOOL_HANDLERS.get(name)
0240     if handler is None:
0241         return f"Unknown tool: {name}", True
0242     try:
0243         return handler(**args), False
0244     except Exception as exc:
0245         return f"{type(exc).__name__}: {exc}", True
0246
0247
0248
0249
0250 def summarize_history(old_messages: list[dict[str, Any]]) -> str:
0251     """Ask the model to compress old state without tools."""
0252     prompt = """Summarize this coding-agent conversation for continuation.
0253 Preserve only durable state:
0254 - user goal and constraints
0255 - repository facts discovered
0256 - files changed and why
0257 - commands/tests run and outcomes
0258 - unresolved hypotheses, TODOs, and errors
0259 - important approvals or denied operations
0260 Do not invent facts. Be concise but concrete.
0261
0262 HISTORY JSON:
0263 """ + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
0264     response = client.messages.create(
0265         model=MODEL,
0266         max_tokens=1800,
0267         system="You compress coding-agent state for a future model turn.",
0268         messages=[{"role": "user", "content": prompt}],
0269     )
0270     return "\n".join(
0271         block.text for block in response.content if getattr(block, "type", None) == "text"
0272     )
0273
0274
0275 def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
0276     while True:
0277         if CONTEXT.needs_compaction(messages):
0278             print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
0279             messages = CONTEXT.compact(messages, summarize_history)
0280         response = client.messages.create(
0281             model=MODEL,
0282             max_tokens=4096,
0283             system=SYSTEM,
0284             tools=TOOLS,
0285             messages=messages,
0286         )
0287
0288         for block in response.content:
0289             if getattr(block, "type", None) == "text":
0290                 print(block.text)
0291
0292         messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0293         if response.stop_reason != "tool_use":
0294             return messages
0295
0296         results = []
0297         for block in response.content:
0298             if getattr(block, "type", None) != "tool_use":
0299                 continue
0300             args = dict(block.input)
0301             print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
0302             allowed, permission_reason = check_permission(block.name, args)
0303             if allowed:
0304                 content, is_error = execute_tool(block.name, args)
0305             else:
0306                 content, is_error = permission_reason, True
0307             results.append(
0308                 {
0309                     "type": "tool_result",
0310                     "tool_use_id": block.id,
0311                     "content": content,

```

```

0312         "is_error": is_error,
0313     }
0314 )
0315     messages.append({"role": "user", "content": results})
0316
0317
0318 def main() -> None:
0319     print(f"Mini Claude Code s04 - workspace: {WORKSPACE}")
0320     messages: List[dict[str, Any]] = []
0321     while True:
0322         try:
0323             prompt = input("\nyou> ").strip()
0324         except (EOFError, KeyboardInterrupt):
0325             print()
0326             break
0327         if not prompt:
0328             continue
0329         if prompt in {"/exit", "/quit"}:
0330             break
0331         if prompt == "/context":
0332             print(CONTEXT.report(messages))
0333             continue
0334         if prompt == "/compact":
0335             messages = CONTEXT.compact(messages, summarize_history)
0336             print("[context] " + CONTEXT.report(messages))
0337             continue
0338         if prompt == "/clear":
0339             messages = []
0340             print("[context] cleared")
0341             continue
0342         messages.append({"role": "user", "content": prompt})
0343         try:
0344             messages = agent_loop(messages)
0345         except Exception as exc:
0346             print(f"ERROR: {exc}")
0347
0348
0349 if __name__ == "__main__":
0350     main()

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (3 Lines)

```

0001 # s04 Context Management
0002
0003 Adds `/context`, `/compact`, `/clear`, a rough token budget, and model-assisted compaction.

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 Lines)

```

0001 """A small context manager: estimate, inspect, and compact message history."""
0002 from __future__ import annotations
0003
0004 import json
0005 from dataclasses import dataclass
0006 from typing import Any, Callable
0007
0008
0009 @dataclass
0010 class ContextStats:
0011     message_count: int
0012     approx_tokens: int
0013     chars: int
0014
0015
0016 class ContextManager:
0017     def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
0018         self.max_approx_tokens = max_approx_tokens
0019         self.keep_recent_messages = keep_recent_messages
0020
0021     @staticmethod
0022     def estimate(messages: List[dict[str, Any]]) -> ContextStats:
0023         raw = json.dumps(messages, ensure_ascii=False, default=str)
0024         # Deliberately simple heuristic. Production systems should use model-aware counting.
0025         return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))
0026
0027     def needs_compaction(self, messages: List[dict[str, Any]]) -> bool:
0028         return self.estimate(messages).approx_tokens >= self.max_approx_tokens
0029
0030     def report(self, messages: List[dict[str, Any]]) -> str:

```



```

0031     stats = self.estimate(messages)
0032     pct = 100 * stats.approx_tokens / self.max_approx_tokens
0033     return (
0034         f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:}, ",
0035         f"budget={self.max_approx_tokens:}, {pct:.1f}%"
0036     )
0037
0038     def compact(
0039         self,
0040         messages: list[dict[str, Any]],
0041         summarizer: Callable[[list[dict[str, Any]]], str],
0042     ) -> list[dict[str, Any]]:
0043         if len(messages) <= self.keep_recent_messages + 2:
0044             return messages
0045
0046         cut = len(messages) - self.keep_recent_messages
0047         old = messages[:cut]
0048         recent = messages[cut:]
0049         summary = summarizer(old)
0050         compacted = {
0051             "role": "user",
0052             "content": (
0053                 "[COMPACTED_CONTEXT]\n"
0054                 "The following is a structured summary of earlier conversation state. "
0055                 "Treat it as historical context, not a new user request.\n\n"
0056                 + summary
0057             ),
0058         }
0059         return [compacted, *recent]

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

6. s05 - Memory

把跨会话知识拆成显式规则与 Agent 记忆

观察项	本阶段
阶段目录	s05_memory
文件数量	4
新增文件	memory.py
继承/修改文件	README.md, code.py, context_manager.py
Python 行数	486

KNOW-WHY：为什么这一层必须存在

- Context 是短期工作记忆，Memory 是跨会话状态。把两者混在一起会让每个新会话重新解释项目约束。
- 持久信息也应分两类：人类显式规定的“应该怎么做”，以及 Agent 工作中积累的“之前学到了什么”。
- Memory 必须可审计、可编辑、可删除，因此这里选用纯 Markdown 文件而不是隐藏数据库。

KNOW-HOW：这一层具体怎么长出来

- 17. 读取项目根 CLAUDE.md 作为 project instructions。
- 18. 把 Agent 自己的持久笔记写到 `.mini_claude/memory/MEMORY.md``，启动时只加载受限前缀。
- 19. 加入 remember 和 recall_memory 工具；系统提示要求只记录真正跨会话有用的事实。
- 20. build_system 每一轮重新组装当前 Memory，使文件修改后不必重启进程。

数据流 / 控制流

CLAUDE.md + MEMORY.md -> build_system -> every model turn; remember -> durable file

最容易踩的坑

- 把每个临时 TODO 都写入 Memory，会形成“记忆垃圾场”。
- 把必须严格执行的安全规则只放 Memory；Memory 是上下文，不是 enforcement。
- 教学版 Memory 存在项目目录中，而正式产品可能采用更复杂的 per-repo/user scope；不要把目录结构当成官方源码实现。

本阶段验收清单

- 建立 CLAUDE.md 并询问项目规则。
- 让 Agent remember 一条构建事实，重启后确认仍可 recall。
- 重复 remember 同一条，确认去重。

Context / CLAUDE.md / Auto Memory 的职责分离

机制	谁写	生命周期	适合内容
Context	对话与工具	当前会话	任务状态/证据
CLAUDE.md	人	跨会话	不变量/约束
Memory	Agent	跨会话	发现/偏好/经验

Claude Code 官方公开文档同样区分 CLAUDE.md 与 auto memory，并强调二者都属于 context，而不是强制策略。

[R4]

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (368 lines)

```
0001 #!/usr/bin/env python3
0002 """s05 - Add project instructions and persistent agent memory."""
0003 from __future__ import annotations
0004
0005 import glob as globlib
0006 import json
0007 import os
0008 import subprocess
0009 import shlex
0010 from enum import Enum
0011 from pathlib import Path
0012 from typing import Any, Callable
0013
0014 from anthropic import Anthropic
0015 from dotenv import load_dotenv
0016 from context_manager import ContextManager
0017 from memory import MemoryStore
0018
0019 load_dotenv()
0020 MODEL = os.environ.get("ANTHROPIC_MODEL")
0021 if not MODEL:
0022     raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")
0023
0024 client = Anthropic()
0025 WORKSPACE = Path.cwd().resolve()
0026 MAX_TOOL_OUTPUT = 12000
0027 CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
0028 MEMORY = MemoryStore(WORKSPACE)
0029
0030
0031 def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
0032     return {
0033         "name": name,
0034         "description": description,
0035         "input_schema": {
0036             "type": "object",
0037             "properties": properties,
0038             "required": required,
0039             "additionalProperties": False,
0040         },
0041     }
0042
0043
0044 TOOLS = [
0045     tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
0046     tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
0047     tool(
0048         "write_file",
0049         "Create or replace a UTF-8 text file inside the workspace.",
0050         {"path": {"type": "string"}, "content": {"type": "string"}},
0051         ["path", "content"],
0052     ),
0053     tool(
0054         "edit_file",
0055         "Replace one exact text occurrence in a workspace file.",
0056         {
0057             "path": {"type": "string"},
0058             "old_text": {"type": "string"},
0059             "new_text": {"type": "string"},
0060         },
0061         ["path", "old_text", "new_text"],
0062     ),
0063     tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
0064     tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}}, ["note"]),
0065     tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}}, ["query"]),
0066 ]
```

```

0067
0068 BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
0069 Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
0070 Make focused changes and verify them before you finish.
0071 Use remember only for durable facts likely to help future sessions, not transient task state.
0072 """
0073
0074
0075 def build_system() -> str:
0076     startup = MEMORY.startup_context()
0077     return BASE_SYSTEM + ("\n\n" + startup if startup else "")
0078
0079
0080 def safe_path(raw: str) -> Path:
0081     """Resolve a path and reject escapes outside the workspace."""
0082     candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
0083     try:
0084         candidate.relative_to(WORKSPACE)
0085     except ValueError as exc:
0086         raise ValueError(f"Path escapes workspace: {raw!r}") from exc
0087     return candidate
0088
0089
0090 def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
0091     if len(text) <= limit:
0092         return text
0093     half = limit // 2
0094     return text[:half] + "\n...<truncated>...\n" + text[-half:]
0095
0096
0097 def run_bash(command: str) -> str:
0098     try:
0099         proc = subprocess.run(
0100             command,
0101             shell=True,
0102             text=True,
0103             capture_output=True,
0104             cwd=WORKSPACE,
0105             timeout=30,
0106         )
0107     except subprocess.TimeoutExpired:
0108         return "ERROR: command timed out after 30 seconds"
0109     return json.dumps(
0110         {
0111             "exit_code": proc.returncode,
0112             "stdout": truncate(proc.stdout),
0113             "stderr": truncate(proc.stderr),
0114         },
0115         ensure_ascii=False,
0116     )
0117
0118
0119 def run_read(path: str) -> str:
0120     p = safe_path(path)
0121     if not p.is_file():
0122         return f"ERROR: file not found: {path}"
0123     return truncate(p.read_text(encoding="utf-8", errors="replace"))
0124
0125
0126 def run_write(path: str, content: str) -> str:
0127     p = safe_path(path)
0128     p.parent.mkdir(parents=True, exist_ok=True)
0129     p.write_text(content, encoding="utf-8")
0130     return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"
0131
0132
0133 def run_edit(path: str, old_text: str, new_text: str) -> str:
0134     p = safe_path(path)
0135     if not p.is_file():
0136         return f"ERROR: file not found: {path}"
0137     text = p.read_text(encoding="utf-8", errors="replace")
0138     count = text.count(old_text)
0139     if count == 0:
0140         return "ERROR: old_text not found"
0141     if count > 1:
0142         return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
0143     p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
0144     return f"OK: edited {p.relative_to(WORKSPACE)}"
0145
0146

```

```

0147 def run_glob(pattern: str) -> str:
0148     # Resolve matches then apply safe_path to each result to keep output inside the workspace.
0149     raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
0150     matches: list[str] = []
0151     for raw in raw_matches[:500]:
0152         try:
0153             p = safe_path(raw)
0154             except ValueError:
0155                 continue
0156             matches.append(str(p.relative_to(WORKSPACE)))
0157     return "\n".join(sorted(matches)) if matches else "(no matches)"
0158
0159
0160
0161
0162 class Decision(str, Enum):
0163     ALLOW = "allow"
0164     ASK = "ask"
0165     DENY = "deny"
0166
0167
0168 # Gate 1: patterns that should never be auto-approved by this tutorial runtime.
0169 HARD_DENY_SUBSTRINGS = (
0170     "sudo ",
0171     "chmod -R 777 /",
0172     "> /dev/sd",
0173     "mkfs.",
0174 )
0175
0176 # Gate 2: commands considered low-risk and repeatable in a coding workspace.
0177 SAFE_COMMAND_PREFIXES = (
0178     "pwd", "ls", "find ", "git status", "git diff", "git log",
0179     "python -m pytest", "pytest", "npm test", "npm run test",
0180     "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
0181 )
0182
0183
0184 def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
0185     """Return a policy decision and human-readable reason.
0186
0187     Order matters: hard deny -> explicit rules -> ask fallback.
0188     """
0189     if name in {"read_file", "glob", "recall_memory"}:
0190         return Decision.ALLOW, "read-only workspace operation"
0191
0192     if name == "remember":
0193         return Decision.ALLOW, "writes only to the agent memory store"
0194
0195     if name in {"write_file", "edit_file"}:
0196         try:
0197             safe_path(args.get("path", ""))
0198             except Exception as exc:
0199                 return Decision.DENY, str(exc)
0200             return Decision.ASK, "file modification inside workspace"
0201
0202     if name == "bash":
0203         command = str(args.get("command", "")).strip()
0204         lowered = command.lower()
0205         if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
0206             return Decision.DENY, "matches hard deny policy"
0207         if command.startswith(SAFE_COMMAND_PREFIXES):
0208             return Decision.ALLOW, "matches low-risk command allowlist"
0209         return Decision.ASK, "shell command is not allowlisted"
0210
0211     return Decision.ASK, "unknown or extensible tool"
0212
0213
0214 def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
0215     print(f"\n[permission] {name}: {reason}")
0216     print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
0217     while True:
0218         answer = input("Allow once? [y/N] ").strip().lower()
0219         if answer in {"y", "yes"}:
0220             return True
0221         if answer in {"", "n", "no"}:
0222             return False
0223
0224
0225 def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
0226     decision, reason = classify_permission(name, args)

```

```

0227     if decision is Decision.DENY:
0228         return False, f"DENIED: {reason}"
0229     if decision is Decision.ALLOW:
0230         return True, reason
0231     if request_approval(name, args, reason):
0232         return True, "approved by user"
0233     return False, "DENIED: user rejected tool call"
0234
0235
0236 TOOL_HANDLERS: dict[str, Callable[..., str]] = {
0237     "bash": run_bash,
0238     "read_file": run_read,
0239     "write_file": run_write,
0240     "edit_file": run_edit,
0241     "glob": run_glob,
0242     "remember": MEMORY.remember,
0243     "recall_memory": MEMORY.search,
0244 }
0245
0246
0247 def block_to_dict(block: Any) -> dict[str, Any]:
0248     if hasattr(block, "model_dump"):
0249         return block.model_dump(exclude_none=True)
0250     return dict(block)
0251
0252
0253 def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
0254     handler = TOOL_HANDLERS.get(name)
0255     if handler is None:
0256         return f"Unknown tool: {name}", True
0257     try:
0258         return handler(**args), False
0259     except Exception as exc:
0260         return f"type(exc).__name__: {exc}", True
0261
0262
0263
0264
0265 def summarize_history(old_messages: list[dict[str, Any]]) -> str:
0266     """Ask the model to compress old state without tools."""
0267     prompt = """Summarize this coding-agent conversation for continuation.
0268 Preserve only durable state:
0269 - user goal and constraints
0270 - repository facts discovered
0271 - files changed and why
0272 - commands/tests run and outcomes
0273 - unresolved hypotheses, TODOs, and errors
0274 - important approvals or denied operations
0275 Do not invent facts. Be concise but concrete.
0276
0277 HISTORY JSON:
0278 """ + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
0279     response = client.messages.create(
0280         model=MODEL,
0281         max_tokens=1800,
0282         system="You compress coding-agent state for a future model turn.",
0283         messages=[{"role": "user", "content": prompt}],
0284     )
0285     return "\n".join(
0286         block.text for block in response.content if getattr(block, "type", None) == "text"
0287     )
0288
0289
0290 def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
0291     while True:
0292         if CONTEXT.needs_compaction(messages):
0293             print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
0294             messages = CONTEXT.compact(messages, summarize_history)
0295             response = client.messages.create(
0296                 model=MODEL,
0297                 max_tokens=4096,
0298                 system=build_system(),
0299                 tools=TOOLS,
0300                 messages=messages,
0301             )
0302
0303             for block in response.content:
0304                 if getattr(block, "type", None) == "text":
0305                     print(block.text)
0306

```

```

0307     messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0308     if response.stop_reason != "tool_use":
0309         return messages
0310
0311     results = []
0312     for block in response.content:
0313         if getattr(block, "type", None) != "tool_use":
0314             continue
0315         args = dict(block.input)
0316         print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
0317         allowed, permission_reason = check_permission(block.name, args)
0318         if allowed:
0319             content, is_error = execute_tool(block.name, args)
0320         else:
0321             content, is_error = permission_reason, True
0322         results.append(
0323             {
0324                 "type": "tool_result",
0325                 "tool_use_id": block.id,
0326                 "content": content,
0327                 "is_error": is_error,
0328             }
0329         )
0330     messages.append({"role": "user", "content": results})
0331
0332
0333 def main() -> None:
0334     print(f"Mini Claude Code s05 - workspace: {WORKSPACE}")
0335     messages: List[dict[str, Any]] = []
0336     while True:
0337         try:
0338             prompt = input("\nyou> ").strip()
0339         except (EOFError, KeyboardInterrupt):
0340             print()
0341             break
0342         if not prompt:
0343             continue
0344         if prompt in {"exit", "/quit"}:
0345             break
0346         if prompt == "/context":
0347             print(CONTEXT.report(messages))
0348             continue
0349         if prompt == "/compact":
0350             messages = CONTEXT.compact(messages, summarize_history)
0351             print("[context] " + CONTEXT.report(messages))
0352             continue
0353         if prompt == "/clear":
0354             messages = []
0355             print("[context] cleared")
0356             continue
0357         if prompt == "/memory":
0358             print(MEMORY.load_auto_memory() or "(memory is empty)")
0359             continue
0360         messages.append({"role": "user", "content": prompt})
0361         try:
0362             messages = agent_loop(messages)
0363         except Exception as exc:
0364             print(f"ERROR: {exc}")
0365
0366
0367 if __name__ == "__main__":
0368     main()

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (7 lines)

```

0001 # s05 Memory
0002
0003 Adds:
0004 - project `CLAUDE.md` loading
0005 - `.mini_claude/memory/MEMORY.md`
0006 - `remember` and `recall_memory` tools
0007 - `/memory`

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```

0001 """A small context manager: estimate, inspect, and compact message history."""
0002 from __future__ import annotations
0003

```

```

0004 import json
0005 from dataclasses import dataclass
0006 from typing import Any, Callable
0007
0008
0009 @dataclass
0010 class ContextStats:
0011     message_count: int
0012     approx_tokens: int
0013     chars: int
0014
0015
0016 class ContextManager:
0017     def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
0018         self.max_approx_tokens = max_approx_tokens
0019         self.keep_recent_messages = keep_recent_messages
0020
0021     @staticmethod
0022     def estimate(messages: list[dict[str, Any]]) -> ContextStats:
0023         raw = json.dumps(messages, ensure_ascii=False, default=str)
0024         # Deliberately simple heuristic. Production systems should use model-aware counting.
0025         return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))
0026
0027     def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
0028         return self.estimate(messages).approx_tokens >= self.max_approx_tokens
0029
0030     def report(self, messages: list[dict[str, Any]]) -> str:
0031         stats = self.estimate(messages)
0032         pct = 100 * stats.approx_tokens / self.max_approx_tokens
0033         return (
0034             f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:}, "
0035             f"budget={self.max_approx_tokens:} ({pct:.1f}%)"
0036         )
0037
0038     def compact(
0039         self,
0040         messages: list[dict[str, Any]],
0041         summarizer: Callable[[list[dict[str, Any]]], str],
0042     ) -> list[dict[str, Any]]:
0043         if len(messages) <= self.keep_recent_messages + 2:
0044             return messages
0045
0046         cut = len(messages) - self.keep_recent_messages
0047         old = messages[:cut]
0048         recent = messages[cut:]
0049         summary = summarizer(old)
0050         compacted = {
0051             "role": "user",
0052             "content": (
0053                 "[COMPACTED_CONTEXT]\n"
0054                 "The following is a structured summary of earlier conversation state. "
0055                 "Treat it as historical context, not a new user request.\n\n"
0056                 + summary
0057             ),
0058         }
0059         return [compacted, *recent]

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```

0001 """Persistent project instructions + agent-written memory for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from pathlib import Path
0005 from typing import Iterable
0006
0007
0008 class MemoryStore:
0009     def __init__(self, workspace: Path):
0010         self.workspace = workspace.resolve()
0011         self.project_instructions = self.workspace / "CLAUDE.md"
0012         self.memory_dir = self.workspace / ".mini_claude" / "memory"
0013         self.memory_file = self.memory_dir / "MEMORY.md"
0014
0015     def load_project_instructions(self) -> str:
0016         if not self.project_instructions.exists():
0017             return ""
0018         return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]
0019
0020     def load_auto_memory(self) -> str:

```



```

0021     if not self.memory_file.exists():
0022         return ""
0023     # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
0024     text = self.memory_file.read_text(encoding="utf-8", errors="replace")
0025     return "\n".join(text.splitlines()[:200])[:25000]
0026
0027     def remember(self, note: str) -> str:
0028         note = note.strip()
0029         if not note:
0030             return "ERROR: empty memory note"
0031         self.memory_dir.mkdir(parents=True, exist_ok=True)
0032         existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
0033         normalized = f"- {note}"
0034         if normalized in existing:
0035             return "OK: note already present"
0036         with self.memory_file.open("a", encoding="utf-8") as fh:
0037             if not existing.endswith("\n"):
0038                 fh.write("\n")
0039             fh.write(normalized + "\n")
0040         return "OK: persisted memory"
0041
0042     def search(self, query: str, limit: int = 20) -> str:
0043         if not self.memory_file.exists():
0044             return "(memory is empty)"
0045         q = query.casefold().strip()
0046         lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
0047         hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
0048         selected = list(hits)[:limit]
0049         return "\n".join(selected) if selected else "(no memory matches)"
0050
0051     def startup_context(self) -> str:
0052         parts = []
0053         project = self.load_project_instructions().strip()
0054         memory = self.load_auto_memory().strip()
0055         if project:
0056             parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
0057         if memory:
0058             parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
0059         return "\n\n".join(parts)

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

7. s06 - Skills

用渐进式披露封装可重复 SOP

观察项	本阶段
阶段目录	s06_skills
文件数量	6
新增文件	.mini_claude/skills/verify-change/SKILL.md, skills.py
继承/修改文件	README.md, code.py, context_manager.py, memory.py
Python 行数	569

KNOW-WHY：为什么这一层必须存在

- CLAUDE.md 适合始终成立的事实，不适合每次都加载十几步 SOP。Skills 通过“描述常驻、正文按需加载”降低上下文常驻成本。
- 一个 Skill 不是新 Tool capability，而是对已有工具的可复用操作程序。
- 把重复 Prompt 固化为可版本化 Skill，能把个体经验变成团队可共享的操作资产。

KNOW-HOW：这一层具体怎么长出来

21. 扫描`.mini\_claude/skills/\*/SKILL.md`，并兼容`.claude/skills/\*/SKILL.md`。
22. 用YAML frontmatter 解析 name/description；启动只把 catalog 注入 system。
23. 模型判断相关时调用 use_skill，Runtime 才读取正文并作为 tool_result 返回。
24. Skill 支持相对路径引用其他资源，真正需要时仍由普通文件工具读取。

数据流 / 控制流

skill metadata -> system catalog -> use_skill(name) -> SKILL.md body -> model follows procedure

最容易踩的坑

- Skill 描述写得太宽，会被频繁误触发；写得太窄，又不会被模型发现。
- 把权限能力塞进 Skill；Skill 只是 instruction，不应绕过 Permission/Sandbox。
- 一次加载过大的 Skill 正文仍会占用上下文，所以需要控制体积并拆支持文件。

本阶段验收清单

- 运行 /skills 查看目录。
- 让 Agent 在完成修改后使用 verify-change Skill。
- 临时新增一个 Skill，重启后确认被发现。

Progressive Disclosure 的 token 经济学

如果有 30 个 1,000-token SOP 全部塞进启动 Prompt，每轮都要支付约 30k token 的常驻成本。Skill catalog 只常驻几十到几百 token 的 name/description；只有真正使用时才加载正文。Claude Code 当前 Skills 文档也明确把“正文只在使用时加载”作为关键特性。[R5]

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (381 lines)

```
0001 #!/usr/bin/env python3
0002 """s06 - Add progressive-disclosure Skills."""
0003 from __future__ import annotations
0004
0005 import glob as globlib
0006 import json
0007 import os
0008 import subprocess
0009 import shlex
0010 from enum import Enum
0011 from pathlib import Path
0012 from typing import Any, Callable
0013
0014 from anthropic import Anthropic
0015 from dotenv import load_dotenv
0016 from context_manager import ContextManager
0017 from memory import MemoryStore
0018 from skills import SkillRegistry
0019
0020 load_dotenv()
0021 MODEL = os.environ.get("ANTHROPIC_MODEL")
0022 if not MODEL:
0023     raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")
0024
0025 client = Anthropic()
0026 WORKSPACE = Path.cwd().resolve()
0027 MAX_TOOL_OUTPUT = 12000
0028 CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
0029 MEMORY = MemoryStore(WORKSPACE)
0030 SKILLS = SkillRegistry(WORKSPACE)
0031
0032
0033 def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
0034     return {
0035         "name": name,
0036         "description": description,
0037         "input_schema": {
0038             "type": "object",
0039             "properties": properties,
0040             "required": required,
0041             "additionalProperties": False,
0042         },
0043     }
0044
0045
0046 TOOLS = [
0047     tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
0048     tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
0049     tool(
0050         "write_file",
0051         "Create or replace a UTF-8 text file inside the workspace.",
0052         {"path": {"type": "string"}, "content": {"type": "string"}},
0053         ["path", "content"],
0054     ),
0055     tool(
0056         "edit_file",
0057         "Replace one exact text occurrence in a workspace file.",
0058         {
0059             "path": {"type": "string"},
0060             "old_text": {"type": "string"},
0061             "new_text": {"type": "string"},
0062         },
0063         ["path", "old_text", "new_text"],
0064     ),
0065     tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
0066     tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}}, ["note"]),
0067     tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}}, ["query"]),
0068     tool("use_skill", "Load a reusable task procedure by name when it is relevant.", {"name": {"type": "string"}}, ["name"]),
0069 ]
0070
0071 BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
```

```

0072 Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
0073 Make focused changes and verify them before you finish.
0074 Use remember only for durable facts likely to help future sessions, not transient task state.
0075 """
0076
0077
0078 def build_system() -> str:
0079     startup = MEMORY.startup_context()
0080     catalog = SKILLS.catalog()
0081     parts = [BASE_SYSTEM]
0082     if startup:
0083         parts.append(startup)
0084     parts.append("<available_skills>\n" + catalog + "\n</available_skills>")
0085     parts.append("Load a skill with use_skill only when its procedure is needed.")
0086     return "\n\n".join(parts)
0087
0088
0089 def safe_path(raw: str) -> Path:
0090     """Resolve a path and reject escapes outside the workspace."""
0091     candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
0092     try:
0093         candidate.relative_to(WORKSPACE)
0094     except ValueError as exc:
0095         raise ValueError(f"Path escapes workspace: {raw!r}") from exc
0096     return candidate
0097
0098
0099 def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
0100     if len(text) <= limit:
0101         return text
0102     half = limit // 2
0103     return text[:half] + "\n...<truncated>...\n" + text[-half:]
0104
0105
0106 def run_bash(command: str) -> str:
0107     try:
0108         proc = subprocess.run(
0109             command,
0110             shell=True,
0111             text=True,
0112             capture_output=True,
0113             cwd=WORKSPACE,
0114             timeout=30,
0115         )
0116     except subprocess.TimeoutExpired:
0117         return "ERROR: command timed out after 30 seconds"
0118     return json.dumps(
0119         {
0120             "exit_code": proc.returncode,
0121             "stdout": truncate(proc.stdout),
0122             "stderr": truncate(proc.stderr),
0123         },
0124         ensure_ascii=False,
0125     )
0126
0127
0128 def run_read(path: str) -> str:
0129     p = safe_path(path)
0130     if not p.is_file():
0131         return f"ERROR: file not found: {path}"
0132     return truncate(p.read_text(encoding="utf-8", errors="replace"))
0133
0134
0135 def run_write(path: str, content: str) -> str:
0136     p = safe_path(path)
0137     p.parent.mkdir(parents=True, exist_ok=True)
0138     p.write_text(content, encoding="utf-8")
0139     return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"
0140
0141
0142 def run_edit(path: str, old_text: str, new_text: str) -> str:
0143     p = safe_path(path)
0144     if not p.is_file():
0145         return f"ERROR: file not found: {path}"
0146     text = p.read_text(encoding="utf-8", errors="replace")
0147     count = text.count(old_text)
0148     if count == 0:
0149         return "ERROR: old_text not found"
0150     if count > 1:
0151         return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"

```

```

0152     p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
0153     return f"OK: edited {p.relative_to(WORKSPACE)}"
0154
0155
0156 def run_glob(pattern: str) -> str:
0157     # Resolve matches then apply safe_path to each result to keep output inside the workspace.
0158     raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
0159     matches: list[str] = []
0160     for raw in raw_matches[:500]:
0161         try:
0162             p = safe_path(raw)
0163             except ValueError:
0164                 continue
0165             matches.append(str(p.relative_to(WORKSPACE)))
0166     return "\n".join(sorted(matches)) if matches else "(no matches)"
0167
0168
0169
0170
0171 class Decision(str, Enum):
0172     ALLOW = "allow"
0173     ASK = "ask"
0174     DENY = "deny"
0175
0176
0177 # Gate 1: patterns that should never be auto-approved by this tutorial runtime.
0178 HARD_DENY_SUBSTRINGS = (
0179     "sudo ",
0180     "chmod -R 777 /",
0181     "> /dev/sd",
0182     "mkfs.",
0183 )
0184
0185 # Gate 2: commands considered low-risk and repeatable in a coding workspace.
0186 SAFE_COMMAND_PREFIXES = (
0187     "pwd", "ls", "find ", "git status", "git diff", "git log",
0188     "python -m pytest", "pytest", "npm test", "npm run test",
0189     "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
0190 )
0191
0192
0193 def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
0194     """Return a policy decision and human-readable reason.
0195
0196     Order matters: hard deny -> explicit rules -> ask fallback.
0197     """
0198     if name in {"read_file", "glob", "recall_memory", "use_skill"}:
0199         return Decision.ALLOW, "read-only workspace operation"
0200
0201     if name == "remember":
0202         return Decision.ALLOW, "writes only to the agent memory store"
0203
0204     if name in {"write_file", "edit_file"}:
0205         try:
0206             safe_path(args.get("path", ""))
0207         except Exception as exc:
0208             return Decision.DENY, str(exc)
0209         return Decision.ASK, "file modification inside workspace"
0210
0211     if name == "bash":
0212         command = str(args.get("command", "")).strip()
0213         lowered = command.lower()
0214         if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
0215             return Decision.DENY, "matches hard deny policy"
0216         if command.startswith(SAFE_COMMAND_PREFIXES):
0217             return Decision.ALLOW, "matches low-risk command allowlist"
0218         return Decision.ASK, "shell command is not allowlisted"
0219
0220     return Decision.ASK, "unknown or extensible tool"
0221
0222
0223 def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
0224     print(f"\n[permission] {name}: {reason}")
0225     print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
0226     while True:
0227         answer = input("Allow once? [y/N] ").strip().lower()
0228         if answer in {"y", "yes"}:
0229             return True
0230         if answer in {"", "n", "no"}:
0231             return False

```

```

0232
0233
0234 def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
0235     decision, reason = classify_permission(name, args)
0236     if decision is Decision.DENY:
0237         return False, f"DENIED: {reason}"
0238     if decision is Decision.ALLOW:
0239         return True, reason
0240     if request_approval(name, args, reason):
0241         return True, "approved by user"
0242     return False, "DENIED: user rejected tool call"
0243
0244
0245 TOOL_HANDLERS: dict[str, Callable[..., str]] = {
0246     "bash": run_bash,
0247     "read_file": run_read,
0248     "write_file": run_write,
0249     "edit_file": run_edit,
0250     "glob": run_glob,
0251     "remember": MEMORY.remember,
0252     "recall_memory": MEMORY.search,
0253     "use_skill": SKILLS.load,
0254 }
0255
0256
0257 def block_to_dict(block: Any) -> dict[str, Any]:
0258     if hasattr(block, "model_dump"):
0259         return block.model_dump(exclude_none=True)
0260     return dict(block)
0261
0262
0263 def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
0264     handler = TOOL_HANDLERS.get(name)
0265     if handler is None:
0266         return f"Unknown tool: {name}", True
0267     try:
0268         return handler(**args), False
0269     except Exception as exc:
0270         return f"{type(exc).__name__}: {exc}", True
0271
0272
0273
0274
0275 def summarize_history(old_messages: list[dict[str, Any]]) -> str:
0276     """Ask the model to compress old state without tools."""
0277     prompt = """Summarize this coding-agent conversation for continuation.
0278 Preserve only durable state:
0279 - user goal and constraints
0280 - repository facts discovered
0281 - files changed and why
0282 - commands/tests run and outcomes
0283 - unresolved hypotheses, TODOs, and errors
0284 - important approvals or denied operations
0285 Do not invent facts. Be concise but concrete.
0286
0287 HISTORY JSON:
0288 """ + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
0289     response = client.messages.create(
0290         model=MODEL,
0291         max_tokens=1800,
0292         system="You compress coding-agent state for a future model turn.",
0293         messages=[{"role": "user", "content": prompt}],
0294     )
0295     return "\n".join(
0296         block.text for block in response.content if getattr(block, "type", None) == "text"
0297     )
0298
0299
0300 def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
0301     while True:
0302         if CONTEXT.needs_compaction(messages):
0303             print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
0304             messages = CONTEXT.compact(messages, summarize_history)
0305             response = client.messages.create(
0306                 model=MODEL,
0307                 max_tokens=4096,
0308                 system=build_system(),
0309                 tools=TOOLS,
0310                 messages=messages,
0311             )

```

```

0312
0313     for block in response.content:
0314         if getattr(block, "type", None) == "text":
0315             print(block.text)
0316
0317     messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0318     if response.stop_reason != "tool_use":
0319         return messages
0320
0321     results = []
0322     for block in response.content:
0323         if getattr(block, "type", None) != "tool_use":
0324             continue
0325         args = dict(block.input)
0326         print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
0327         allowed, permission_reason = check_permission(block.name, args)
0328         if allowed:
0329             content, is_error = execute_tool(block.name, args)
0330         else:
0331             content, is_error = permission_reason, True
0332         results.append(
0333             {
0334                 "type": "tool_result",
0335                 "tool_use_id": block.id,
0336                 "content": content,
0337                 "is_error": is_error,
0338             }
0339         )
0340     messages.append({"role": "user", "content": results})
0341
0342
0343 def main() -> None:
0344     print(f"Mini Claude Code s06 - workspace: {WORKSPACE}")
0345     messages: List[dict[str, Any]] = []
0346     while True:
0347         try:
0348             prompt = input("\nyou> ").strip()
0349         except (EOFError, KeyboardInterrupt):
0350             print()
0351             break
0352         if not prompt:
0353             continue
0354         if prompt in {"exit", "/quit"}:
0355             break
0356         if prompt == "/context":
0357             print(CONTEXT.report(messages))
0358             continue
0359         if prompt == "/compact":
0360             messages = CONTEXT.compact(messages, summarize_history)
0361             print("[context] " + CONTEXT.report(messages))
0362             continue
0363         if prompt == "/clear":
0364             messages = []
0365             print("[context] cleared")
0366             continue
0367         if prompt == "/memory":
0368             print(MEMORY.load_auto_memory() or "(memory is empty)")
0369             continue
0370         if prompt == "/skills":
0371             print(SKILLS.catalog())
0372             continue
0373         messages.append({"role": "user", "content": prompt})
0374         try:
0375             messages = agent_loop(messages)
0376         except Exception as exc:
0377             print(f"ERROR: {exc}")
0378
0379
0380 if __name__ == "__main__":
0381     main()

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/skills/verify-change/SKILL.md (12 lines)

```

0001 ---
0002 name: verify-change
0003 description: Verify a code change before declaring the task complete.
0004 ---
0005
0006 When implementation appears complete:

```

```
0007 1. inspect `git diff --stat` and `git diff`
0008 2. run the narrowest relevant test first
0009 3. run the broader regression suite if practical
0010 4. run lint/typecheck if the repository has them
0011 5. report exact commands and outcomes
0012 6. flag anything that was not verified
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (6 Lines)

```
0001 # s06 Skills
0002
0003 Adds skill discovery from `.mini_claude/skills/*/SKILL.md` and `.claude/skills/*/SKILL.md`.
0004 Only names/descriptions are injected at startup. The body loads through `use_skill` on demand.
0005
0006 Dependency added: `pyyaml`.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 Lines)

```
0001 """A small context manager: estimate, inspect, and compact message history."""
0002 from __future__ import annotations
0003
0004 import json
0005 from dataclasses import dataclass
0006 from typing import Any, Callable
0007
0008
0009 @dataclass
0010 class ContextStats:
0011     message_count: int
0012     approx_tokens: int
0013     chars: int
0014
0015
0016 class ContextManager:
0017     def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
0018         self.max_approx_tokens = max_approx_tokens
0019         self.keep_recent_messages = keep_recent_messages
0020
0021     @staticmethod
0022     def estimate(messages: list[dict[str, Any]]) -> ContextStats:
0023         raw = json.dumps(messages, ensure_ascii=False, default=str)
0024         # Deliberately simple heuristic. Production systems should use model-aware counting.
0025         return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))
0026
0027     def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
0028         return self.estimate(messages).approx_tokens >= self.max_approx_tokens
0029
0030     def report(self, messages: list[dict[str, Any]]) -> str:
0031         stats = self.estimate(messages)
0032         pct = 100 * stats.approx_tokens / self.max_approx_tokens
0033         return (
0034             f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:}, ",
0035             f"budget={self.max_approx_tokens:} ({pct:.1f}%)"
0036         )
0037
0038     def compact(
0039         self,
0040         messages: list[dict[str, Any]],
0041         summarizer: Callable[[list[dict[str, Any]]], str],
0042     ) -> list[dict[str, Any]]:
0043         if len(messages) <= self.keep_recent_messages + 2:
0044             return messages
0045
0046         cut = len(messages) - self.keep_recent_messages
0047         old = messages[:cut]
0048         recent = messages[cut:]
0049         summary = summarizer(old)
0050         compacted = {
0051             "role": "user",
0052             "content": (
0053                 "[COMPACTED_CONTEXT]\n"
0054                 "The following is a structured summary of earlier conversation state. "
0055                 "Treat it as historical context, not a new user request.\n\n"
0056                 + summary
0057             ),
0058         }
0059         return [compacted, *recent]
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```
0001 """Persistent project instructions + agent-written memory for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from pathlib import Path
0005 from typing import Iterable
0006
0007
0008 class MemoryStore:
0009     def __init__(self, workspace: Path):
0010         self.workspace = workspace.resolve()
0011         self.project_instructions = self.workspace / "CLAUDE.md"
0012         self.memory_dir = self.workspace / ".mini_claude" / "memory"
0013         self.memory_file = self.memory_dir / "MEMORY.md"
0014
0015     def load_project_instructions(self) -> str:
0016         if not self.project_instructions.exists():
0017             return ""
0018         return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]
0019
0020     def load_auto_memory(self) -> str:
0021         if not self.memory_file.exists():
0022             return ""
0023         # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
0024         text = self.memory_file.read_text(encoding="utf-8", errors="replace")
0025         return "\n".join(text.splitlines()[:200])[:25000]
0026
0027     def remember(self, note: str) -> str:
0028         note = note.strip()
0029         if not note:
0030             return "ERROR: empty memory note"
0031         self.memory_dir.mkdir(parents=True, exist_ok=True)
0032         existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
0033         normalized = f"- {note}"
0034         if normalized in existing:
0035             return "OK: note already present"
0036         with self.memory_file.open("a", encoding="utf-8") as fh:
0037             if not existing.endswith("\n"):
0038                 fh.write("\n")
0039             fh.write(normalized + "\n")
0040         return "OK: persisted memory"
0041
0042     def search(self, query: str, limit: int = 20) -> str:
0043         if not self.memory_file.exists():
0044             return "(memory is empty)"
0045         q = query.casefold().strip()
0046         lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
0047         hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
0048         selected = list(hits)[:limit]
0049         return "\n".join(selected) if selected else "(no memory matches)"
0050
0051     def startup_context(self) -> str:
0052         parts = []
0053         project = self.load_project_instructions().strip()
0054         memory = self.load_auto_memory().strip()
0055         if project:
0056             parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
0057         if memory:
0058             parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
0059         return "\n\n".join(parts)
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

skills.py (70 lines)

```
0001 """Progressive-disclosure skills for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class Skill:
0013     name: str
0014     description: str
0015     path: Path
```

```

0016     body: str
0017     metadata: dict[str, Any]
0018
0019
0020 class SkillRegistry:
0021     def __init__(self, workspace: Path):
0022         self.workspace = workspace.resolve()
0023         self.roots = [
0024             self.workspace / ".mini_claude" / "skills",
0025             self.workspace / ".claude" / "skills", # optional compatibility
0026         ]
0027
0028     @staticmethod
0029     def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0030         if not text.startswith("---\n"):
0031             return {}, text
0032         end = text.find("\n---\n", 4)
0033         if end == -1:
0034             return {}, text
0035         raw = text[4:end]
0036         body = text[end + 5 :]
0037         data = yaml.safe_load(raw) or {}
0038         if not isinstance(data, dict):
0039             data = {}
0040         return data, body
0041
0042     def discover(self) -> dict[str, Skill]:
0043         found: dict[str, Skill] = {}
0044         for root in self.roots:
0045             if not root.exists():
0046                 continue
0047             for skill_file in sorted(root.glob("*/SKILL.md")):
0048                 text = skill_file.read_text(encoding="utf-8", errors="replace")
0049                 meta, body = self._split_frontmatter(text)
0050                 name = str(meta.get("name") or skill_file.parent.name)
0051                 description = str(meta.get("description") or "No description provided")
0052                 found[name] = Skill(name, description, skill_file, body.strip(), meta)
0053         return found
0054
0055     def catalog(self) -> str:
0056         skills = self.discover()
0057         if not skills:
0058             return "(no skills installed)"
0059         return "\n".join(f"- {name}: {skill.description}" for name, skill in skills.items())
0060
0061     def load(self, name: str) -> str:
0062         skill = self.discover().get(name)
0063         if skill is None:
0064             return f"ERROR: unknown skill {name!r}. Available:\n{self.catalog()}"
0065         # A skill can reference support files by relative path; normal file tools can read them later.
0066         return (
0067             f"# Skill: {skill.name}\n"
0068             f"Source: {skill.path.relative_to(self.workspace)}\n\n"
0069             f"{skill.body[:30000]}"
0070         )

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

8. s07 - Hooks

把“建议”升级为确定性的生命周期控制

观察项	本阶段
阶段目录	s07_hooks
文件数量	9
新增文件	.mini_claude/audit_hook.py, .mini_claude/hooks.json, hooks.py
继承/修改文件	.mini_claude/skills/verify-change/SKILL.md, README.md, code.py, context_manager.py, memory.py, skills.py
Python 行数	690

KNOW-WHY：为什么这一层必须存在

- Prompt/Skill 的本质仍是“模型应该这么做”；Hook 用代码在固定生命周期点执行，适合审计、阻断、格式化和策略注入。
- 把 deterministic automation 放在 Runtime，而不是期待模型永远记得执行，可以减少遗漏和 Prompt 竞争。
- PreToolUse 是副作用发生前最后一个统一拦截点；PostToolUse 是统一审计和后处理点。

KNOW-HOW：这一层具体怎么长出来

- 25. 从`.mini\_claude/hooks.json`读取 event -> command 列表。
- 26. Hook 从 stdin 接收 JSON envelope，并可返回 allow/ask/deny、reason、updated_input、output。
- 27. 工具执行顺序变成：PreHook -> Permission -> Tool -> PostHook。
- 28. 同时对 session start/end、pre/post compact 提供事件。

数据流 / 控制流

tool_use -> PreHook -> Permission -> Tool -> PostHook -> tool_result

最容易踩的坑

- Hook 自己也是可执行代码，配置来源必须可信。
- Hook 卡住会把整个 Agent Loop 卡住，因此必须设置 timeout。
- Hook 和 Permission 冲突时需要明确优先级；本实现 PreHook deny 优先，ask 可升级为人工批准。

本阶段验收清单

- 运行一条工具调用后检查`.mini\_claude/audit.jsonl`。
- 让 Hook 返回 deny，确认工具未执行。
- 执行 /compact，检查 pre/post compact 事件。

Instruction vs Enforcement

设计准则

“最好这么做”放 Prompt/Skill；“一定要发生”放 Hook/Policy/Sandbox。Hook 适合格式化、审计、阻断、资源清理和确定性检查。Claude Code 公开 Hook 生命周期覆盖 tool、session、compact、subagent、worktree 等事件。[R6]

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (408 lines)

```
0001 #!/usr/bin/env python3
0002 """s07 - Add deterministic lifecycle Hooks."""
0003 from __future__ import annotations
0004
0005 import glob as globlib
0006 import json
0007 import os
0008 import subprocess
0009 import shlex
0010 from enum import Enum
0011 from pathlib import Path
0012 from typing import Any, Callable
0013
0014 from anthropic import Anthropic
0015 from dotenv import load_dotenv
0016 from context_manager import ContextManager
0017 from memory import MemoryStore
0018 from skills import SkillRegistry
0019 from hooks import HookManager
0020
0021 load_dotenv()
0022 MODEL = os.environ.get("ANTHROPIC_MODEL")
0023 if not MODEL:
0024     raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")
0025
0026 client = Anthropic()
0027 WORKSPACE = Path.cwd().resolve()
0028 MAX_TOOL_OUTPUT = 12000
0029 CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
0030 MEMORY = MemoryStore(WORKSPACE)
0031 SKILLS = SkillRegistry(WORKSPACE)
0032 HOOKS = HookManager(WORKSPACE)
0033
0034
0035 def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
0036     return {
0037         "name": name,
0038         "description": description,
0039         "input_schema": {
0040             "type": "object",
0041             "properties": properties,
0042             "required": required,
0043             "additionalProperties": False,
0044         },
0045     }
0046
0047
0048 TOOLS = [
0049     tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
0050     tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
0051     tool(
0052         "write_file",
0053         "Create or replace a UTF-8 text file inside the workspace.",
0054         {"path": {"type": "string"}, "content": {"type": "string"}},
0055         ["path", "content"],
0056     ),
0057     tool(
0058         "edit_file",
0059         "Replace one exact text occurrence in a workspace file.",
0060         {
0061             "path": {"type": "string"},
0062             "old_text": {"type": "string"},
0063             "new_text": {"type": "string"},
0064         },
0065         ["path", "old_text", "new_text"],
0066     ),
0067     tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
0068     tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}}, ["note"]),
0069     tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}}, ["query"]),
0070     tool("use_skill", "Load a reusable task procedure by name when it is relevant.", {"name": {"type": "string"}}, ["name"]),
0071 ]
```

```

0072
0073 BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
0074 Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
0075 Make focused changes and verify them before you finish.
0076 Use remember only for durable facts likely to help future sessions, not transient task state.
0077 """
0078
0079
0080 def build_system() -> str:
0081     startup = MEMORY.startup_context()
0082     catalog = SKILLS.catalog()
0083     parts = [BASE_SYSTEM]
0084     if startup:
0085         parts.append(startup)
0086     parts.append("<available_skills>\n" + catalog + "\n</available_skills>")
0087     parts.append("Load a skill with use_skill only when its procedure is needed.")
0088     return "\n\n".join(parts)
0089
0090
0091 def safe_path(raw: str) -> Path:
0092     """Resolve a path and reject escapes outside the workspace."""
0093     candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
0094     try:
0095         candidate.relative_to(WORKSPACE)
0096     except ValueError as exc:
0097         raise ValueError(f"Path escapes workspace: {raw!r}") from exc
0098     return candidate
0099
0100
0101 def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
0102     if len(text) <= limit:
0103         return text
0104     half = limit // 2
0105     return text[:half] + "\n...<truncated>...\n" + text[-half:]
0106
0107
0108 def run_bash(command: str) -> str:
0109     try:
0110         proc = subprocess.run(
0111             command,
0112             shell=True,
0113             text=True,
0114             capture_output=True,
0115             cwd=WORKSPACE,
0116             timeout=30,
0117         )
0118     except subprocess.TimeoutExpired:
0119         return "ERROR: command timed out after 30 seconds"
0120     return json.dumps(
0121         {
0122             "exit_code": proc.returncode,
0123             "stdout": truncate(proc.stdout),
0124             "stderr": truncate(proc.stderr),
0125         },
0126         ensure_ascii=False,
0127     )
0128
0129
0130 def run_read(path: str) -> str:
0131     p = safe_path(path)
0132     if not p.is_file():
0133         return f"ERROR: file not found: {path}"
0134     return truncate(p.read_text(encoding="utf-8", errors="replace"))
0135
0136
0137 def run_write(path: str, content: str) -> str:
0138     p = safe_path(path)
0139     p.parent.mkdir(parents=True, exist_ok=True)
0140     p.write_text(content, encoding="utf-8")
0141     return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"
0142
0143
0144 def run_edit(path: str, old_text: str, new_text: str) -> str:
0145     p = safe_path(path)
0146     if not p.is_file():
0147         return f"ERROR: file not found: {path}"
0148     text = p.read_text(encoding="utf-8", errors="replace")
0149     count = text.count(old_text)
0150     if count == 0:
0151         return "ERROR: old_text not found"

```

```

0152     if count > 1:
0153         return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
0154     p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
0155     return f"OK: edited {p.relative_to(WORKSPACE)}"
0156
0157
0158 def run_glob(pattern: str) -> str:
0159     # Resolve matches then apply safe_path to each result to keep output inside the workspace.
0160     raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
0161     matches: list[str] = []
0162     for raw in raw_matches[:500]:
0163         try:
0164             p = safe_path(raw)
0165         except ValueError:
0166             continue
0167         matches.append(str(p.relative_to(WORKSPACE)))
0168     return "\n".join(sorted(matches)) if matches else "(no matches)"
0169
0170
0171
0172
0173 class Decision(str, Enum):
0174     ALLOW = "allow"
0175     ASK = "ask"
0176     DENY = "deny"
0177
0178
0179 # Gate 1: patterns that should never be auto-approved by this tutorial runtime.
0180 HARD_DENY_SUBSTRINGS = (
0181     "sudo ",
0182     "chmod -R 777 /",
0183     "> /dev/sd",
0184     "mkfs.",
0185 )
0186
0187 # Gate 2: commands considered low-risk and repeatable in a coding workspace.
0188 SAFE_COMMAND_PREFIXES = (
0189     "pwd", "ls", "find", "git status", "git diff", "git log",
0190     "python -m pytest", "pytest", "npm test", "npm run test",
0191     "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
0192 )
0193
0194
0195 def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
0196     """Return a policy decision and human-readable reason.
0197
0198     Order matters: hard deny -> explicit rules -> ask fallback.
0199     """
0200     if name in {"read_file", "glob", "recall_memory", "use_skill"}:
0201         return Decision.ALLOW, "read-only workspace operation"
0202
0203     if name == "remember":
0204         return Decision.ALLOW, "writes only to the agent memory store"
0205
0206     if name in {"write_file", "edit_file"}:
0207         try:
0208             safe_path(args.get("path", ""))
0209         except Exception as exc:
0210             return Decision.DENY, str(exc)
0211         return Decision.ASK, "file modification inside workspace"
0212
0213     if name == "bash":
0214         command = str(args.get("command", "")).strip()
0215         lowered = command.lower()
0216         if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
0217             return Decision.DENY, "matches hard deny policy"
0218         if command.startswith(SAFE_COMMAND_PREFIXES):
0219             return Decision.ALLOW, "matches low-risk command allowlist"
0220         return Decision.ASK, "shell command is not allowlisted"
0221
0222     return Decision.ASK, "unknown or extensible tool"
0223
0224
0225 def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
0226     print(f"\n[permission] {name}: {reason}")
0227     print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
0228     while True:
0229         answer = input("Allow once? [y/N] ").strip().lower()
0230         if answer in {"y", "yes"}:
0231             return True

```

```

0232         if answer in {"", "n", "no"}:
0233             return False
0234
0235
0236 def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
0237     decision, reason = classify_permission(name, args)
0238     if decision is Decision.DENY:
0239         return False, f"DENIED: {reason}"
0240     if decision is Decision.ALLOW:
0241         return True, reason
0242     if request_approval(name, args, reason):
0243         return True, "approved by user"
0244     return False, "DENIED: user rejected tool call"
0245
0246
0247 TOOL_HANDLERS: dict[str, Callable[..., str]] = {
0248     "bash": run_bash,
0249     "read_file": run_read,
0250     "write_file": run_write,
0251     "edit_file": run_edit,
0252     "glob": run_glob,
0253     "remember": MEMORY.remember,
0254     "recall_memory": MEMORY.search,
0255     "use_skill": SKILLS.load,
0256 }
0257
0258
0259 def block_to_dict(block: Any) -> dict[str, Any]:
0260     if hasattr(block, "model_dump"):
0261         return block.model_dump(exclude_none=True)
0262     return dict(block)
0263
0264
0265 def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
0266     handler = TOOL_HANDLERS.get(name)
0267     if handler is None:
0268         return f"Unknown tool: {name}", True
0269     try:
0270         return handler(**args), False
0271     except Exception as exc:
0272         return f"type(exc).__name__: {exc}", True
0273
0274
0275
0276
0277 def summarize_history(old_messages: list[dict[str, Any]]) -> str:
0278     """Ask the model to compress old state without tools."""
0279     prompt = """Summarize this coding-agent conversation for continuation.
0280 Preserve only durable state:
0281 - user goal and constraints
0282 - repository facts discovered
0283 - files changed and why
0284 - commands/tests run and outcomes
0285 - unresolved hypotheses, TODOs, and errors
0286 - important approvals or denied operations
0287 Do not invent facts. Be concise but concrete.
0288
0289 HISTORY JSON:
0290 """ + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
0291     response = client.messages.create(
0292         model=MODEL,
0293         max_tokens=1800,
0294         system="You compress coding-agent state for a future model turn.",
0295         messages=[{"role": "user", "content": prompt}],
0296     )
0297     return "\n".join(
0298         block.text for block in response.content if getattr(block, "type", None) == "text"
0299     )
0300
0301
0302 def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
0303     while True:
0304         if CONTEXT.needs_compaction(messages):
0305             print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
0306             HOOKS.fire("pre_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
0307             messages = CONTEXT.compact(messages, summarize_history)
0308             HOOKS.fire("post_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
0309             response = client.messages.create(
0310                 model=MODEL,
0311                 max_tokens=4096,

```

```

0312         system=build_system(),
0313         tools=TOOLS,
0314         messages=messages,
0315     )
0316
0317     for block in response.content:
0318         if getattr(block, "type", None) == "text":
0319             print(block.text)
0320
0321     messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0322     if response.stop_reason != "tool_use":
0323         return messages
0324
0325     results = []
0326     for block in response.content:
0327         if getattr(block, "type", None) != "tool_use":
0328             continue
0329         args = dict(block.input)
0330         print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
0331
0332         pre = HOOKS.fire("pre_tool_use", {"tool_name": block.name, "tool_input": args})
0333         if pre.updated_input is not None:
0334             args = pre.updated_input
0335         if pre.decision == "deny":
0336             content, is_error = f"DENIED BY HOOK: {pre.reason}", True
0337         else:
0338             if pre.decision == "ask":
0339                 allowed = request_approval(block.name, args, pre.reason or "hook requested approval")
0340                 permission_reason = "approved by user" if allowed else "DENIED: user rejected hook approval"
0341             else:
0342                 allowed, permission_reason = check_permission(block.name, args)
0343             if allowed:
0344                 content, is_error = execute_tool(block.name, args)
0345             else:
0346                 content, is_error = permission_reason, True
0347
0348         post = HOOKS.fire(
0349             "post_tool_use",
0350             {"tool_name": block.name, "tool_input": args, "tool_output": content, "is_error": is_error},
0351         )
0352         if post.output:
0353             content += "\n[hook output]\n" + post.output.strip()
0354         results.append(
0355             {
0356                 "type": "tool_result",
0357                 "tool_use_id": block.id,
0358                 "content": content,
0359                 "is_error": is_error,
0360             }
0361         )
0362     messages.append({"role": "user", "content": results})
0363
0364
0365 def main() -> None:
0366     print(f"Mini Claude Code s07 - workspace: {WORKSPACE}")
0367     HOOKS.fire("session_start", {"model": MODEL})
0368     messages: list[dict[str, Any]] = []
0369     while True:
0370         try:
0371             prompt = input("\nyou> ").strip()
0372         except (EOFError, KeyboardInterrupt):
0373             print()
0374             break
0375         if not prompt:
0376             continue
0377         if prompt in {" /exit", " /quit"}:
0378             break
0379         if prompt == " /context":
0380             print(CONTEXT.report(messages))
0381             continue
0382         if prompt == " /compact":
0383             HOOKS.fire("pre_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
0384             messages = CONTEXT.compact(messages, summarize_history)
0385             HOOKS.fire("post_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
0386             print("[context] " + CONTEXT.report(messages))
0387             continue
0388         if prompt == " /clear":
0389             messages = []
0390             print("[context] cleared")
0391             continue

```



```

0392         if prompt == "/memory":
0393             print(MEMORY.load_auto_memory() or "(memory is empty)")
0394             continue
0395         if prompt == "/skills":
0396             print(SKILLS.catalog())
0397             continue
0398         messages.append({"role": "user", "content": prompt})
0399         try:
0400             messages = agent_loop(messages)
0401         except Exception as exc:
0402             print(f"ERROR: {exc}")
0403
0404     HOOKS.fire("session_end", {"message_count": len(messages)})
0405
0406
0407 if __name__ == "__main__":
0408     main()

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/audit_hook.py (12 lines)

```

0001 #!/usr/bin/env python3
0002 """Example hook: append compact JSON events to .mini_claude/audit.jsonl."""
0003 import json
0004 import sys
0005 from pathlib import Path
0006
0007 event = json.load(sys.stdin)
0008 path = Path('.mini_claude/audit.jsonl')
0009 path.parent.mkdir(parents=True, exist_ok=True)
0010 with path.open('a', encoding='utf-8') as fh:
0011     fh.write(json.dumps(event, ensure_ascii=False, default=str) + '\n')
0012 print(json.dumps({"decision": "allow"}))

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/hooks.json (8 lines)

```

0001 {
0002     "session_start": ["python .mini_claude/audit_hook.py"],
0003     "pre_tool_use": ["python .mini_claude/audit_hook.py"],
0004     "post_tool_use": ["python .mini_claude/audit_hook.py"],
0005     "pre_compact": ["python .mini_claude/audit_hook.py"],
0006     "post_compact": ["python .mini_claude/audit_hook.py"],
0007     "session_end": ["python .mini_claude/audit_hook.py"]
0008 }

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/skills/verify-change/SKILL.md (12 lines)

```

0001 ---
0002 name: verify-change
0003 description: Verify a code change before declaring the task complete.
0004 ---
0005
0006 When implementation appears complete:
0007 1. inspect `git diff --stat` and `git diff`
0008 2. run the narrowest relevant test first
0009 3. run the broader regression suite if practical
0010 4. run lint/typecheck if the repository has them
0011 5. report exact commands and outcomes
0012 6. flag anything that was not verified

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (10 lines)

```

0001 # s07 Hooks
0002
0003 Adds deterministic lifecycle events configured in `.mini_claude/hooks.json`.
0004 Command hooks receive one JSON object on stdin and may return JSON with:
0005 - `decision`: `allow`, `ask`, `deny`
0006 - `reason`
0007 - `updated_input`
0008 - `output`
0009
0010 Implemented events: session start/end, pre/post tool use, pre/post compact.

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```

0001 """A small context manager: estimate, inspect, and compact message history."""
0002 from __future__ import annotations

```

```

0003
0004 import json
0005 from dataclasses import dataclass
0006 from typing import Any, Callable
0007
0008
0009 @dataclass
0010 class ContextStats:
0011     message_count: int
0012     approx_tokens: int
0013     chars: int
0014
0015
0016 class ContextManager:
0017     def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
0018         self.max_approx_tokens = max_approx_tokens
0019         self.keep_recent_messages = keep_recent_messages
0020
0021     @staticmethod
0022     def estimate(messages: list[dict[str, Any]]) -> ContextStats:
0023         raw = json.dumps(messages, ensure_ascii=False, default=str)
0024         # Deliberately simple heuristic. Production systems should use model-aware counting.
0025         return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))
0026
0027     def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
0028         return self.estimate(messages).approx_tokens >= self.max_approx_tokens
0029
0030     def report(self, messages: list[dict[str, Any]]) -> str:
0031         stats = self.estimate(messages)
0032         pct = 100 * stats.approx_tokens / self.max_approx_tokens
0033         return (
0034             f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:}, "
0035             f"budget={self.max_approx_tokens:} ({{pct:.1f}}%)"
0036         )
0037
0038     def compact(
0039         self,
0040         messages: list[dict[str, Any]],
0041         summarizer: Callable[[list[dict[str, Any]]], str],
0042     ) -> list[dict[str, Any]]:
0043         if len(messages) <= self.keep_recent_messages + 2:
0044             return messages
0045
0046         cut = len(messages) - self.keep_recent_messages
0047         old = messages[:cut]
0048         recent = messages[cut:]
0049         summary = summarizer(old)
0050         compacted = {
0051             "role": "user",
0052             "content": (
0053                 "[COMPACTED_CONTEXT]\n"
0054                 "The following is a structured summary of earlier conversation state. "
0055                 "Treat it as historical context, not a new user request.\n\n"
0056                 + summary
0057             ),
0058         }
0059         return [compacted, *recent]

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

hooks.py (82 lines)

```

0001 """Deterministic lifecycle hooks for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 import json
0005 import subprocess
0006 from dataclasses import dataclass
0007 from pathlib import Path
0008 from typing import Any
0009
0010
0011 @dataclass
0012 class HookResult:
0013     decision: str = "allow" # allow | deny | ask
0014     reason: str = ""
0015     updated_input: dict[str, Any] | None = None
0016     output: str = ""
0017
0018
0019 class HookManager:

```

```

0020     def __init__(self, workspace: Path):
0021         self.workspace = workspace.resolve()
0022         self.config_path = self.workspace / ".mini_claude" / "hooks.json"
0023
0024     def _config(self) -> dict[str, Any]:
0025         if not self.config_path.exists():
0026             return {}
0027         try:
0028             data = json.loads(self.config_path.read_text(encoding="utf-8"))
0029             return data if isinstance(data, dict) else {}
0030         except Exception as exc:
0031             print(f"[hook] invalid config: {exc}")
0032             return {}
0033
0034     def fire(self, event: str, payload: dict[str, Any]) -> HookResult:
0035         commands = self._config().get(event, [])
0036         if isinstance(commands, str):
0037             commands = [commands]
0038         result = HookResult()
0039         for command in commands:
0040             if not isinstance(command, str) or not command.strip():
0041                 continue
0042             envelope = {"event": event, "cwd": str(self.workspace), **payload}
0043             try:
0044                 proc = subprocess.run(
0045                     command,
0046                     shell=True,
0047                     cwd=self.workspace,
0048                     input=json.dumps(envelope, ensure_ascii=False),
0049                     text=True,
0050                     capture_output=True,
0051                     timeout=15,
0052                 )
0053             except subprocess.TimeoutExpired:
0054                 return HookResult(decision="deny", reason=f"hook timed out: {command}")
0055
0056             if proc.stderr.strip():
0057                 print(f"[hook:{event}:stderr] {proc.stderr.strip()[:2000]}")
0058             if proc.returncode != 0:
0059                 return HookResult(decision="deny", reason=f"hook failed ({proc.returncode}): {command}")
0060
0061             stdout = proc.stdout.strip()
0062             if not stdout:
0063                 continue
0064             try:
0065                 data = json.loads(stdout)
0066             except json.JSONDecodeError:
0067                 result.output += stdout + "\n"
0068                 continue
0069
0070             if isinstance(data, dict):
0071                 decision = str(data.get("decision", "allow"))
0072                 if decision in {"allow", "deny", "ask"}:
0073                     result.decision = decision
0074                 if data.get("reason"):
0075                     result.reason = str(data["reason"])
0076                 if isinstance(data.get("updated_input"), dict):
0077                     result.updated_input = data["updated_input"]
0078                 if data.get("output"):
0079                     result.output += str(data["output"]) + "\n"
0080                 if result.decision == "deny":
0081                     return result
0082         return result

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```

0001 """Persistent project instructions + agent-written memory for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from pathlib import Path
0005 from typing import Iterable
0006
0007
0008 class MemoryStore:
0009     def __init__(self, workspace: Path):
0010         self.workspace = workspace.resolve()
0011         self.project_instructions = self.workspace / "CLAUDE.md"
0012         self.memory_dir = self.workspace / ".mini_claude" / "memory"
0013         self.memory_file = self.memory_dir / "MEMORY.md"

```

```

0014
0015     def load_project_instructions(self) -> str:
0016         if not self.project_instructions.exists():
0017             return ""
0018         return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]
0019
0020     def load_auto_memory(self) -> str:
0021         if not self.memory_file.exists():
0022             return ""
0023         # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
0024         text = self.memory_file.read_text(encoding="utf-8", errors="replace")
0025         return "\n".join(text.splitlines()[:200])[:25000]
0026
0027     def remember(self, note: str) -> str:
0028         note = note.strip()
0029         if not note:
0030             return "ERROR: empty memory note"
0031         self.memory_dir.mkdir(parents=True, exist_ok=True)
0032         existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
0033         normalized = f"- {note}"
0034         if normalized in existing:
0035             return "OK: note already present"
0036         with self.memory_file.open("a", encoding="utf-8") as fh:
0037             if not existing.endswith("\n"):
0038                 fh.write("\n")
0039             fh.write(normalized + "\n")
0040         return "OK: persisted memory"
0041
0042     def search(self, query: str, limit: int = 20) -> str:
0043         if not self.memory_file.exists():
0044             return "(memory is empty)"
0045         q = query.casefold().strip()
0046         lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
0047         hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
0048         selected = list(hits)[:limit]
0049         return "\n".join(selected) if selected else "(no memory matches)"
0050
0051     def startup_context(self) -> str:
0052         parts = []
0053         project = self.load_project_instructions().strip()
0054         memory = self.load_auto_memory().strip()
0055         if project:
0056             parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
0057         if memory:
0058             parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
0059         return "\n\n".join(parts)

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

skills.py (70 lines)

```

0001 """Progressive-disclosure skills for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class Skill:
0013     name: str
0014     description: str
0015     path: Path
0016     body: str
0017     metadata: dict[str, Any]
0018
0019
0020 class SkillRegistry:
0021     def __init__(self, workspace: Path):
0022         self.workspace = workspace.resolve()
0023         self.roots = [
0024             self.workspace / ".mini_claude" / "skills",
0025             self.workspace / ".claude" / "skills", # optional compatibility
0026         ]
0027
0028     @staticmethod
0029     def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0030         if not text.startswith("---\n"):

```

```

0031         return {}, text
0032     end = text.find("\n---\n", 4)
0033     if end == -1:
0034         return {}, text
0035     raw = text[4:end]
0036     body = text[end + 5 :]
0037     data = yaml.safe_load(raw) or {}
0038     if not isinstance(data, dict):
0039         data = {}
0040     return data, body
0041
0042     def discover(self) -> dict[str, Skill]:
0043         found: dict[str, Skill] = {}
0044         for root in self.roots:
0045             if not root.exists():
0046                 continue
0047             for skill_file in sorted(root.glob("**/SKILL.md")):
0048                 text = skill_file.read_text(encoding="utf-8", errors="replace")
0049                 meta, body = self._split_frontmatter(text)
0050                 name = str(meta.get("name") or skill_file.parent.name)
0051                 description = str(meta.get("description") or "No description provided")
0052                 found[name] = Skill(name, description, skill_file, body.strip(), meta)
0053         return found
0054
0055     def catalog(self) -> str:
0056         skills = self.discover()
0057         if not skills:
0058             return "(no skills installed)"
0059         return "\n".join(f"- {name}: {skill.description}" for name, skill in skills.items())
0060
0061     def load(self, name: str) -> str:
0062         skill = self.discover().get(name)
0063         if skill is None:
0064             return f"ERROR: unknown skill {name!r}. Available:\n{self.catalog()}"
0065         # A skill can reference support files by relative path; normal file tools can read them later.
0066         return (
0067             f"# Skill: {skill.name}\n"
0068             f"Source: {skill.path.relative_to(self.workspace)}\n\n"
0069             f"{skill.body[:30000]}"
0070         )

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

9. s08 - Subagents

用独立 Context Window 隔离探索与评审

观察项	本阶段
阶段目录	s08_subagents
文件数量	12
新增文件	.mini_claude/agents/explorer.md, .mini_claude/agents/reviewer.md, agents.py
继承/修改文件	.mini_claude/audit_hook.py, .mini_claude/hooks.json, .mini_claude/skills/verify-change/SKILL.md, README.md, code.py, context_manager.py, hooks.py, memory.py ...
Python 行数	855

KNOW-WHY：为什么这一层必须存在

- Subagent 的最大价值往往不是“并行”，而是 Context Isolation：把大量搜索、日志和代码阅读留在独立窗口，只把结论返回主 Agent。
- 角色化 Subagent 可以配置更小的工具集合、不同模型和 turn budget，从而把能力与风险边界一起下放。
- 让生成者与评审者使用不同 Context，可以降低自我确认偏误。

KNOW-HOW：这一层具体怎么长出来

- 29. 解析`.mini\_claude/agents/\*.md`的 frontmatter：name、description、tools、model、max_turns。
- 30. spawn_subagent 创建全新的 message history，注入角色 prompt，并只暴露 allowlist 工具。
- 31. Subagent 内部仍复用 Permission/Hook/Tool Runtime。
- 32. 禁止嵌套 spawn_subagent，避免教学实现出现递归爆炸。

数据流 / 控制流

main context -> spawn_subagent(task) -> new context + restricted tools -> compact result -> main

最容易踩的坑

- 把主会话全部历史复制给 Subagent，会失去 Context Isolation 的主要价值。
- Subagent 工具列表过宽，会使“只读探索者”也具备修改能力。
- 主 Agent 把每个小步骤都委托出去会增加成本和协调开销。

本阶段验收清单

- 用 explorer 调查某个模块并确认不会修改文件。
- 用 reviewer 独立审查 git diff。
- 把 max_turns 调低，观察预算耗尽路径。

为什么结果应该“回摘要”，而不是回完整轨迹

主 Agent 委托代码库探索时，真正需要的通常是“关键文件、调用链、证据、风险”，而不是几十次 grep/read 的原始输出。Subagent 独立 context 能把噪声留在侧边窗口，只返回压缩后的高价值状态；这也是 Claude Code 公开文档强调的 context preservation 价值。[R7]

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (504 lines)

```
0001 #!/usr/bin/env python3
0002 """s08 - Add context-isolated custom Subagents."""
0003 from __future__ import annotations
0004
0005 import glob as globlib
0006 import json
0007 import os
0008 import subprocess
0009 import shlex
0010 from enum import Enum
0011 from pathlib import Path
0012 from typing import Any, Callable
0013
0014 from anthropic import Anthropic
0015 from dotenv import load_dotenv
0016 from context_manager import ContextManager
0017 from memory import MemoryStore
0018 from skills import SkillRegistry
0019 from hooks import HookManager
0020 from agents import AgentRegistry
0021
0022 load_dotenv()
0023 MODEL = os.environ.get("ANTHROPIC_MODEL")
0024 if not MODEL:
0025     raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")
0026
0027 client = Anthropic()
0028 WORKSPACE = Path.cwd().resolve()
0029 MAX_TOOL_OUTPUT = 12000
0030 CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
0031 MEMORY = MemoryStore(WORKSPACE)
0032 SKILLS = SkillRegistry(WORKSPACE)
0033 HOOKS = HookManager(WORKSPACE)
0034 AGENTS = AgentRegistry(WORKSPACE)
0035
0036
0037 def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
0038     return {
0039         "name": name,
0040         "description": description,
0041         "input_schema": {
0042             "type": "object",
0043             "properties": properties,
0044             "required": required,
0045             "additionalProperties": False,
0046         },
0047     }
0048
0049
0050 TOOLS = [
0051     tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
0052     tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
0053     tool(
0054         "write_file",
0055         "Create or replace a UTF-8 text file inside the workspace.",
0056         {"path": {"type": "string"}, "content": {"type": "string"}},
0057         ["path", "content"],
0058     ),
0059     tool(
0060         "edit_file",
0061         "Replace one exact text occurrence in a workspace file.",
0062         {
0063             "path": {"type": "string"},
0064             "old_text": {"type": "string"},
0065             "new_text": {"type": "string"},
0066         },
0067         ["path", "old_text", "new_text"],
0068     ),
0069     tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
0070     tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}}, ["note"]),
0071     tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}}, ["query"]),
```

```

0072     tool("use_skill", "Load a reusable task procedure by name when it is relevant.", {"name": {"type": "string"}}, [{"name"}],
0073     tool(
0074         "spawn_subagent",
0075         "Delegate a focused side task to a named subagent with its own context window.",
0076         {"agent": {"type": "string"}, "task": {"type": "string"}},
0077         ["agent", "task"],
0078     ),
0079 ]
0080
0081 BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
0082 Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
0083 Make focused changes and verify them before you finish.
0084 Use remember only for durable facts likely to help future sessions, not transient task state.
0085 """
0086
0087
0088 def build_system() -> str:
0089     startup = MEMORY.startup_context()
0090     catalog = SKILLS.catalog()
0091     parts = [BASE_SYSTEM]
0092     if startup:
0093         parts.append(startup)
0094     parts.append("<available_skills>\n" + catalog + "\n</available_skills>")
0095     parts.append("Load a skill with use_skill only when its procedure is needed.")
0096     parts.append("<available_subagents>\n" + AGENTS.catalog() + "\n</available_subagents>")
0097     parts.append("Delegate noisy research or independent review with spawn_subagent when useful.")
0098     return "\n\n".join(parts)
0099
0100
0101 def safe_path(raw: str) -> Path:
0102     """Resolve a path and reject escapes outside the workspace."""
0103     candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
0104     try:
0105         candidate.relative_to(WORKSPACE)
0106     except ValueError as exc:
0107         raise ValueError(f"Path escapes workspace: {raw!r}") from exc
0108     return candidate
0109
0110
0111 def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
0112     if len(text) <= limit:
0113         return text
0114     half = limit // 2
0115     return text[:half] + "\n...<truncated>...\n" + text[-half:]
0116
0117
0118 def run_bash(command: str) -> str:
0119     try:
0120         proc = subprocess.run(
0121             command,
0122             shell=True,
0123             text=True,
0124             capture_output=True,
0125             cwd=WORKSPACE,
0126             timeout=30,
0127         )
0128     except subprocess.TimeoutExpired:
0129         return "ERROR: command timed out after 30 seconds"
0130     return json.dumps(
0131         {
0132             "exit_code": proc.returncode,
0133             "stdout": truncate(proc.stdout),
0134             "stderr": truncate(proc.stderr),
0135         },
0136         ensure_ascii=False,
0137     )
0138
0139
0140 def run_read(path: str) -> str:
0141     p = safe_path(path)
0142     if not p.is_file():
0143         return f"ERROR: file not found: {path}"
0144     return truncate(p.read_text(encoding="utf-8", errors="replace"))
0145
0146
0147 def run_write(path: str, content: str) -> str:
0148     p = safe_path(path)
0149     p.parent.mkdir(parents=True, exist_ok=True)
0150     p.write_text(content, encoding="utf-8")
0151     return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"

```



```

0152
0153
0154 def run_edit(path: str, old_text: str, new_text: str) -> str:
0155     p = safe_path(path)
0156     if not p.is_file():
0157         return f"ERROR: file not found: {path}"
0158     text = p.read_text(encoding="utf-8", errors="replace")
0159     count = text.count(old_text)
0160     if count == 0:
0161         return "ERROR: old_text not found"
0162     if count > 1:
0163         return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
0164     p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
0165     return f"OK: edited {p.relative_to(WORKSPACE)}"
0166
0167
0168 def run_glob(pattern: str) -> str:
0169     # Resolve matches then apply safe_path to each result to keep output inside the workspace.
0170     raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
0171     matches: list[str] = []
0172     for raw in raw_matches[:500]:
0173         try:
0174             p = safe_path(raw)
0175             except ValueError:
0176                 continue
0177             matches.append(str(p.relative_to(WORKSPACE)))
0178     return "\n".join(sorted(matches)) if matches else "(no matches)"
0179
0180
0181
0182
0183 class Decision(str, Enum):
0184     ALLOW = "allow"
0185     ASK = "ask"
0186     DENY = "deny"
0187
0188
0189 # Gate 1: patterns that should never be auto-approved by this tutorial runtime.
0190 HARD_DENY_SUBSTRINGS = (
0191     "sudo ",
0192     "chmod -R 777 /",
0193     "> /dev/sd",
0194     "mkfs.",
0195 )
0196
0197 # Gate 2: commands considered low-risk and repeatable in a coding workspace.
0198 SAFE_COMMAND_PREFIXES = (
0199     "pwd", "ls", "find ", "git status", "git diff", "git log",
0200     "python -m pytest", "pytest", "npm test", "npm run test",
0201     "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
0202 )
0203
0204
0205 def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
0206     """Return a policy decision and human-readable reason.
0207
0208     Order matters: hard deny -> explicit rules -> ask fallback.
0209     """
0210     if name in {"read_file", "glob", "recall_memory", "use_skill", "spawn_subagent"}:
0211         return Decision.ALLOW, "read-only workspace operation"
0212
0213     if name == "remember":
0214         return Decision.ALLOW, "writes only to the agent memory store"
0215
0216     if name in {"write_file", "edit_file"}:
0217         try:
0218             safe_path(args.get("path", ""))
0219         except Exception as exc:
0220             return Decision.DENY, str(exc)
0221         return Decision.ASK, "file modification inside workspace"
0222
0223     if name == "bash":
0224         command = str(args.get("command", "")).strip()
0225         lowered = command.lower()
0226         if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
0227             return Decision.DENY, "matches hard deny policy"
0228         if command.startswith(SAFE_COMMAND_PREFIXES):
0229             return Decision.ALLOW, "matches low-risk command allowlist"
0230         return Decision.ASK, "shell command is not allowlisted"
0231

```

```

0232     return Decision.ASK, "unknown or extensible tool"
0233
0234
0235 def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
0236     print(f"\n[permission] {name}: {reason}")
0237     print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
0238     while True:
0239         answer = input("Allow once? [y/N] ").strip().lower()
0240         if answer in {"y", "yes"}:
0241             return True
0242         if answer in {"", "n", "no"}:
0243             return False
0244
0245
0246 def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
0247     decision, reason = classify_permission(name, args)
0248     if decision is Decision.DENY:
0249         return False, f"DENIED: {reason}"
0250     if decision is Decision.ALLOW:
0251         return True, reason
0252     if request_approval(name, args, reason):
0253         return True, "approved by user"
0254     return False, "DENIED: user rejected tool call"
0255
0256
0257 TOOL_HANDLERS: dict[str, Callable[..., str]] = {
0258     "bash": run_bash,
0259     "read_file": run_read,
0260     "write_file": run_write,
0261     "edit_file": run_edit,
0262     "glob": run_glob,
0263     "remember": MEMORY.remember,
0264     "recall_memory": MEMORY.search,
0265     "use_skill": SKILLS.load,
0266 }
0267
0268
0269 def block_to_dict(block: Any) -> dict[str, Any]:
0270     if hasattr(block, "model_dump"):
0271         return block.model_dump(exclude_none=True)
0272     return dict(block)
0273
0274
0275 def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
0276     handler = TOOL_HANDLERS.get(name)
0277     if handler is None:
0278         return f"Unknown tool: {name}", True
0279     try:
0280         return handler(**args), False
0281     except Exception as exc:
0282         return f"{type(exc).__name__}: {exc}", True
0283
0284
0285
0286
0287
0288 def run_subagent(agent: str, task: str) -> str:
0289     definition = AGENTS.get(agent)
0290     if definition is None:
0291         return f"ERROR: unknown subagent {agent!r}. Available:\n{AGENTS.catalog()}"
0292
0293     allowed_names = [name for name in definition.tools if name != "spawn_subagent"]
0294     tool_defs = [item for item in TOOLS if item["name"] in allowed_names]
0295     sub_messages: list[dict[str, Any]] = [{"role": "user", "content": task}]
0296     final_text: list[str] = []
0297
0298     HOOKS.fire("subagent_start", {"agent": definition.name, "task": task})
0299     for turn in range(definition.max_turns):
0300         response = client.messages.create(
0301             model=definition.model or MODEL,
0302             max_tokens=3000,
0303             system=(
0304                 build_system()
0305                 + "\n\n<subagent_role>\n"
0306                 + definition.prompt
0307                 + "\n\n</subagent_role>\n"
0308                 + "You are a delegated worker. Do not spawn nested subagents. Return a compact result to the parent."
0309             ),
0310             tools=tool_defs,
0311             messages=sub_messages,

```

```

0312     )
0313     sub_messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0314     final_text = [b.text for b in response.content if getattr(b, "type", None) == "text"]
0315     if response.stop_reason != "tool_use":
0316         break
0317
0318     results: List[dict[str, Any]] = []
0319     for block in response.content:
0320         if getattr(block, "type", None) != "tool_use":
0321             continue
0322         args = dict(block.input)
0323         if block.name not in allowed_names:
0324             content, is_error = f"DENIED: tool {block.name!r} is not allowed for subagent {definition.name}", True
0325         else:
0326             pre = HOOKS.fire(
0327                 "pre_tool_use",
0328                 {"tool_name": block.name, "tool_input": args, "subagent": definition.name},
0329             )
0330             if pre.updated_input is not None:
0331                 args = pre.updated_input
0332             if pre.decision == "deny":
0333                 content, is_error = f"DENIED BY HOOK: {pre.reason}", True
0334             else:
0335                 allowed, reason = check_permission(block.name, args)
0336                 if allowed:
0337                     content, is_error = execute_tool(block.name, args)
0338                 else:
0339                     content, is_error = reason, True
0340             HOOKS.fire(
0341                 "post_tool_use",
0342                 {
0343                     "tool_name": block.name,
0344                     "tool_input": args,
0345                     "tool_output": content,
0346                     "is_error": is_error,
0347                     "subagent": definition.name,
0348                 },
0349             )
0350             results.append(
0351                 {
0352                     "type": "tool_result",
0353                     "tool_use_id": block.id,
0354                     "content": content,
0355                     "is_error": is_error,
0356                 }
0357             )
0358     sub_messages.append({"role": "user", "content": results})
0359     else:
0360         final_text = [f"Subagent reached max_turns={definition.max_turns} before a clean stop."]
0361
0362     result = "\n".join(final_text).strip() or "(subagent returned no text)"
0363     HOOKS.fire("subagent_stop", {"agent": definition.name, "result": result})
0364     return result
0365
0366
0367 # Register after the function exists; this keeps the generic dispatch table from s02.
0368 TOOL_HANDLERS["spawn_subagent"] = run_subagent
0369
0370 def summarize_history(old_messages: List[dict[str, Any]]) -> str:
0371     """Ask the model to compress old state without tools."""
0372     prompt = """Summarize this coding-agent conversation for continuation.
0373 Preserve only durable state:
0374 - user goal and constraints
0375 - repository facts discovered
0376 - files changed and why
0377 - commands/tests run and outcomes
0378 - unresolved hypotheses, TODOs, and errors
0379 - important approvals or denied operations
0380 Do not invent facts. Be concise but concrete.
0381
0382 HISTORY JSON:
0383 """ + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
0384     response = client.messages.create(
0385         model=MODEL,
0386         max_tokens=1800,
0387         system="You compress coding-agent state for a future model turn.",
0388         messages=[{"role": "user", "content": prompt}],
0389     )
0390     return "\n".join(
0391         block.text for block in response.content if getattr(block, "type", None) == "text"

```

```

0392     )
0393
0394
0395 def agent_loop(messages: List[dict[str, Any]]) -> List[dict[str, Any]]:
0396     while True:
0397         if CONTEXT.needs_compaction(messages):
0398             print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
0399             HOOKS.fire("pre_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
0400             messages = CONTEXT.compact(messages, summarize_history)
0401             HOOKS.fire("post_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
0402             response = client.messages.create(
0403                 model=MODEL,
0404                 max_tokens=4096,
0405                 system=build_system(),
0406                 tools=TOOLS,
0407                 messages=messages,
0408             )
0409
0410             for block in response.content:
0411                 if getattr(block, "type", None) == "text":
0412                     print(block.text)
0413
0414             messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0415             if response.stop_reason != "tool_use":
0416                 return messages
0417
0418             results = []
0419             for block in response.content:
0420                 if getattr(block, "type", None) != "tool_use":
0421                     continue
0422                 args = dict(block.input)
0423                 print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
0424
0425                 pre = HOOKS.fire("pre_tool_use", {"tool_name": block.name, "tool_input": args})
0426                 if pre.updated_input is not None:
0427                     args = pre.updated_input
0428                 if pre.decision == "deny":
0429                     content, is_error = f"DENIED BY HOOK: {pre.reason}", True
0430                 else:
0431                     if pre.decision == "ask":
0432                         allowed = request_approval(block.name, args, pre.reason or "hook requested approval")
0433                         permission_reason = "approved by user" if allowed else "DENIED: user rejected hook approval"
0434                     else:
0435                         allowed, permission_reason = check_permission(block.name, args)
0436                     if allowed:
0437                         content, is_error = execute_tool(block.name, args)
0438                     else:
0439                         content, is_error = permission_reason, True
0440
0441                 post = HOOKS.fire(
0442                     "post_tool_use",
0443                     {"tool_name": block.name, "tool_input": args, "tool_output": content, "is_error": is_error},
0444                 )
0445                 if post.output:
0446                     content += "\n[hook output]\n" + post.output.strip()
0447                 results.append(
0448                     {
0449                         "type": "tool_result",
0450                         "tool_use_id": block.id,
0451                         "content": content,
0452                         "is_error": is_error,
0453                     }
0454                 )
0455             messages.append({"role": "user", "content": results})
0456
0457
0458 def main() -> None:
0459     print(f"Mini Claude Code s08 - workspace: {WORKSPACE}")
0460     HOOKS.fire("session_start", {"model": MODEL})
0461     messages: List[dict[str, Any]] = []
0462     while True:
0463         try:
0464             prompt = input("\nyou> ").strip()
0465         except (EOFError, KeyboardInterrupt):
0466             print()
0467             break
0468         if not prompt:
0469             continue
0470         if prompt in {"exit", "/quit"}:
0471             break

```

```

0472     if prompt == "/context":
0473         print(CONTEXT.report(messages))
0474         continue
0475     if prompt == "/compact":
0476         HOOKS.fire("pre_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
0477         messages = CONTEXT.compact(messages, summarize_history)
0478         HOOKS.fire("post_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
0479         print("[context] " + CONTEXT.report(messages))
0480         continue
0481     if prompt == "/clear":
0482         messages = []
0483         print("[context] cleared")
0484         continue
0485     if prompt == "/memory":
0486         print(MEMORY.load_auto_memory() or "(memory is empty)")
0487         continue
0488     if prompt == "/skills":
0489         print(SKILLS.catalog())
0490         continue
0491     if prompt == "/agents":
0492         print(AGENTS.catalog())
0493         continue
0494     messages.append({"role": "user", "content": prompt})
0495     try:
0496         messages = agent_loop(messages)
0497     except Exception as exc:
0498         print(f"ERROR: {exc}")
0499
0500     HOOKS.fire("session_end", {"message_count": len(messages)})
0501
0502
0503 if __name__ == "__main__":
0504     main()

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/explorer.md (13 lines)

```

0001 ---
0002 name: explorer
0003 description: Read-only codebase exploration and evidence gathering.
0004 tools:
0005     - read_file
0006     - glob
0007     - bash
0008 max_turns: 10
0009 ---
0010
0011 You are a read-only exploration specialist.
0012 Do not modify files. Trace code paths, gather evidence, and return a concise report with exact file paths.
0013 Prefer search and focused reads over dumping large files.

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/reviewer.md (12 lines)

```

0001 ---
0002 name: reviewer
0003 description: Review a proposed change for correctness, regressions, and missing tests.
0004 tools:
0005     - read_file
0006     - glob
0007     - bash
0008 max_turns: 10
0009 ---
0010
0011 Review independently. Inspect the diff and relevant tests. Return findings ordered by severity, with evidence.
0012 Do not edit files.

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/audit_hook.py (12 lines)

```

0001 #!/usr/bin/env python3
0002 """Example hook: append compact JSON events to .mini_claude/audit.jsonl."""
0003 import json
0004 import sys
0005 from pathlib import Path
0006
0007 event = json.load(sys.stdin)
0008 path = Path('.mini_claude/audit.jsonl')
0009 path.parent.mkdir(parents=True, exist_ok=True)
0010 with path.open('a', encoding='utf-8') as fh:

```

```

0011 fh.write(json.dumps(event, ensure_ascii=False, default=str) + '\n')
0012 print(json.dumps({"decision": "allow"}))

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/hooks.json (8 Lines)

```

0001 {
0002   "session_start": ["python .mini_claude/audit_hook.py"],
0003   "pre_tool_use": ["python .mini_claude/audit_hook.py"],
0004   "post_tool_use": ["python .mini_claude/audit_hook.py"],
0005   "pre_compact": ["python .mini_claude/audit_hook.py"],
0006   "post_compact": ["python .mini_claude/audit_hook.py"],
0007   "session_end": ["python .mini_claude/audit_hook.py"]
0008 }

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/skills/verify-change/SKILL.md (12 Lines)

```

0001 ---
0002 name: verify-change
0003 description: Verify a code change before declaring the task complete.
0004 ---
0005
0006 When implementation appears complete:
0007 1. inspect `git diff --stat` and `git diff`
0008 2. run the narrowest relevant test first
0009 3. run the broader regression suite if practical
0010 4. run lint/typecheck if the repository has them
0011 5. report exact commands and outcomes
0012 6. flag anything that was not verified

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (11 Lines)

```

0001 # s08 Subagents
0002
0003 Adds `.mini_claude/agents/*.md` definitions and a `spawn_subagent` tool.
0004 Each subagent gets:
0005 - independent message history/context
0006 - a focused role prompt
0007 - an allowlist of tools
0008 - a max-turn budget
0009 - no nested subagent spawning
0010
0011 Use `/agents` to list definitions.

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

agents.py (69 Lines)

```

0001 """Custom subagent definitions for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class AgentDefinition:
0013     name: str
0014     description: str
0015     tools: list[str]
0016     model: str | None
0017     max_turns: int
0018     prompt: str
0019     metadata: dict[str, Any]
0020
0021
0022 class AgentRegistry:
0023     def __init__(self, workspace: Path):
0024         self.workspace = workspace.resolve()
0025         self.root = self.workspace / ".mini_claude" / "agents"
0026
0027     @staticmethod
0028     def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0029         if not text.startswith("---\n"):
0030             return {}, text
0031         end = text.find("\n---\n", 4)

```

```

0032         if end == -1:
0033             return {}, text
0034         meta = yaml.safe_load(text[4:end]) or {}
0035         return (meta if isinstance(meta, dict) else {}), text[end + 5 :].strip()
0036
0037     def discover(self) -> dict[str, AgentDefinition]:
0038         found: dict[str, AgentDefinition] = {}
0039         if not self.root.exists():
0040             return found
0041         for path in sorted(self.root.glob("**.md")):
0042             meta, body = self._split_frontmatter(path.read_text(encoding="utf-8", errors="replace"))
0043             name = str(meta.get("name") or path.stem)
0044             raw_tools = meta.get("tools") or ["read_file", "glob", "bash"]
0045             if isinstance(raw_tools, str):
0046                 raw_tools = [x.strip() for x in raw_tools.split(",") if x.strip()]
0047             tools = [str(x) for x in raw_tools]
0048             found[name] = AgentDefinition(
0049                 name=name,
0050                 description=str(meta.get("description") or "No description provided"),
0051                 tools=tools,
0052                 model=str(meta.get("model") if meta.get("model") else None),
0053                 max_turns=int(meta.get("max_turns", 12)),
0054                 prompt=body,
0055                 metadata=meta,
0056             )
0057         return found
0058
0059     def catalog(self) -> str:
0060         agents = self.discover()
0061         if not agents:
0062             return "(no custom subagents installed)"
0063         return "\n".join(
0064             f"- {name}: {agent.description} [tools={' '.join(agent.tools)}]"
0065             for name, agent in agents.items()
0066         )
0067
0068     def get(self, name: str) -> AgentDefinition | None:
0069         return self.discover().get(name)

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```

0001 """A small context manager: estimate, inspect, and compact message history."""
0002 from __future__ import annotations
0003
0004 import json
0005 from dataclasses import dataclass
0006 from typing import Any, Callable
0007
0008
0009 @dataclass
0010 class ContextStats:
0011     message_count: int
0012     approx_tokens: int
0013     chars: int
0014
0015
0016 class ContextManager:
0017     def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
0018         self.max_approx_tokens = max_approx_tokens
0019         self.keep_recent_messages = keep_recent_messages
0020
0021     @staticmethod
0022     def estimate(messages: list[dict[str, Any]]) -> ContextStats:
0023         raw = json.dumps(messages, ensure_ascii=False, default=str)
0024         # Deliberately simple heuristic. Production systems should use model-aware counting.
0025         return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))
0026
0027     def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
0028         return self.estimate(messages).approx_tokens >= self.max_approx_tokens
0029
0030     def report(self, messages: list[dict[str, Any]]) -> str:
0031         stats = self.estimate(messages)
0032         pct = 100 * stats.approx_tokens / self.max_approx_tokens
0033         return (
0034             f"messages={stats.message_count}, approx_tokens={stats.approx_tokens}, "
0035             f"budget={self.max_approx_tokens}, {pct:.1f}%"
0036         )
0037
0038     def compact(

```

```

0039         self,
0040         messages: List[dict[str, Any]],
0041         summarizer: Callable[[List[dict[str, Any]]], str],
0042     ) -> List[dict[str, Any]]:
0043         if len(messages) <= self.keep_recent_messages + 2:
0044             return messages
0045
0046         cut = len(messages) - self.keep_recent_messages
0047         old = messages[:cut]
0048         recent = messages[cut:]
0049         summary = summarizer(old)
0050         compacted = {
0051             "role": "user",
0052             "content": (
0053                 "[COMPACTED_CONTEXT]\n"
0054                 "The following is a structured summary of earlier conversation state. "
0055                 "Treat it as historical context, not a new user request.\n\n"
0056                 + summary
0057             ),
0058         }
0059         return [compacted, *recent]

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

hooks.py (82 lines)

```

0001 """Deterministic lifecycle hooks for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 import json
0005 import subprocess
0006 from dataclasses import dataclass
0007 from pathlib import Path
0008 from typing import Any
0009
0010
0011 @dataclass
0012 class HookResult:
0013     decision: str = "allow" # allow | deny | ask
0014     reason: str = ""
0015     updated_input: dict[str, Any] | None = None
0016     output: str = ""
0017
0018
0019 class HookManager:
0020     def __init__(self, workspace: Path):
0021         self.workspace = workspace.resolve()
0022         self.config_path = self.workspace / ".mini_claude" / "hooks.json"
0023
0024     def _config(self) -> dict[str, Any]:
0025         if not self.config_path.exists():
0026             return {}
0027         try:
0028             data = json.loads(self.config_path.read_text(encoding="utf-8"))
0029             return data if isinstance(data, dict) else {}
0030         except Exception as exc:
0031             print(f"[hook] invalid config: {exc}")
0032             return {}
0033
0034     def fire(self, event: str, payload: dict[str, Any]) -> HookResult:
0035         commands = self._config().get(event, [])
0036         if isinstance(commands, str):
0037             commands = [commands]
0038         result = HookResult()
0039         for command in commands:
0040             if not isinstance(command, str) or not command.strip():
0041                 continue
0042             envelope = {"event": event, "cwd": str(self.workspace), **payload}
0043             try:
0044                 proc = subprocess.run(
0045                     command,
0046                     shell=True,
0047                     cwd=self.workspace,
0048                     input=json.dumps(envelope, ensure_ascii=False),
0049                     text=True,
0050                     capture_output=True,
0051                     timeout=15,
0052                 )
0053             except subprocess.TimeoutExpired:
0054                 return HookResult(decision="deny", reason=f"hook timed out: {command}")
0055

```



```

0056         if proc.stderr.strip():
0057             print(f"[hook:{event}:stderr] {proc.stderr.strip()[:2000]}")
0058         if proc.returncode != 0:
0059             return HookResult(decision="deny", reason=f"hook failed ({proc.returncode}): {command}")
0060
0061         stdout = proc.stdout.strip()
0062         if not stdout:
0063             continue
0064         try:
0065             data = json.loads(stdout)
0066         except json.JSONDecodeError:
0067             result.output += stdout + "\n"
0068             continue
0069
0070         if isinstance(data, dict):
0071             decision = str(data.get("decision", "allow"))
0072             if decision in {"allow", "deny", "ask"}:
0073                 result.decision = decision
0074             if data.get("reason"):
0075                 result.reason = str(data["reason"])
0076             if isinstance(data.get("updated_input"), dict):
0077                 result.updated_input = data["updated_input"]
0078             if data.get("output"):
0079                 result.output += str(data["output"]) + "\n"
0080             if result.decision == "deny":
0081                 return result
0082         return result

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```

0001 """Persistent project instructions + agent-written memory for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from pathlib import Path
0005 from typing import Iterable
0006
0007
0008 class MemoryStore:
0009     def __init__(self, workspace: Path):
0010         self.workspace = workspace.resolve()
0011         self.project_instructions = self.workspace / "CLAUDE.md"
0012         self.memory_dir = self.workspace / ".mini_claude" / "memory"
0013         self.memory_file = self.memory_dir / "MEMORY.md"
0014
0015     def load_project_instructions(self) -> str:
0016         if not self.project_instructions.exists():
0017             return ""
0018         return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]
0019
0020     def load_auto_memory(self) -> str:
0021         if not self.memory_file.exists():
0022             return ""
0023         # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
0024         text = self.memory_file.read_text(encoding="utf-8", errors="replace")
0025         return "\n".join(text.splitlines()[:200])[:25000]
0026
0027     def remember(self, note: str) -> str:
0028         note = note.strip()
0029         if not note:
0030             return "ERROR: empty memory note"
0031         self.memory_dir.mkdir(parents=True, exist_ok=True)
0032         existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
0033         normalized = f"- {note}"
0034         if normalized in existing:
0035             return "OK: note already present"
0036         with self.memory_file.open("a", encoding="utf-8") as fh:
0037             if not existing.endswith("\n"):
0038                 fh.write("\n")
0039             fh.write(normalized + "\n")
0040         return "OK: persisted memory"
0041
0042     def search(self, query: str, limit: int = 20) -> str:
0043         if not self.memory_file.exists():
0044             return "(memory is empty)"
0045         q = query.casefold().strip()
0046         lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
0047         hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
0048         selected = list(hits)[:limit]
0049         return "\n".join(selected) if selected else "(no memory matches)"

```

```

0050
0051     def startup_context(self) -> str:
0052         parts = []
0053         project = self.load_project_instructions().strip()
0054         memory = self.load_auto_memory().strip()
0055         if project:
0056             parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
0057         if memory:
0058             parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
0059         return "\n\n".join(parts)

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

skills.py (70 lines)

```

0001 """Progressive-disclosure skills for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class Skill:
0013     name: str
0014     description: str
0015     path: Path
0016     body: str
0017     metadata: dict[str, Any]
0018
0019
0020 class SkillRegistry:
0021     def __init__(self, workspace: Path):
0022         self.workspace = workspace.resolve()
0023         self.roots = [
0024             self.workspace / ".mini_claude" / "skills",
0025             self.workspace / ".claude" / "skills", # optional compatibility
0026         ]
0027
0028     @staticmethod
0029     def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0030         if not text.startswith("---\n"):
0031             return {}, text
0032         end = text.find("\n---\n", 4)
0033         if end == -1:
0034             return {}, text
0035         raw = text[4:end]
0036         body = text[end + 5 :]
0037         data = yaml.safe_load(raw) or {}
0038         if not isinstance(data, dict):
0039             data = {}
0040         return data, body
0041
0042     def discover(self) -> dict[str, Skill]:
0043         found: dict[str, Skill] = {}
0044         for root in self.roots:
0045             if not root.exists():
0046                 continue
0047             for skill_file in sorted(root.glob("*/SKILL.md")):
0048                 text = skill_file.read_text(encoding="utf-8", errors="replace")
0049                 meta, body = self._split_frontmatter(text)
0050                 name = str(meta.get("name") or skill_file.parent.name)
0051                 description = str(meta.get("description") or "No description provided")
0052                 found[name] = Skill(name, description, skill_file, body.strip(), meta)
0053         return found
0054
0055     def catalog(self) -> str:
0056         skills = self.discover()
0057         if not skills:
0058             return "(no skills installed)"
0059         return "\n".join(f"- {name}: {skill.description}" for name, skill in skills.items())
0060
0061     def load(self, name: str) -> str:
0062         skill = self.discover().get(name)
0063         if skill is None:
0064             return f"ERROR: unknown skill {name!r}. Available:\n{self.catalog()}"
0065         # A skill can reference support files by relative path; normal file tools can read them later.
0066         return (

```

```
0067         f"# Skill: {skill.name}\n"
0068         f"Source: {skill.path.relative_to(self.workspace)}\n\n"
0069         f"{skill.body[:30000]}"
0070     )
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

10. s09 - Sandbox

把模型判断之外再加一层执行边界

观察项	本阶段
阶段目录	s09_sandbox
文件数量	13
新增文件	sandbox.py
继承/修改文件	.mini_claude/agents/explorer.md, .mini_claude/agents/reviewer.md, .mini_claude/audit_hook.py, .mini_claude/hooks.json, .mini_claude/skills/verify-change/SKILL.md, README.md, agents.py, code.py ...
Python 行数	964

KNOW-WHY：为什么这一层必须存在

- Permission 是“这次是否允许做”，Sandbox 是“即使允许，最多能碰到哪里”。两者解决不同问题。
- 模型判断和审批都可能出错，操作系统级 containment 可以把最坏情况限制在更小 blast radius。
- shell 是 Coding Agent 最强也最危险的通用工具，因此先对 shell 建边界最有收益。

KNOW-HOW：这一层具体怎么长出来

- 33. 新增 SandboxRunner。soft 模式做 cwd 固定、环境变量净化、timeout；明确声明它不是强安全边界。
- 34. Linux 环境若存在 Bubblewrap，则 bwrap 模式把工作区作为可写目录、系统目录只读，并可 unshare network。
- 35. Shell 子进程不继承 ANTHROPIC_API_KEY 等敏感环境变量。
- 36. 加入 sandbox_status 和 /sandbox，避免用户误以为当前处于强隔离。

数据流 / 控制流

Permission allow -> SandboxRunner -> sanitized env / cwd / OS boundary -> command result

最容易踩的坑

- 把“cwd 在工作区”误认为文件系统隔离；普通进程仍可能访问其他路径。
- Bubblewrap 配置依赖 Linux 发行版，生产环境要针对依赖、设备、网络白名单做更严格设计。
- 网络完全关闭会让 package manager、测试依赖等失败；生产系统通常需要代理/allowlist，而非简单全开。

本阶段验收清单

- 运行 /sandbox 查看 active mode。
- 确认 shell 环境中看不到 API key。
- 在支持 bwrap 的 Linux 上验证网络/路径边界。

Permission 与 Sandbox 的二维模型

层	回答的问题	失后果
Permission	这次调用允许吗？	错误批准可能产生副作用
Sandbox	即使运行，最多能访问哪里？	限制错误批准的 blast radius

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (511 lines)

```
0001 #!/usr/bin/env python3
0002 """s09 - Add a sandboxed shell execution boundary."""
0003 from __future__ import annotations
0004
0005 import glob as globlib
0006 import json
0007 import os
0008 import subprocess
0009 import shlex
0010 from enum import Enum
0011 from pathlib import Path
0012 from typing import Any, Callable
0013
0014 from anthropic import Anthropic
0015 from dotenv import load_dotenv
0016 from context_manager import ContextManager
0017 from memory import MemoryStore
0018 from skills import SkillRegistry
0019 from hooks import HookManager
0020 from agents import AgentRegistry
0021 from sandbox import SandboxRunner
0022
0023 load_dotenv()
0024 MODEL = os.environ.get("ANTHROPIC_MODEL")
0025 if not MODEL:
0026     raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")
0027
0028 client = Anthropic()
0029 WORKSPACE = Path.cwd().resolve()
0030 MAX_TOOL_OUTPUT = 12000
0031 CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
0032 MEMORY = MemoryStore(WORKSPACE)
0033 SKILLS = SkillRegistry(WORKSPACE)
0034 HOOKS = HookManager(WORKSPACE)
0035 AGENTS = AgentRegistry(WORKSPACE)
0036 SANDBOX = SandboxRunner(
0037     WORKSPACE,
0038     mode=os.getenv("MINICLAUDE_SANDBOX", "auto"),
0039     disable_network=os.getenv("MINICLAUDE_ALLOW_NETWORK", "0") != "1",
0040 )
0041
0042
0043 def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
0044     return {
0045         "name": name,
0046         "description": description,
0047         "input_schema": {
0048             "type": "object",
0049             "properties": properties,
0050             "required": required,
0051             "additionalProperties": False,
0052         },
0053     }
0054
0055
0056 TOOLS = [
0057     tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
0058     tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
0059     tool(
0060         "write_file",
0061         "Create or replace a UTF-8 text file inside the workspace.",
0062         {"path": {"type": "string"}, "content": {"type": "string"}},
0063         ["path", "content"],
0064     ),
0065     tool(
0066         "edit_file",
0067         "Replace one exact text occurrence in a workspace file.",
0068         {
```

```

0069         "path": {"type": "string"},
0070         "old_text": {"type": "string"},
0071         "new_text": {"type": "string"},
0072     },
0073     ["path", "old_text", "new_text"],
0074 ),
0075 tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
0076 tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}}, ["note"]),
0077 tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}}, ["query"]),
0078 tool("use_skill", "Load a reusable task procedure by name when it is relevant.", {"name": {"type": "string"}}, ["name"]),
0079 tool(
0080     "spawn_subagent",
0081     "Delegate a focused side task to a named subagent with its own context window.",
0082     {"agent": {"type": "string"}, "task": {"type": "string"}},
0083     ["agent", "task"],
0084 ),
0085 tool("sandbox_status", "Report the active shell sandbox mode.", {}, []),
0086 ]
0087
0088 BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
0089 Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
0090 Make focused changes and verify them before you finish.
0091 Use remember only for durable facts likely to help future sessions, not transient task state.
0092 """
0093
0094
0095 def build_system() -> str:
0096     startup = MEMORY.startup_context()
0097     catalog = SKILLS.catalog()
0098     parts = [BASE_SYSTEM]
0099     if startup:
0100         parts.append(startup)
0101     parts.append("<available_skills>\n" + catalog + "\n</available_skills>")
0102     parts.append("Load a skill with use_skill only when its procedure is needed.")
0103     parts.append("<available_subagents>\n" + AGENTS.catalog() + "\n</available_subagents>")
0104     parts.append("Delegate noisy research or independent review with spawn_subagent when useful.")
0105     return "\n\n".join(parts)
0106
0107
0108 def safe_path(raw: str) -> Path:
0109     """Resolve a path and reject escapes outside the workspace."""
0110     candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
0111     try:
0112         candidate.relative_to(WORKSPACE)
0113     except ValueError as exc:
0114         raise ValueError(f"Path escapes workspace: {raw!r}") from exc
0115     return candidate
0116
0117
0118 def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
0119     if len(text) <= limit:
0120         return text
0121     half = limit // 2
0122     return text[:half] + "\n...<truncated>...\n" + text[-half:]
0123
0124
0125 def run_bash(command: str) -> str:
0126     try:
0127         result = SANDBOX.run(command, timeout=30)
0128     except subprocess.TimeoutExpired:
0129         return "ERROR: command timed out after 30 seconds"
0130     except Exception as exc:
0131         return f"ERROR: sandbox execution failed: {type(exc).__name__}: {exc}"
0132     return json.dumps(
0133         {
0134             "sandbox": result.mode,
0135             "exit_code": result.exit_code,
0136             "stdout": truncate(result.stdout),
0137             "stderr": truncate(result.stderr),
0138         },
0139         ensure_ascii=False,
0140     )
0141
0142
0143 def run_read(path: str) -> str:
0144     p = safe_path(path)
0145     if not p.is_file():
0146         return f"ERROR: file not found: {path}"
0147     return truncate(p.read_text(encoding="utf-8", errors="replace"))
0148

```

```

0149
0150 def run_write(path: str, content: str) -> str:
0151     p = safe_path(path)
0152     p.parent.mkdir(parents=True, exist_ok=True)
0153     p.write_text(content, encoding="utf-8")
0154     return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"
0155
0156
0157 def run_edit(path: str, old_text: str, new_text: str) -> str:
0158     p = safe_path(path)
0159     if not p.is_file():
0160         return f"ERROR: file not found: {path}"
0161     text = p.read_text(encoding="utf-8", errors="replace")
0162     count = text.count(old_text)
0163     if count == 0:
0164         return "ERROR: old_text not found"
0165     if count > 1:
0166         return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
0167     p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
0168     return f"OK: edited {p.relative_to(WORKSPACE)}"
0169
0170
0171 def run_glob(pattern: str) -> str:
0172     # Resolve matches then apply safe_path to each result to keep output inside the workspace.
0173     raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
0174     matches: list[str] = []
0175     for raw in raw_matches[:500]:
0176         try:
0177             p = safe_path(raw)
0178         except ValueError:
0179             continue
0180         matches.append(str(p.relative_to(WORKSPACE)))
0181     return "\n".join(sorted(matches)) if matches else "(no matches)"
0182
0183
0184
0185
0186 class Decision(str, Enum):
0187     ALLOW = "allow"
0188     ASK = "ask"
0189     DENY = "deny"
0190
0191
0192 # Gate 1: patterns that should never be auto-approved by this tutorial runtime.
0193 HARD_DENY_SUBSTRINGS = (
0194     "sudo ",
0195     "chmod -R 777 /",
0196     "> /dev/sd",
0197     "mkfs.",
0198 )
0199
0200 # Gate 2: commands considered low-risk and repeatable in a coding workspace.
0201 SAFE_COMMAND_PREFIXES = (
0202     "pwd", "ls", "find ", "git status", "git diff", "git log",
0203     "python -m pytest", "pytest", "npm test", "npm run test",
0204     "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
0205 )
0206
0207
0208 def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
0209     """Return a policy decision and human-readable reason.
0210
0211     Order matters: hard deny -> explicit rules -> ask fallback.
0212     """
0213     if name in {"read_file", "glob", "recall_memory", "use_skill", "spawn_subagent", "sandbox_status"}:
0214         return Decision.ALLOW, "read-only workspace operation"
0215
0216     if name == "remember":
0217         return Decision.ALLOW, "writes only to the agent memory store"
0218
0219     if name in {"write_file", "edit_file"}:
0220         try:
0221             safe_path(args.get("path", ""))
0222         except Exception as exc:
0223             return Decision.DENY, str(exc)
0224         return Decision.ASK, "file modification inside workspace"
0225
0226     if name == "bash":
0227         command = str(args.get("command", "")).strip()
0228         lowered = command.lower()

```

```

0229         if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
0230             return Decision.DENY, "matches hard deny policy"
0231         if command.startswith(SAFE_COMMAND_PREFIXES):
0232             return Decision.ALLOW, "matches low-risk command allowlist"
0233         return Decision.ASK, "shell command is not allowlisted"
0234
0235     return Decision.ASK, "unknown or extensible tool"
0236
0237
0238 def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
0239     print(f"\n[permission] {name}: {reason}")
0240     print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
0241     while True:
0242         answer = input("Allow once? [y/N] ").strip().lower()
0243         if answer in {"y", "yes"}:
0244             return True
0245         if answer in {"", "n", "no"}:
0246             return False
0247
0248
0249 def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
0250     decision, reason = classify_permission(name, args)
0251     if decision is Decision.DENY:
0252         return False, f"DENIED: {reason}"
0253     if decision is Decision.ALLOW:
0254         return True, reason
0255     if request_approval(name, args, reason):
0256         return True, "approved by user"
0257     return False, "DENIED: user rejected tool call"
0258
0259
0260 TOOL_HANDLERS: dict[str, Callable[..., str]] = {
0261     "bash": run_bash,
0262     "read_file": run_read,
0263     "write_file": run_write,
0264     "edit_file": run_edit,
0265     "glob": run_glob,
0266     "remember": MEMORY.remember,
0267     "recall_memory": MEMORY.search,
0268     "use_skill": SKILLS.load,
0269     "sandbox_status": lambda: SANDBOX.status(),
0270 }
0271
0272
0273 def block_to_dict(block: Any) -> dict[str, Any]:
0274     if hasattr(block, "model_dump"):
0275         return block.model_dump(exclude_none=True)
0276     return dict(block)
0277
0278
0279 def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
0280     handler = TOOL_HANDLERS.get(name)
0281     if handler is None:
0282         return f"Unknown tool: {name}", True
0283     try:
0284         return handler(**args), False
0285     except Exception as exc:
0286         return f"{type(exc).__name__}: {exc}", True
0287
0288
0289
0290
0291
0292 def run_subagent(agent: str, task: str) -> str:
0293     definition = AGENTS.get(agent)
0294     if definition is None:
0295         return f"ERROR: unknown subagent {agent!r}. Available:\n{AGENTS.catalog()}"
0296
0297     allowed_names = [name for name in definition.tools if name != "spawn_subagent"]
0298     tool_defs = [item for item in TOOLS if item["name"] in allowed_names]
0299     sub_messages: list[dict[str, Any]] = [{"role": "user", "content": task}]
0300     final_text: list[str] = []
0301
0302     HOOKS.fire("subagent_start", {"agent": definition.name, "task": task})
0303     for turn in range(definition.max_turns):
0304         response = client.messages.create(
0305             model=definition.model or MODEL,
0306             max_tokens=3000,
0307             system=(
0308                 build_system()

```



```

0309         + "\n\n<subagent_role>\n"
0310         + definition.prompt
0311         + "\n\n</subagent_role>\n"
0312         + "You are a delegated worker. Do not spawn nested subagents. Return a compact result to the parent."
0313     ),
0314     tools=tool_defs,
0315     messages=sub_messages,
0316 )
0317 sub_messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0318 final_text = [b.text for b in response.content if getattr(b, "type", None) == "text"]
0319 if response.stop_reason != "tool_use":
0320     break
0321
0322 results: List[dict[str, Any]] = []
0323 for block in response.content:
0324     if getattr(block, "type", None) != "tool_use":
0325         continue
0326     args = dict(block.input)
0327     if block.name not in allowed_names:
0328         content, is_error = f"DENIED: tool {block.name!r} is not allowed for subagent {definition.name}", True
0329     else:
0330         pre = HOOKS.fire(
0331             "pre_tool_use",
0332             {"tool_name": block.name, "tool_input": args, "subagent": definition.name},
0333         )
0334         if pre.updated_input is not None:
0335             args = pre.updated_input
0336         if pre.decision == "deny":
0337             content, is_error = f"DENIED BY HOOK: {pre.reason}", True
0338         else:
0339             allowed, reason = check_permission(block.name, args)
0340             if allowed:
0341                 content, is_error = execute_tool(block.name, args)
0342             else:
0343                 content, is_error = reason, True
0344         HOOKS.fire(
0345             "post_tool_use",
0346             {
0347                 "tool_name": block.name,
0348                 "tool_input": args,
0349                 "tool_output": content,
0350                 "is_error": is_error,
0351                 "subagent": definition.name,
0352             },
0353         )
0354         results.append(
0355             {
0356                 "type": "tool_result",
0357                 "tool_use_id": block.id,
0358                 "content": content,
0359                 "is_error": is_error,
0360             }
0361         )
0362     sub_messages.append({"role": "user", "content": results})
0363 else:
0364     final_text = [f"Subagent reached max_turns={definition.max_turns} before a clean stop."]
0365
0366 result = "\n".join(final_text).strip() or "(subagent returned no text)"
0367 HOOKS.fire("subagent_stop", {"agent": definition.name, "result": result})
0368 return result
0369
0370
0371 # Register after the function exists; this keeps the generic dispatch table from s02.
0372 TOOL_HANDLERS["spawn_subagent"] = run_subagent
0373
0374 def summarize_history(old_messages: List[dict[str, Any]]) -> str:
0375     """Ask the model to compress old state without tools."""
0376     prompt = """Summarize this coding-agent conversation for continuation.
0377 Preserve only durable state:
0378 - user goal and constraints
0379 - repository facts discovered
0380 - files changed and why
0381 - commands/tests run and outcomes
0382 - unresolved hypotheses, TODOs, and errors
0383 - important approvals or denied operations
0384 Do not invent facts. Be concise but concrete.
0385
0386 HISTORY JSON:
0387 """ + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
0388 response = client.messages.create(

```

```

0389     model=MODEL,
0390     max_tokens=1800,
0391     system="You compress coding-agent state for a future model turn.",
0392     messages=[{"role": "user", "content": prompt}],
0393 )
0394 return "\n".join(
0395     block.text for block in response.content if getattr(block, "type", None) == "text"
0396 )
0397
0398
0399 def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
0400     while True:
0401         if CONTEXT.needs_compaction(messages):
0402             print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
0403             HOOKS.fire("pre_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
0404             messages = CONTEXT.compact(messages, summarize_history)
0405             HOOKS.fire("post_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
0406             response = client.messages.create(
0407                 model=MODEL,
0408                 max_tokens=4096,
0409                 system=build_system(),
0410                 tools=TOOLS,
0411                 messages=messages,
0412             )
0413
0414             for block in response.content:
0415                 if getattr(block, "type", None) == "text":
0416                     print(block.text)
0417
0418             messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0419             if response.stop_reason != "tool_use":
0420                 return messages
0421
0422             results = []
0423             for block in response.content:
0424                 if getattr(block, "type", None) != "tool_use":
0425                     continue
0426                 args = dict(block.input)
0427                 print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
0428
0429                 pre = HOOKS.fire("pre_tool_use", {"tool_name": block.name, "tool_input": args})
0430                 if pre.updated_input is not None:
0431                     args = pre.updated_input
0432                 if pre.decision == "deny":
0433                     content, is_error = f"DENIED BY HOOK: {pre.reason}", True
0434                 else:
0435                     if pre.decision == "ask":
0436                         allowed = request_approval(block.name, args, pre.reason or "hook requested approval")
0437                         permission_reason = "approved by user" if allowed else "DENIED: user rejected hook approval"
0438                     else:
0439                         allowed, permission_reason = check_permission(block.name, args)
0440                     if allowed:
0441                         content, is_error = execute_tool(block.name, args)
0442                     else:
0443                         content, is_error = permission_reason, True
0444
0445                 post = HOOKS.fire(
0446                     "post_tool_use",
0447                     {"tool_name": block.name, "tool_input": args, "tool_output": content, "is_error": is_error},
0448                 )
0449                 if post.output:
0450                     content += "\n[hook output]\n" + post.output.strip()
0451                 results.append(
0452                     {
0453                         "type": "tool_result",
0454                         "tool_use_id": block.id,
0455                         "content": content,
0456                         "is_error": is_error,
0457                     }
0458                 )
0459             messages.append({"role": "user", "content": results})
0460
0461
0462 def main() -> None:
0463     print(f"Mini Claude Code s09 - workspace: {WORKSPACE}")
0464     HOOKS.fire("session_start", {"model": MODEL})
0465     messages: list[dict[str, Any]] = []
0466     while True:
0467         try:
0468             prompt = input("\nyou> ").strip()

```

```

0469     except (EOFError, KeyboardInterrupt):
0470         print()
0471         break
0472     if not prompt:
0473         continue
0474     if prompt in {"exit", "/quit"}:
0475         break
0476     if prompt == "/context":
0477         print(CONTEXT.report(messages))
0478         continue
0479     if prompt == "/compact":
0480         HOOKS.fire("pre_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
0481         messages = CONTEXT.compact(messages, summarize_history)
0482         HOOKS.fire("post_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
0483         print("[context] " + CONTEXT.report(messages))
0484         continue
0485     if prompt == "/clear":
0486         messages = []
0487         print("[context] cleared")
0488         continue
0489     if prompt == "/memory":
0490         print(MEMORY.load_auto_memory() or "(memory is empty)")
0491         continue
0492     if prompt == "/skills":
0493         print(SKILLS.catalog())
0494         continue
0495     if prompt == "/agents":
0496         print(AGENTS.catalog())
0497         continue
0498     if prompt == "/sandbox":
0499         print(SANDBOX.status())
0500         continue
0501     messages.append({"role": "user", "content": prompt})
0502     try:
0503         messages = agent_loop(messages)
0504     except Exception as exc:
0505         print(f"ERROR: {exc}")
0506
0507     HOOKS.fire("session_end", {"message_count": len(messages)})
0508
0509
0510 if __name__ == "__main__":
0511     main()

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/explorer.md (13 lines)

```

0001 ---
0002 name: explorer
0003 description: Read-only codebase exploration and evidence gathering.
0004 tools:
0005     - read_file
0006     - glob
0007     - bash
0008 max_turns: 10
0009 ---
0010
0011 You are a read-only exploration specialist.
0012 Do not modify files. Trace code paths, gather evidence, and return a concise report with exact file paths.
0013 Prefer search and focused reads over dumping large files.

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/reviewer.md (12 lines)

```

0001 ---
0002 name: reviewer
0003 description: Review a proposed change for correctness, regressions, and missing tests.
0004 tools:
0005     - read_file
0006     - glob
0007     - bash
0008 max_turns: 10
0009 ---
0010
0011 Review independently. Inspect the diff and relevant tests. Return findings ordered by severity, with evidence.
0012 Do not edit files.

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/audit_hook.py (12 lines)

```
0001 #!/usr/bin/env python3
0002 """Example hook: append compact JSON events to .mini_claude/audit.jsonl."""
0003 import json
0004 import sys
0005 from pathlib import Path
0006
0007 event = json.load(sys.stdin)
0008 path = Path('.mini_claude/audit.jsonl')
0009 path.parent.mkdir(parents=True, exist_ok=True)
0010 with path.open('a', encoding='utf-8') as fh:
0011     fh.write(json.dumps(event, ensure_ascii=False, default=str) + '\n')
0012 print(json.dumps({"decision": "allow"}))
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/hooks.json (8 lines)

```
0001 {
0002     "session_start": ["python .mini_claude/audit_hook.py"],
0003     "pre_tool_use": ["python .mini_claude/audit_hook.py"],
0004     "post_tool_use": ["python .mini_claude/audit_hook.py"],
0005     "pre_compact": ["python .mini_claude/audit_hook.py"],
0006     "post_compact": ["python .mini_claude/audit_hook.py"],
0007     "session_end": ["python .mini_claude/audit_hook.py"]
0008 }
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/skills/verify-change/SKILL.md (12 lines)

```
0001 ---
0002 name: verify-change
0003 description: Verify a code change before declaring the task complete.
0004 ---
0005
0006 When implementation appears complete:
0007 1. inspect `git diff --stat` and `git diff`
0008 2. run the narrowest relevant test first
0009 3. run the broader regression suite if practical
0010 4. run lint/typecheck if the repository has them
0011 5. report exact commands and outcomes
0012 6. flag anything that was not verified
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (11 lines)

```
0001 # s09 Sandbox
0002
0003 Adds a shell execution boundary:
0004 - `MINICLAUDE_SANDBOX=auto|soft|bwrap`
0005 - sanitized environment (API keys are not passed to commands)
0006 - workspace cwd
0007 - timeout
0008 - optional Bubblewrap filesystem/network isolation on Linux
0009
0010 Important: `soft` mode is a policy convenience, not a strong OS security boundary.
0011 Use `/sandbox` to inspect the active mode.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

agents.py (69 lines)

```
0001 """Custom subagent definitions for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class AgentDefinition:
0013     name: str
0014     description: str
0015     tools: list[str]
0016     model: str | None
0017     max_turns: int
0018     prompt: str
0019     metadata: dict[str, Any]
```

```

0020
0021
0022 class AgentRegistry:
0023     def __init__(self, workspace: Path):
0024         self.workspace = workspace.resolve()
0025         self.root = self.workspace / ".mini_claude" / "agents"
0026
0027     @staticmethod
0028     def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0029         if not text.startswith("---\n"):
0030             return {}, text
0031         end = text.find("\n---\n", 4)
0032         if end == -1:
0033             return {}, text
0034         meta = yaml.safe_load(text[4:end]) or {}
0035         return (meta if isinstance(meta, dict) else {}), text[end + 5:].strip()
0036
0037     def discover(self) -> dict[str, AgentDefinition]:
0038         found: dict[str, AgentDefinition] = {}
0039         if not self.root.exists():
0040             return found
0041         for path in sorted(self.root.glob("*.md")):
0042             meta, body = self._split_frontmatter(path.read_text(encoding="utf-8", errors="replace"))
0043             name = str(meta.get("name") or path.stem)
0044             raw_tools = meta.get("tools") or ["read_file", "glob", "bash"]
0045             if isinstance(raw_tools, str):
0046                 raw_tools = [x.strip() for x in raw_tools.split(",") if x.strip()]
0047             tools = [str(x) for x in raw_tools]
0048             found[name] = AgentDefinition(
0049                 name=name,
0050                 description=str(meta.get("description") or "No description provided"),
0051                 tools=tools,
0052                 model=str(meta.get("model")) if meta.get("model") else None,
0053                 max_turns=int(meta.get("max_turns", 12)),
0054                 prompt=body,
0055                 metadata=meta,
0056             )
0057         return found
0058
0059     def catalog(self) -> str:
0060         agents = self.discover()
0061         if not agents:
0062             return "(no custom subagents installed)"
0063         return "\n".join(
0064             f"- {name}: {agent.description} [tools='{','.join(agent.tools)}']"
0065             for name, agent in agents.items()
0066         )
0067
0068     def get(self, name: str) -> AgentDefinition | None:
0069         return self.discover().get(name)

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```

0001 """A small context manager: estimate, inspect, and compact message history."""
0002 from __future__ import annotations
0003
0004 import json
0005 from dataclasses import dataclass
0006 from typing import Any, Callable
0007
0008
0009 @dataclass
0010 class ContextStats:
0011     message_count: int
0012     approx_tokens: int
0013     chars: int
0014
0015
0016 class ContextManager:
0017     def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
0018         self.max_approx_tokens = max_approx_tokens
0019         self.keep_recent_messages = keep_recent_messages
0020
0021     @staticmethod
0022     def estimate(messages: list[dict[str, Any]]) -> ContextStats:
0023         raw = json.dumps(messages, ensure_ascii=False, default=str)
0024         # Deliberately simple heuristic. Production systems should use model-aware counting.
0025         return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))
0026

```

```

0027     def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
0028         return self.estimate(messages).approx_tokens >= self.max_approx_tokens
0029
0030     def report(self, messages: list[dict[str, Any]]) -> str:
0031         stats = self.estimate(messages)
0032         pct = 100 * stats.approx_tokens / self.max_approx_tokens
0033         return (
0034             f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:}, "
0035             f"budget={self.max_approx_tokens:}, ({pct:.1f}%)"
0036         )
0037
0038     def compact(
0039         self,
0040         messages: list[dict[str, Any]],
0041         summarizer: Callable[[list[dict[str, Any]]], str],
0042     ) -> list[dict[str, Any]]:
0043         if len(messages) <= self.keep_recent_messages + 2:
0044             return messages
0045
0046         cut = len(messages) - self.keep_recent_messages
0047         old = messages[:cut]
0048         recent = messages[cut:]
0049         summary = summarizer(old)
0050         compacted = {
0051             "role": "user",
0052             "content": (
0053                 "[COMPACTED_CONTEXT]\n"
0054                 "The following is a structured summary of earlier conversation state. "
0055                 "Treat it as historical context, not a new user request.\n\n"
0056                 + summary
0057             ),
0058         }
0059         return [compacted, *recent]

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

hooks.py (82 lines)

```

0001 """Deterministic lifecycle hooks for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 import json
0005 import subprocess
0006 from dataclasses import dataclass
0007 from pathlib import Path
0008 from typing import Any
0009
0010
0011 @dataclass
0012 class HookResult:
0013     decision: str = "allow" # allow | deny | ask
0014     reason: str = ""
0015     updated_input: dict[str, Any] | None = None
0016     output: str = ""
0017
0018
0019 class HookManager:
0020     def __init__(self, workspace: Path):
0021         self.workspace = workspace.resolve()
0022         self.config_path = self.workspace / ".mini_claude" / "hooks.json"
0023
0024     def _config(self) -> dict[str, Any]:
0025         if not self.config_path.exists():
0026             return {}
0027         try:
0028             data = json.loads(self.config_path.read_text(encoding="utf-8"))
0029             return data if isinstance(data, dict) else {}
0030         except Exception as exc:
0031             print(f"[hook] invalid config: {exc}")
0032             return {}
0033
0034     def fire(self, event: str, payload: dict[str, Any]) -> HookResult:
0035         commands = self._config().get(event, [])
0036         if isinstance(commands, str):
0037             commands = [commands]
0038         result = HookResult()
0039         for command in commands:
0040             if not isinstance(command, str) or not command.strip():
0041                 continue
0042             envelope = {"event": event, "cwd": str(self.workspace), **payload}
0043             try:

```

```

0044         proc = subprocess.run(
0045             command,
0046             shell=True,
0047             cwd=self.workspace,
0048             input=json.dumps(envelope, ensure_ascii=False),
0049             text=True,
0050             capture_output=True,
0051             timeout=15,
0052         )
0053     except subprocess.TimeoutExpired:
0054         return HookResult(decision="deny", reason=f"hook timed out: {command}")
0055
0056     if proc.stderr.strip():
0057         print(f"[hook:{event}:stderr] {proc.stderr.strip()[:2000]}")
0058     if proc.returncode != 0:
0059         return HookResult(decision="deny", reason=f"hook failed ({proc.returncode}): {command}")
0060
0061     stdout = proc.stdout.strip()
0062     if not stdout:
0063         continue
0064     try:
0065         data = json.loads(stdout)
0066     except json.JSONDecodeError:
0067         result.output += stdout + "\n"
0068         continue
0069
0070     if isinstance(data, dict):
0071         decision = str(data.get("decision", "allow"))
0072         if decision in {"allow", "deny", "ask"}:
0073             result.decision = decision
0074         if data.get("reason"):
0075             result.reason = str(data["reason"])
0076         if isinstance(data.get("updated_input"), dict):
0077             result.updated_input = data["updated_input"]
0078         if data.get("output"):
0079             result.output += str(data["output"]) + "\n"
0080         if result.decision == "deny":
0081             return result
0082     return result

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```

0001 """Persistent project instructions + agent-written memory for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from pathlib import Path
0005 from typing import Iterable
0006
0007
0008 class MemoryStore:
0009     def __init__(self, workspace: Path):
0010         self.workspace = workspace.resolve()
0011         self.project_instructions = self.workspace / "CLAUDE.md"
0012         self.memory_dir = self.workspace / ".mini_claude" / "memory"
0013         self.memory_file = self.memory_dir / "MEMORY.md"
0014
0015     def load_project_instructions(self) -> str:
0016         if not self.project_instructions.exists():
0017             return ""
0018         return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]
0019
0020     def load_auto_memory(self) -> str:
0021         if not self.memory_file.exists():
0022             return ""
0023         # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
0024         text = self.memory_file.read_text(encoding="utf-8", errors="replace")
0025         return "\n".join(text.splitlines()[:200])[:25000]
0026
0027     def remember(self, note: str) -> str:
0028         note = note.strip()
0029         if not note:
0030             return "ERROR: empty memory note"
0031         self.memory_dir.mkdir(parents=True, exist_ok=True)
0032         existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
0033         normalized = f"- {note}"
0034         if normalized in existing:
0035             return "OK: note already present"
0036         with self.memory_file.open("a", encoding="utf-8") as fh:
0037             if not existing.endswith("\n"):

```

```

0038         fh.write("\n")
0039         fh.write(normalized + "\n")
0040         return "OK: persisted memory"
0041
0042     def search(self, query: str, limit: int = 20) -> str:
0043         if not self.memory_file.exists():
0044             return "(memory is empty)"
0045         q = query.casefold().strip()
0046         lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
0047         hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
0048         selected = list(hits)[:limit]
0049         return "\n".join(selected) if selected else "(no memory matches)"
0050
0051     def startup_context(self) -> str:
0052         parts = []
0053         project = self.load_project_instructions().strip()
0054         memory = self.load_auto_memory().strip()
0055         if project:
0056             parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
0057         if memory:
0058             parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
0059         return "\n\n".join(parts)

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

sandbox.py (102 Lines)

```

0001 """Execution boundary for shell tools.
0002
0003 `soft` mode is portability-oriented and is NOT a strong OS security boundary.
0004 `bwrap` mode uses Bubblewrap on Linux when available and can isolate network + filesystem.
0005 """
0006 from __future__ import annotations
0007
0008 import os
0009 import shutil
0010 import subprocess
0011 import tempfile
0012 from dataclasses import dataclass
0013 from pathlib import Path
0014
0015
0016 @dataclass(frozen=True)
0017 class SandboxResult:
0018     exit_code: int
0019     stdout: str
0020     stderr: str
0021     mode: str
0022
0023
0024 class SandboxRunner:
0025     def __init__(self, workspace: Path, *, mode: str = "auto", disable_network: bool = True):
0026         self.workspace = workspace.resolve()
0027         self.requested_mode = mode
0028         self.disable_network = disable_network
0029         self.bwrap = shutil.which("bwrap")
0030
0031     @property
0032     def mode(self) -> str:
0033         if self.requested_mode == "auto":
0034             return "bwrap" if self.bwrap else "soft"
0035         if self.requested_mode == "bwrap" and not self.bwrap:
0036             raise RuntimeError("MINICLAUDE_SANDBOX=bwrap requested but `bwrap` is not installed")
0037         if self.requested_mode not in {"soft", "bwrap"}:
0038             raise ValueError("sandbox mode must be auto, soft, or bwrap")
0039         return self.requested_mode
0040
0041     def sanitized_env(self) -> dict[str, str]:
0042         allowed = {"PATH", "LANG", "LC_ALL", "TERM", "SHELL", "USER"}
0043         env = {k: v for k, v in os.environ.items() if k in allowed}
0044         env["HOME"] = str(self.workspace / ".mini_claude" / "sandbox_home")
0045         env["MINICLAUDE_SANDBOX"] = self.mode
0046         Path(env["HOME"]).mkdir(parents=True, exist_ok=True)
0047         return env
0048
0049     def run(self, command: str, *, timeout: int = 30) -> SandboxResult:
0050         mode = self.mode
0051         env = self.sanitized_env()
0052         if mode == "soft":
0053             proc = subprocess.run(
0054                 ["bash", "-lc", command],

```



```

0055         cwd=self.workspace,
0056         env=env,
0057         text=True,
0058         capture_output=True,
0059         timeout=timeout,
0060     )
0061     return SandboxResult(proc.returncode, proc.stdout, proc.stderr, mode)
0062
0063     # Bubblewrap: workspace is writable; common system paths are read-only.
0064     # This is intentionally conservative and may need distro-specific adjustments.
0065     tmp = tempfile.mkdtemp(prefix="mini-claude-")
0066     argv = [
0067         self.bwrap or "bwrap",
0068         "--die-with-parent",
0069         "--new-session",
0070         "--proc", "/proc",
0071         "--dev", "/dev",
0072         "--tmpfs", "/tmp",
0073         "--bind", str(self.workspace), str(self.workspace),
0074         "--chdir", str(self.workspace),
0075     ]
0076     for system_path in ("/usr", "/bin", "/lib", "/lib64", "/etc"):
0077         if Path(system_path).exists():
0078             argv += ["--ro-bind", system_path, system_path]
0079     if self.disable_network:
0080         argv.append("--unshare-net")
0081     argv += ["--setenv", "HOME", str(self.workspace / ".mini_claude" / "sandbox_home")]
0082     argv += ["bash", "-lc", command]
0083
0084     try:
0085         proc = subprocess.run(
0086             argv,
0087             cwd=self.workspace,
0088             env=env,
0089             text=True,
0090             capture_output=True,
0091             timeout=timeout,
0092         )
0093         return SandboxResult(proc.returncode, proc.stdout, proc.stderr, mode)
0094     finally:
0095         shutil.rmtree(tmp, ignore_errors=True)
0096
0097     def status(self) -> str:
0098         return (
0099             f"requested={self.requested_mode}, active={self.mode}, "
0100             f"network={'disabled' if self.disable_network and self.mode == 'bwrap' else 'not strongly isolated'}, "
0101             f"workspace={self.workspace}"
0102         )

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

skills.py (70 lines)

```

0001 """Progressive-disclosure skills for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class Skill:
0013     name: str
0014     description: str
0015     path: Path
0016     body: str
0017     metadata: dict[str, Any]
0018
0019
0020 class SkillRegistry:
0021     def __init__(self, workspace: Path):
0022         self.workspace = workspace.resolve()
0023         self.roots = [
0024             self.workspace / ".mini_claude" / "skills",
0025             self.workspace / ".claude" / "skills", # optional compatibility
0026         ]
0027
0028     @staticmethod

```

```

0029     def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0030         if not text.startswith("---\n"):
0031             return {}, text
0032         end = text.find("\n---\n", 4)
0033         if end == -1:
0034             return {}, text
0035         raw = text[4:end]
0036         body = text[end + 5 :]
0037         data = yaml.safe_load(raw) or {}
0038         if not isinstance(data, dict):
0039             data = {}
0040         return data, body
0041
0042     def discover(self) -> dict[str, Skill]:
0043         found: dict[str, Skill] = {}
0044         for root in self.roots:
0045             if not root.exists():
0046                 continue
0047             for skill_file in sorted(root.glob("*/SKILL.md")):
0048                 text = skill_file.read_text(encoding="utf-8", errors="replace")
0049                 meta, body = self._split_frontmatter(text)
0050                 name = str(meta.get("name") or skill_file.parent.name)
0051                 description = str(meta.get("description") or "No description provided")
0052                 found[name] = Skill(name, description, skill_file, body.strip(), meta)
0053         return found
0054
0055     def catalog(self) -> str:
0056         skills = self.discover()
0057         if not skills:
0058             return "(no skills installed)"
0059         return "\n".join(f"- {name}: {skill.description}" for name, skill in skills.items())
0060
0061     def load(self, name: str) -> str:
0062         skill = self.discover().get(name)
0063         if skill is None:
0064             return f"ERROR: unknown skill {name!r}. Available:\n{self.catalog()}"
0065         # A skill can reference support files by relative path; normal file tools can read them later.
0066         return (
0067             f"# Skill: {skill.name}\n"
0068             f"Source: {skill.path.relative_to(self.workspace)}\n\n"
0069             f"{skill.body[:30000]}"
0070         )

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

11. s10 - Multi-Agent Runtime

Worktree + 并行 Worker + Mailbox 的最小多 Agent Runtime

观察项	本阶段
阶段目录	s10_multi_agent
文件数量	15
新增文件	.mini_claude/agents/implementer.md, team.py
继承/修改文件	.mini_claude/agents/explorer.md, .mini_claude/agents/reviewer.md, .mini_claude/audit_hook.py, .mini_claude/hooks.json, .mini_claude/skills/verify-change/SKILL.md, README.md, agents.py, code.py ...
Python 行数	1293

KNOW-WHY：为什么这一层必须存在

- 多个 Agent 同时修改同一个 checkout 会产生文件竞争和 Git 状态污染。真正的并行软件工程需要 Context Isolation + Filesystem Isolation + Branch Isolation。
- 多 Agent 不应该等同于“多开几个模型”。需要任务切分、身份、通信、结果收集、资源上限和合并策略。
- 自动 merge 会把并发冲突从执行期推到隐藏的集成期，因此教学 Runtime 选择永不自动合并，只留下 branch/worktree 供 Lead 或人类审查。

KNOW-HOW：这一层具体怎么长出来

- 37. run_team 接收最多 8 个独立 WorkerSpec，并为每个 worker 创建 `mini/<team>/<worker>` Git branch + worktree。
- 38. 每个 worker 是独立 Python 子进程，在自己的 cwd 中运行 Mini Claude Code；因此工具路径、安全策略和 Context 都自然隔离。
- 39. ThreadPoolExecutor 并发运行 worker；工作完成后收集 stdout/stderr、git status、diff stat、branch/worktree。
- 40. TeamBus 使用共享 JSONL mailbox 提供 team_send/team_inbox，足以演示最小跨 Agent 通信。
- 41. 非交互 worker 对隔离 worktree 内的 write/edit 自动允许，但未知 shell 仍不会弹窗等待。

数据流 / 控制流

Lead -> run_team -> N worktrees -> N worker processes -> mailbox/results -> Lead reviews branches

最容易踩的坑

- 并行度越高 Token/API 成本越高，不应把线性任务强行拆成团队。
- 工作任务如果修改同一逻辑区域，即使 worktree 不冲突，最终合并仍会冲突。
- 文件 mailbox 只是教学实现，不处理可靠投递、锁、幂等、背压和实时事件；生产系统应使用更成熟的协调层。
- Worker 进程复用 API key，因此 Host 仍需承担凭据管理；Shell sandbox 会主动剥离这些敏感变量。

本阶段验收清单

- 在 Git repo 中启动两个只读 worker，确认并行完成。
- 让两个 implementer 修改互不重叠文件，确认得到两个独立 branch/worktree。
- 让 worker team_send 一条消息，并由另一 worker team_inbox 读取。
- 确认 Runtime 不会自动 merge。

Subagent、Worktree、Team 三个维度不要混为一谈

机制	隔离对象	是否并行	主要用途
Subagent	Context	可并行/可串行	委托研究/评审
Worktree	Filesystem + branch	支持	避免并发编辑冲突
Team Runtime	Context + workers + coordination	是	拆分可独立任务

Claude Code 当前公开文档把 subagents、worktrees 与 agent teams 明确作为不同并行方式：worktree 隔离文件，subagent 在单会话内委托独立 context，agent teams 增加共享任务与消息协调；Agent Teams 当前文档仍标记 experimental。[R7][R8][R9]

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (580 lines)

```
0001 #!/usr/bin/env python3
0002 """s10 - Add a worktree-isolated parallel Multi-Agent Runtime."""
0003 from __future__ import annotations
0004
0005 import glob as globlib
0006 import argparse
0007 import json
0008 import os
0009 import subprocess
0010 import shlex
0011 from enum import Enum
0012 from pathlib import Path
0013 from typing import Any, Callable
0014
0015 from anthropic import Anthropic
0016 from dotenv import load_dotenv
0017 from context_manager import ContextManager
0018 from memory import MemoryStore
0019 from skills import SkillRegistry
0020 from hooks import HookManager
0021 from agents import AgentRegistry
0022 from sandbox import SandboxRunner
0023 from team import TeamBus, TeamRuntime
0024
0025 load_dotenv()
0026 MODEL = os.environ.get("ANTHROPIC_MODEL")
0027 if not MODEL:
0028     raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")
0029
0030 client = Anthropic()
0031 WORKSPACE = Path.cwd().resolve()
0032 MAX_TOOL_OUTPUT = 12000
0033 CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
0034 MEMORY = MemoryStore(WORKSPACE)
0035 SKILLS = SkillRegistry(WORKSPACE)
0036 HOOKS = HookManager(WORKSPACE)
0037 AGENTS = AgentRegistry(WORKSPACE)
0038 SANDBOX = SandboxRunner(
0039     WORKSPACE,
0040     mode=os.getenv("MINICLAUDE_SANDBOX", "auto"),
0041     disable_network=os.getenv("MINICLAUDE_ALLOW_NETWORK", "0") != "1",
0042 )
0043 TEAM_BUS = TeamBus()
0044 TEAM = TeamRuntime(WORKSPACE, Path(__file__))
0045
0046
0047 def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
0048     return {
0049         "name": name,
0050         "description": description,
0051         "input_schema": {
```

```

0052         "type": "object",
0053         "properties": properties,
0054         "required": required,
0055         "additionalProperties": False,
0056     },
0057 }
0058
0059
0060 TOOLS = [
0061     tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}, ["command"]},
0062     tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}, ["path"]},
0063     tool(
0064         "write_file",
0065         "Create or replace a UTF-8 text file inside the workspace.",
0066         {"path": {"type": "string"}, "content": {"type": "string"}},
0067         ["path", "content"],
0068     ),
0069     tool(
0070         "edit_file",
0071         "Replace one exact text occurrence in a workspace file.",
0072         {
0073             "path": {"type": "string"},
0074             "old_text": {"type": "string"},
0075             "new_text": {"type": "string"},
0076         },
0077         ["path", "old_text", "new_text"],
0078     ),
0079     tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}, ["pattern"]},
0080     tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}, ["note"]},
0081     tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}, ["query"]},
0082     tool("use_skill", "Load a reusable task procedure by name when it is relevant.", {"name": {"type": "string"}, ["name"]},
0083     tool(
0084         "spawn_subagent",
0085         "Delegate a focused side task to a named subagent with its own context window.",
0086         {"agent": {"type": "string"}, "task": {"type": "string"}},
0087         ["agent", "task"],
0088     ),
0089     tool("sandbox_status", "Report the active shell sandbox mode.", {}, []),
0090     tool("team_send", "Send a concise message to another worker when running inside a team.", {"to": {"type": "string"}, "message": {"type":
"string"}}, ["to", "message"]),
0091     tool("team_inbox", "Read recent teammate messages when running inside a team.", {}, []),
0092     tool(
0093         "run_team",
0094         "Run independent worker agents in parallel Git worktrees. Use for genuinely separable tasks; results are never auto-merged.",
0095         {
0096             "team_name": {"type": "string"},
0097             "tasks": {
0098                 "type": "array",
0099                 "minItems": 1,
0100                 "maxItems": 8,
0101                 "items": {
0102                     "type": "object",
0103                     "properties": {
0104                         "name": {"type": "string"},
0105                         "agent": {"type": "string"},
0106                         "prompt": {"type": "string"}
0107                     },
0108                     "required": ["name", "prompt"],
0109                     "additionalProperties": False
0110                 }
0111             }
0112         },
0113         ["team_name", "tasks"],
0114     ),
0115 ]
0116
0117 BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
0118 Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
0119 Make focused changes and verify them before you finish.
0120 Use remember only for durable facts likely to help future sessions, not transient task state.
0121 """
0122
0123
0124 def build_system() -> str:
0125     startup = MEMORY.startup_context()
0126     catalog = SKILLS.catalog()
0127     parts = [BASE_SYSTEM]
0128     if startup:
0129         parts.append(startup)
0130     parts.append("<available_skills>\n" + catalog + "\n</available_skills>")

```

```

0131 parts.append("Load a skill with use_skill only when its procedure is needed.")
0132 parts.append("<available_subagents>\n" + AGENTS.catalog() + "\n</available_subagents>")
0133 parts.append("Delegate noisy research or independent review with spawn_subagent when useful.")
0134 team_context = TEAM_BUS.context()
0135 if team_context:
0136     parts.append("<team_context>\n" + team_context + "\n</team_context>")
0137 return "\n\n".join(parts)
0138
0139
0140 def safe_path(raw: str) -> Path:
0141     """Resolve a path and reject escapes outside the workspace."""
0142     candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
0143     try:
0144         candidate.relative_to(WORKSPACE)
0145     except ValueError as exc:
0146         raise ValueError(f"Path escapes workspace: {raw!r}") from exc
0147     return candidate
0148
0149
0150 def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
0151     if len(text) <= limit:
0152         return text
0153     half = limit // 2
0154     return text[:half] + "\n...<truncated>...\n" + text[-half:]
0155
0156
0157 def run_bash(command: str) -> str:
0158     try:
0159         result = SANDBOX.run(command, timeout=30)
0160     except subprocess.TimeoutExpired:
0161         return "ERROR: command timed out after 30 seconds"
0162     except Exception as exc:
0163         return f"ERROR: sandbox execution failed: {type(exc).__name__}: {exc}"
0164     return json.dumps(
0165         {
0166             "sandbox": result.mode,
0167             "exit_code": result.exit_code,
0168             "stdout": truncate(result.stdout),
0169             "stderr": truncate(result.stderr),
0170         },
0171         ensure_ascii=False,
0172     )
0173
0174
0175 def run_read(path: str) -> str:
0176     p = safe_path(path)
0177     if not p.is_file():
0178         return f"ERROR: file not found: {path}"
0179     return truncate(p.read_text(encoding="utf-8", errors="replace"))
0180
0181
0182 def run_write(path: str, content: str) -> str:
0183     p = safe_path(path)
0184     p.parent.mkdir(parents=True, exist_ok=True)
0185     p.write_text(content, encoding="utf-8")
0186     return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"
0187
0188
0189 def run_edit(path: str, old_text: str, new_text: str) -> str:
0190     p = safe_path(path)
0191     if not p.is_file():
0192         return f"ERROR: file not found: {path}"
0193     text = p.read_text(encoding="utf-8", errors="replace")
0194     count = text.count(old_text)
0195     if count == 0:
0196         return "ERROR: old_text not found"
0197     if count > 1:
0198         return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
0199     p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
0200     return f"OK: edited {p.relative_to(WORKSPACE)}"
0201
0202
0203 def run_glob(pattern: str) -> str:
0204     # Resolve matches then apply safe_path to each result to keep output inside the workspace.
0205     raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
0206     matches: list[str] = []
0207     for raw in raw_matches[:500]:
0208         try:
0209             p = safe_path(raw)
0210         except ValueError:

```

```

0211         continue
0212         matches.append(str(p.relative_to(WORKSPACE)))
0213         return "\n".join(sorted(matches)) if matches else "(no matches)"
0214
0215
0216
0217
0218 class Decision(str, Enum):
0219     ALLOW = "allow"
0220     ASK = "ask"
0221     DENY = "deny"
0222
0223
0224 # Gate 1: patterns that should never be auto-approved by this tutorial runtime.
0225 HARD_DENY_SUBSTRINGS = (
0226     "sudo ",
0227     "chmod -R 777 /",
0228     "> /dev/sd",
0229     "mkfs.",
0230 )
0231
0232 # Gate 2: commands considered low-risk and repeatable in a coding workspace.
0233 SAFE_COMMAND_PREFIXES = (
0234     "pwd", "ls", "find ", "git status", "git diff", "git log",
0235     "python -m pytest", "pytest", "npm test", "npm run test",
0236     "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
0237 )
0238
0239
0240 def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
0241     """Return a policy decision and human-readable reason.
0242
0243     Order matters: hard deny -> explicit rules -> ask fallback.
0244     """
0245     if name in {"read_file", "glob", "recall_memory", "use_skill", "spawn_subagent", "sandbox_status", "team_send", "team_inbox"}:
0246         return Decision.ALLOW, "read-only or coordination operation"
0247
0248     if name == "run_team":
0249         return Decision.ASK, "spawns multiple model workers and creates Git worktrees"
0250
0251     if name == "remember":
0252         return Decision.ALLOW, "writes only to the agent memory store"
0253
0254     if name in {"write_file", "edit_file"}:
0255         if os.getenv("MINICLAUDE_WORKER_ALLOW_EDITS") == "1":
0256             return Decision.ALLOW, "isolated team worker edit policy"
0257         try:
0258             safe_path(args.get("path", ""))
0259         except Exception as exc:
0260             return Decision.DENY, str(exc)
0261         return Decision.ASK, "file modification inside workspace"
0262
0263     if name == "bash":
0264         command = str(args.get("command", "")).strip()
0265         lowered = command.lower()
0266         if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
0267             return Decision.DENY, "matches hard deny policy"
0268         if command.startswith(SAFE_COMMAND_PREFIXES):
0269             return Decision.ALLOW, "matches low-risk command allowlist"
0270         return Decision.ASK, "shell command is not allowlisted"
0271
0272     return Decision.ASK, "unknown or extensible tool"
0273
0274
0275 def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
0276     if os.getenv("MINICLAUDE_NONINTERACTIVE") == "1":
0277         print(f"[permission] noninteractive denial for {name}: {reason}")
0278         return False
0279     print(f"\n[permission] {name}: {reason}")
0280     print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
0281     while True:
0282         answer = input("Allow once? [y/N] ").strip().lower()
0283         if answer in {"y", "yes"}:
0284             return True
0285         if answer in {"", "n", "no"}:
0286             return False
0287
0288
0289 def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
0290     decision, reason = classify_permission(name, args)

```

```

0291     if decision is Decision.DENY:
0292         return False, f"DENIED: {reason}"
0293     if decision is Decision.ALLOW:
0294         return True, reason
0295     if request_approval(name, args, reason):
0296         return True, "approved by user"
0297     return False, "DENIED: user rejected tool call"
0298
0299
0300 TOOL_HANDLERS: dict[str, Callable[..., str]] = {
0301     "bash": run_bash,
0302     "read_file": run_read,
0303     "write_file": run_write,
0304     "edit_file": run_edit,
0305     "glob": run_glob,
0306     "remember": MEMORY.remember,
0307     "recall_memory": MEMORY.search,
0308     "use_skill": SKILLS.load,
0309     "sandbox_status": lambda: SANDBOX.status(),
0310     "team_send": TEAM_BUS.send,
0311     "team_inbox": TEAM_BUS.inbox,
0312     "run_team": TEAM.run,
0313 }
0314
0315
0316 def block_to_dict(block: Any) -> dict[str, Any]:
0317     if hasattr(block, "model_dump"):
0318         return block.model_dump(exclude_none=True)
0319     return dict(block)
0320
0321
0322 def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
0323     handler = TOOL_HANDLERS.get(name)
0324     if handler is None:
0325         return f"Unknown tool: {name}", True
0326     try:
0327         return handler(**args), False
0328     except Exception as exc:
0329         return f"{type(exc).__name__}: {exc}", True
0330
0331
0332
0333
0334
0335 def run_subagent(agent: str, task: str) -> str:
0336     definition = AGENTS.get(agent)
0337     if definition is None:
0338         return f"ERROR: unknown subagent {agent!r}. Available:\n{AGENTS.catalog()}"
0339
0340     allowed_names = [name for name in definition.tools if name != "spawn_subagent"]
0341     tool_defs = [item for item in TOOLS if item["name"] in allowed_names]
0342     sub_messages: list[dict[str, Any]] = [{"role": "user", "content": task}]
0343     final_text: list[str] = []
0344
0345     HOOKS.fire("subagent_start", {"agent": definition.name, "task": task})
0346     for turn in range(definition.max_turns):
0347         response = client.messages.create(
0348             model=definition.model or MODEL,
0349             max_tokens=3000,
0350             system=(
0351                 build_system()
0352                 + "\n\n<subagent_role>\n"
0353                 + definition.prompt
0354                 + "\n\n</subagent_role>\n"
0355                 + "You are a delegated worker. Do not spawn nested subagents. Return a compact result to the parent."
0356             ),
0357             tools=tool_defs,
0358             messages=sub_messages,
0359         )
0360         sub_messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0361         final_text = [b.text for b in response.content if getattr(b, "type", None) == "text"]
0362         if response.stop_reason != "tool_use":
0363             break
0364
0365     results: list[dict[str, Any]] = []
0366     for block in response.content:
0367         if getattr(block, "type", None) != "tool_use":
0368             continue
0369         args = dict(block.input)
0370         if block.name not in allowed_names:

```



```

0371         content, is_error = f"DENIED: tool {block.name!r} is not allowed for subagent {definition.name}", True
0372     else:
0373         pre = HOOKS.fire(
0374             "pre_tool_use",
0375             {"tool_name": block.name, "tool_input": args, "subagent": definition.name},
0376         )
0377         if pre.updated_input is not None:
0378             args = pre.updated_input
0379         if pre.decision == "deny":
0380             content, is_error = f"DENIED BY HOOK: {pre.reason}", True
0381         else:
0382             allowed, reason = check_permission(block.name, args)
0383             if allowed:
0384                 content, is_error = execute_tool(block.name, args)
0385             else:
0386                 content, is_error = reason, True
0387         HOOKS.fire(
0388             "post_tool_use",
0389             {
0390                 "tool_name": block.name,
0391                 "tool_input": args,
0392                 "tool_output": content,
0393                 "is_error": is_error,
0394                 "subagent": definition.name,
0395             },
0396         )
0397         results.append(
0398             {
0399                 "type": "tool_result",
0400                 "tool_use_id": block.id,
0401                 "content": content,
0402                 "is_error": is_error,
0403             }
0404         )
0405         sub_messages.append({"role": "user", "content": results})
0406     else:
0407         final_text = [f"Subagent reached max_turns={definition.max_turns} before a clean stop."]
0408
0409     result = "\n".join(final_text).strip() or "(subagent returned no text)"
0410     HOOKS.fire("subagent_stop", {"agent": definition.name, "result": result})
0411     return result
0412
0413
0414 # Register after the function exists; this keeps the generic dispatch table from s02.
0415 TOOL_HANDLERS["spawn_subagent"] = run_subagent
0416
0417 def summarize_history(old_messages: List[Dict[str, Any]]) -> str:
0418     """Ask the model to compress old state without tools."""
0419     prompt = """Summarize this coding-agent conversation for continuation.
0420 Preserve only durable state:
0421 - user goal and constraints
0422 - repository facts discovered
0423 - files changed and why
0424 - commands/tests run and outcomes
0425 - unresolved hypotheses, TODOs, and errors
0426 - important approvals or denied operations
0427 Do not invent facts. Be concise but concrete.
0428
0429 HISTORY JSON:
0430 """ + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
0431     response = client.messages.create(
0432         model=MODEL,
0433         max_tokens=1800,
0434         system="You compress coding-agent state for a future model turn.",
0435         messages=[{"role": "user", "content": prompt}],
0436     )
0437     return "\n".join(
0438         block.text for block in response.content if getattr(block, "type", None) == "text"
0439     )
0440
0441
0442 def agent_loop(messages: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
0443     while True:
0444         if CONTEXT.needs_compaction(messages):
0445             print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
0446             HOOKS.fire("pre_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
0447             messages = CONTEXT.compact(messages, summarize_history)
0448             HOOKS.fire("post_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
0449         response = client.messages.create(
0450             model=MODEL,

```

```

0451         max_tokens=4096,
0452         system=build_system(),
0453         tools=TOOLS,
0454         messages=messages,
0455     )
0456
0457     for block in response.content:
0458         if getattr(block, "type", None) == "text":
0459             print(block.text)
0460
0461     messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0462     if response.stop_reason != "tool_use":
0463         return messages
0464
0465     results = []
0466     for block in response.content:
0467         if getattr(block, "type", None) != "tool_use":
0468             continue
0469         args = dict(block.input)
0470         print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
0471
0472         pre = HOOKS.fire("pre_tool_use", {"tool_name": block.name, "tool_input": args})
0473         if pre.updated_input is not None:
0474             args = pre.updated_input
0475         if pre.decision == "deny":
0476             content, is_error = f"DENIED BY HOOK: {pre.reason}", True
0477         else:
0478             if pre.decision == "ask":
0479                 allowed = request_approval(block.name, args, pre.reason or "hook requested approval")
0480                 permission_reason = "approved by user" if allowed else "DENIED: user rejected hook approval"
0481             else:
0482                 allowed, permission_reason = check_permission(block.name, args)
0483             if allowed:
0484                 content, is_error = execute_tool(block.name, args)
0485             else:
0486                 content, is_error = permission_reason, True
0487
0488         post = HOOKS.fire(
0489             "post_tool_use",
0490             {"tool_name": block.name, "tool_input": args, "tool_output": content, "is_error": is_error},
0491         )
0492         if post.output:
0493             content += "\n[hook output]\n" + post.output.strip()
0494         results.append(
0495             {
0496                 "type": "tool_result",
0497                 "tool_use_id": block.id,
0498                 "content": content,
0499                 "is_error": is_error,
0500             }
0501         )
0502     messages.append({"role": "user", "content": results})
0503
0504
0505 def run_one_prompt(prompt: str, agent: str | None = None) -> int:
0506     HOOKS.fire("session_start", {"model": MODEL, "noninteractive": True})
0507     try:
0508         if agent:
0509             print(run_subagent(agent, prompt))
0510         else:
0511             messages: list[dict[str, Any]] = [{"role": "user", "content": prompt}]
0512             agent_loop(messages)
0513             return 0
0514     except Exception as exc:
0515         print(f"ERROR: {type(exc).__name__}: {exc}")
0516         return 1
0517     finally:
0518         HOOKS.fire("session_end", {"noninteractive": True})
0519
0520
0521 def main() -> None:
0522     parser = argparse.ArgumentParser(add_help=True)
0523     parser.add_argument("--prompt", help="Run one non-interactive task and exit")
0524     parser.add_argument("--agent", help="Run --prompt through a named subagent")
0525     parser.add_argument("--worker", action="store_true", help="Internal marker for team worker processes")
0526     args = parser.parse_args()
0527
0528     if args.prompt:
0529         raise SystemExit(run_one_prompt(args.prompt, args.agent))
0530

```

```

0531 print(f"Mini Claude Code s10 - workspace: {WORKSPACE}")
0532 print("Commands: /context /compact /clear /memory /skills /agents /sandbox /exit")
0533 HOOKS.fire("session_start", {"model": MODEL})
0534 messages: List[dict[str, Any]] = []
0535 while True:
0536     try:
0537         prompt = input("\nyou> ").strip()
0538     except (EOFError, KeyboardInterrupt):
0539         print()
0540         break
0541     if not prompt:
0542         continue
0543     if prompt in {"/exit", "/quit"}:
0544         break
0545     if prompt == "/context":
0546         print(CONTEXT.report(messages))
0547         continue
0548     if prompt == "/compact":
0549         HOOKS.fire("pre_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
0550         messages = CONTEXT.compact(messages, summarize_history)
0551         HOOKS.fire("post_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
0552         print("[context] " + CONTEXT.report(messages))
0553         continue
0554     if prompt == "/clear":
0555         messages = []
0556         print("[context] cleared")
0557         continue
0558     if prompt == "/memory":
0559         print(MEMORY.load_auto_memory() or "(memory is empty)")
0560         continue
0561     if prompt == "/skills":
0562         print(SKILLS.catalog())
0563         continue
0564     if prompt == "/agents":
0565         print(AGENTS.catalog())
0566         continue
0567     if prompt == "/sandbox":
0568         print(SANDBOX.status())
0569         continue
0570     messages.append({"role": "user", "content": prompt})
0571     try:
0572         messages = agent_loop(messages)
0573     except Exception as exc:
0574         print(f"ERROR: {exc}")
0575
0576     HOOKS.fire("session_end", {"message_count": len(messages)})
0577
0578
0579 if __name__ == "__main__":
0580     main()

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/explorer.md (13 lines)

```

0001 ---
0002 name: explorer
0003 description: Read-only codebase exploration and evidence gathering.
0004 tools:
0005     - read_file
0006     - glob
0007     - bash
0008 max_turns: 10
0009 ---
0010
0011 You are a read-only exploration specialist.
0012 Do not modify files. Trace code paths, gather evidence, and return a concise report with exact file paths.
0013 Prefer search and focused reads over dumping large files.

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/implementer.md (18 lines)

```

0001 ---
0002 name: implementer
0003 description: Implement a narrowly scoped change in an isolated worktree and verify it.
0004 tools:
0005     - read_file
0006     - glob
0007     - write_file
0008     - edit_file
0009     - bash

```

```

0010 - use_skill
0011 - team_send
0012 - team_inbox
0013 max_turns: 18
0014 ---
0015
0016 You own the assigned task in an isolated Git worktree.
0017 Inspect before editing, keep the diff focused, run relevant tests, and report what changed.
0018 Do not merge branches. If another worker needs a discovery, send a concise message with team_send.

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/reviewer.md (12 Lines)

```

0001 ---
0002 name: reviewer
0003 description: Review a proposed change for correctness, regressions, and missing tests.
0004 tools:
0005   - read_file
0006   - glob
0007   - bash
0008 max_turns: 10
0009 ---
0010
0011 Review independently. Inspect the diff and relevant tests. Return findings ordered by severity, with evidence.
0012 Do not edit files.

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/audit_hook.py (12 Lines)

```

0001 #!/usr/bin/env python3
0002 """Example hook: append compact JSON events to .mini_claude/audit.jsonl."""
0003 import json
0004 import sys
0005 from pathlib import Path
0006
0007 event = json.load(sys.stdin)
0008 path = Path('.mini_claude/audit.jsonl')
0009 path.parent.mkdir(parents=True, exist_ok=True)
0010 with path.open('a', encoding='utf-8') as fh:
0011     fh.write(json.dumps(event, ensure_ascii=False, default=str) + '\n')
0012 print(json.dumps({"decision": "allow"}))

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/hooks.json (8 Lines)

```

0001 {
0002   "session_start": ["python .mini_claude/audit_hook.py"],
0003   "pre_tool_use": ["python .mini_claude/audit_hook.py"],
0004   "post_tool_use": ["python .mini_claude/audit_hook.py"],
0005   "pre_compact": ["python .mini_claude/audit_hook.py"],
0006   "post_compact": ["python .mini_claude/audit_hook.py"],
0007   "session_end": ["python .mini_claude/audit_hook.py"]
0008 }

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/skills/verify-change/SKILL.md (12 Lines)

```

0001 ---
0002 name: verify-change
0003 description: Verify a code change before declaring the task complete.
0004 ---
0005
0006 When implementation appears complete:
0007 1. inspect `git diff --stat` and `git diff`
0008 2. run the narrowest relevant test first
0009 3. run the broader regression suite if practical
0010 4. run lint/typecheck if the repository has them
0011 5. report exact commands and outcomes
0012 6. flag anything that was not verified

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (15 Lines)

```

0001 # s10 Multi-Agent Runtime
0002
0003 Adds:
0004 - `run_team` tool
0005 - parallel worker processes
0006 - per-worker Git worktrees and branches
0007 - file-backed team mailbox (`team_send` / `team_inbox`)
0008 - non-interactive worker mode

```

```

0009 - no automatic merge: changed worktrees are left for review
0010
0011 Example user request inside the REPL:
0012
0013 > Split this migration into three independent tasks. Use run_team with implementer workers and keep each worker on its own files.
0014
0015 The lead receives a JSON report with branch/worktree, diff stat, Git status, and worker output.

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

agents.py (69 lines)

```

0001 """Custom subagent definitions for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class AgentDefinition:
0013     name: str
0014     description: str
0015     tools: list[str]
0016     model: str | None
0017     max_turns: int
0018     prompt: str
0019     metadata: dict[str, Any]
0020
0021
0022 class AgentRegistry:
0023     def __init__(self, workspace: Path):
0024         self.workspace = workspace.resolve()
0025         self.root = self.workspace / ".mini_claude" / "agents"
0026
0027     @staticmethod
0028     def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0029         if not text.startswith("---\n"):
0030             return {}, text
0031         end = text.find("\n---\n", 4)
0032         if end == -1:
0033             return {}, text
0034         meta = yaml.safe_load(text[4:end]) or {}
0035         return (meta if isinstance(meta, dict) else {}), text[end + 5:].strip()
0036
0037     def discover(self) -> dict[str, AgentDefinition]:
0038         found: dict[str, AgentDefinition] = {}
0039         if not self.root.exists():
0040             return found
0041         for path in sorted(self.root.glob("*.md")):
0042             meta, body = self._split_frontmatter(path.read_text(encoding="utf-8", errors="replace"))
0043             name = str(meta.get("name") or path.stem)
0044             raw_tools = meta.get("tools") or ["read_file", "glob", "bash"]
0045             if isinstance(raw_tools, str):
0046                 raw_tools = [x.strip() for x in raw_tools.split(",") if x.strip()]
0047             tools = [str(x) for x in raw_tools]
0048             found[name] = AgentDefinition(
0049                 name=name,
0050                 description=str(meta.get("description") or "No description provided"),
0051                 tools=tools,
0052                 model=str(meta["model"]) if meta.get("model") else None,
0053                 max_turns=int(meta.get("max_turns", 12)),
0054                 prompt=body,
0055                 metadata=meta,
0056             )
0057         return found
0058
0059     def catalog(self) -> str:
0060         agents = self.discover()
0061         if not agents:
0062             return "(no custom subagents installed)"
0063         return "\n".join(
0064             f"- {name}: {agent.description} [tools={' '.join(agent.tools)}]"
0065             for name, agent in agents.items()
0066         )
0067
0068     def get(self, name: str) -> AgentDefinition | None:
0069         return self.discover().get(name)

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```
0001 """A small context manager: estimate, inspect, and compact message history."""
0002 from __future__ import annotations
0003
0004 import json
0005 from dataclasses import dataclass
0006 from typing import Any, Callable
0007
0008
0009 @dataclass
0010 class ContextStats:
0011     message_count: int
0012     approx_tokens: int
0013     chars: int
0014
0015
0016 class ContextManager:
0017     def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
0018         self.max_approx_tokens = max_approx_tokens
0019         self.keep_recent_messages = keep_recent_messages
0020
0021     @staticmethod
0022     def estimate(messages: list[dict[str, Any]]) -> ContextStats:
0023         raw = json.dumps(messages, ensure_ascii=False, default=str)
0024         # Deliberately simple heuristic. Production systems should use model-aware counting.
0025         return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))
0026
0027     def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
0028         return self.estimate(messages).approx_tokens >= self.max_approx_tokens
0029
0030     def report(self, messages: list[dict[str, Any]]) -> str:
0031         stats = self.estimate(messages)
0032         pct = 100 * stats.approx_tokens / self.max_approx_tokens
0033         return (
0034             f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:}, "
0035             f"budget={self.max_approx_tokens:}, ({pct:.1f}%)"
0036         )
0037
0038     def compact(
0039         self,
0040         messages: list[dict[str, Any]],
0041         summarizer: Callable[[list[dict[str, Any]], str], str],
0042     ) -> list[dict[str, Any]]:
0043         if len(messages) <= self.keep_recent_messages + 2:
0044             return messages
0045
0046         cut = len(messages) - self.keep_recent_messages
0047         old = messages[:cut]
0048         recent = messages[cut:]
0049         summary = summarizer(old)
0050         compacted = {
0051             "role": "user",
0052             "content": (
0053                 "[COMPACTED_CONTEXT]\n"
0054                 "The following is a structured summary of earlier conversation state. "
0055                 "Treat it as historical context, not a new user request.\n\n"
0056                 + summary
0057             ),
0058         }
0059         return [compacted, *recent]
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

hooks.py (82 lines)

```
0001 """Deterministic lifecycle hooks for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 import json
0005 import subprocess
0006 from dataclasses import dataclass
0007 from pathlib import Path
0008 from typing import Any
0009
0010
0011 @dataclass
0012 class HookResult:
0013     decision: str = "allow" # allow | deny | ask
```

```

0014     reason: str = ""
0015     updated_input: dict[str, Any] | None = None
0016     output: str = ""
0017
0018
0019 class HookManager:
0020     def __init__(self, workspace: Path):
0021         self.workspace = workspace.resolve()
0022         self.config_path = self.workspace / ".mini_claude" / "hooks.json"
0023
0024     def _config(self) -> dict[str, Any]:
0025         if not self.config_path.exists():
0026             return {}
0027         try:
0028             data = json.loads(self.config_path.read_text(encoding="utf-8"))
0029             return data if isinstance(data, dict) else {}
0030         except Exception as exc:
0031             print(f"[hook] invalid config: {exc}")
0032             return {}
0033
0034     def fire(self, event: str, payload: dict[str, Any]) -> HookResult:
0035         commands = self._config().get(event, [])
0036         if isinstance(commands, str):
0037             commands = [commands]
0038         result = HookResult()
0039         for command in commands:
0040             if not isinstance(command, str) or not command.strip():
0041                 continue
0042             envelope = {"event": event, "cwd": str(self.workspace), **payload}
0043             try:
0044                 proc = subprocess.run(
0045                     command,
0046                     shell=True,
0047                     cwd=self.workspace,
0048                     input=json.dumps(envelope, ensure_ascii=False),
0049                     text=True,
0050                     capture_output=True,
0051                     timeout=15,
0052                 )
0053             except subprocess.TimeoutExpired:
0054                 return HookResult(decision="deny", reason=f"hook timed out: {command}")
0055
0056             if proc.stderr.strip():
0057                 print(f"[hook:{event}:stderr] {proc.stderr.strip()[:2000]}")
0058             if proc.returncode != 0:
0059                 return HookResult(decision="deny", reason=f"hook failed ({proc.returncode}): {command}")
0060
0061             stdout = proc.stdout.strip()
0062             if not stdout:
0063                 continue
0064             try:
0065                 data = json.loads(stdout)
0066             except json.JSONDecodeError:
0067                 result.output += stdout + "\n"
0068                 continue
0069
0070             if isinstance(data, dict):
0071                 decision = str(data.get("decision", "allow"))
0072                 if decision in {"allow", "deny", "ask"}:
0073                     result.decision = decision
0074                 if data.get("reason"):
0075                     result.reason = str(data["reason"])
0076                 if isinstance(data.get("updated_input"), dict):
0077                     result.updated_input = data["updated_input"]
0078                 if data.get("output"):
0079                     result.output += str(data["output"]) + "\n"
0080                 if result.decision == "deny":
0081                     return result
0082         return result

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```

0001 """Persistent project instructions + agent-written memory for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from pathlib import Path
0005 from typing import Iterable
0006
0007

```

```

0008 class MemoryStore:
0009     def __init__(self, workspace: Path):
0010         self.workspace = workspace.resolve()
0011         self.project_instructions = self.workspace / "CLAUDE.md"
0012         self.memory_dir = self.workspace / ".mini_claude" / "memory"
0013         self.memory_file = self.memory_dir / "MEMORY.md"
0014
0015     def load_project_instructions(self) -> str:
0016         if not self.project_instructions.exists():
0017             return ""
0018         return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]
0019
0020     def load_auto_memory(self) -> str:
0021         if not self.memory_file.exists():
0022             return ""
0023         # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
0024         text = self.memory_file.read_text(encoding="utf-8", errors="replace")
0025         return "\n".join(text.splitlines()[:200])[:25000]
0026
0027     def remember(self, note: str) -> str:
0028         note = note.strip()
0029         if not note:
0030             return "ERROR: empty memory note"
0031         self.memory_dir.mkdir(parents=True, exist_ok=True)
0032         existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
0033         normalized = f"- {note}"
0034         if normalized in existing:
0035             return "OK: note already present"
0036         with self.memory_file.open("a", encoding="utf-8") as fh:
0037             if not existing.endswith("\n"):
0038                 fh.write("\n")
0039             fh.write(normalized + "\n")
0040         return "OK: persisted memory"
0041
0042     def search(self, query: str, limit: int = 20) -> str:
0043         if not self.memory_file.exists():
0044             return "(memory is empty)"
0045         q = query.casefold().strip()
0046         lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
0047         hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
0048         selected = list(hits)[:limit]
0049         return "\n".join(selected) if selected else "(no memory matches)"
0050
0051     def startup_context(self) -> str:
0052         parts = []
0053         project = self.load_project_instructions().strip()
0054         memory = self.load_auto_memory().strip()
0055         if project:
0056             parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
0057         if memory:
0058             parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
0059         return "\n\n".join(parts)

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

sandbox.py (102 lines)

```

0001 """Execution boundary for shell tools.
0002
0003 `soft` mode is portability-oriented and is NOT a strong OS security boundary.
0004 `bwrap` mode uses Bubblewrap on Linux when available and can isolate network + filesystem.
0005 """
0006 from __future__ import annotations
0007
0008 import os
0009 import shutil
0010 import subprocess
0011 import tempfile
0012 from dataclasses import dataclass
0013 from pathlib import Path
0014
0015
0016 @dataclass(frozen=True)
0017 class SandboxResult:
0018     exit_code: int
0019     stdout: str
0020     stderr: str
0021     mode: str
0022
0023
0024 class SandboxRunner:

```



```

0025 def __init__(self, workspace: Path, *, mode: str = "auto", disable_network: bool = True):
0026     self.workspace = workspace.resolve()
0027     self.requested_mode = mode
0028     self.disable_network = disable_network
0029     self.bwrap = shutil.which("bwrap")
0030
0031 @property
0032 def mode(self) -> str:
0033     if self.requested_mode == "auto":
0034         return "bwrap" if self.bwrap else "soft"
0035     if self.requested_mode == "bwrap" and not self.bwrap:
0036         raise RuntimeError("MINICLAUDE_SANDBOX=bwrap requested but `bwrap` is not installed")
0037     if self.requested_mode not in {"soft", "bwrap"}:
0038         raise ValueError("sandbox mode must be auto, soft, or bwrap")
0039     return self.requested_mode
0040
0041 def sanitized_env(self) -> dict[str, str]:
0042     allowed = {"PATH", "LANG", "LC_ALL", "TERM", "SHELL", "USER"}
0043     env = {k: v for k, v in os.environ.items() if k in allowed}
0044     env["HOME"] = str(self.workspace / ".mini_claude" / "sandbox_home")
0045     env["MINICLAUDE_SANDBOX"] = self.mode
0046     Path(env["HOME"]).mkdir(parents=True, exist_ok=True)
0047     return env
0048
0049 def run(self, command: str, *, timeout: int = 30) -> SandboxResult:
0050     mode = self.mode
0051     env = self.sanitized_env()
0052     if mode == "soft":
0053         proc = subprocess.run(
0054             ["bash", "-lc", command],
0055             cwd=self.workspace,
0056             env=env,
0057             text=True,
0058             capture_output=True,
0059             timeout=timeout,
0060         )
0061     return SandboxResult(proc.returncode, proc.stdout, proc.stderr, mode)
0062
0063 # Bubblewrap: workspace is writable; common system paths are read-only.
0064 # This is intentionally conservative and may need distro-specific adjustments.
0065 tmp = tempfile.mkdtemp(prefix="mini-claude-")
0066 argv = [
0067     self.bwrap or "bwrap",
0068     "--die-with-parent",
0069     "--new-session",
0070     "--proc", "/proc",
0071     "--dev", "/dev",
0072     "--tmpfs", "/tmp",
0073     "--bind", str(self.workspace), str(self.workspace),
0074     "--chdir", str(self.workspace),
0075 ]
0076 for system_path in ("/usr", "/bin", "/lib", "/lib64", "/etc"):
0077     if Path(system_path).exists():
0078         argv += ["--ro-bind", system_path, system_path]
0079 if self.disable_network:
0080     argv.append("--unshare-net")
0081 argv += ["--setenv", "HOME", str(self.workspace / ".mini_claude" / "sandbox_home")]
0082 argv += ["bash", "-lc", command]
0083
0084 try:
0085     proc = subprocess.run(
0086         argv,
0087         cwd=self.workspace,
0088         env=env,
0089         text=True,
0090         capture_output=True,
0091         timeout=timeout,
0092     )
0093     return SandboxResult(proc.returncode, proc.stdout, proc.stderr, mode)
0094 finally:
0095     shutil.rmtree(tmp, ignore_errors=True)
0096
0097 def status(self) -> str:
0098     return (
0099         f"requested={self.requested_mode}, active={self.mode}, "
0100         f"network={'disabled' if self.disable_network and self.mode == 'bwrap' else 'not strongly isolated'}, "
0101         f"workspace={self.workspace}"
0102     )

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

skills.py (70 lines)

```
0001 """Progressive-disclosure skills for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class Skill:
0013     name: str
0014     description: str
0015     path: Path
0016     body: str
0017     metadata: dict[str, Any]
0018
0019
0020 class SkillRegistry:
0021     def __init__(self, workspace: Path):
0022         self.workspace = workspace.resolve()
0023         self.roots = [
0024             self.workspace / ".mini_claude" / "skills",
0025             self.workspace / ".claude" / "skills", # optional compatibility
0026         ]
0027
0028     @staticmethod
0029     def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0030         if not text.startswith("---\n"):
0031             return {}, text
0032         end = text.find("\n---\n", 4)
0033         if end == -1:
0034             return {}, text
0035         raw = text[4:end]
0036         body = text[end + 5 :]
0037         data = yaml.safe_load(raw) or {}
0038         if not isinstance(data, dict):
0039             data = {}
0040         return data, body
0041
0042     def discover(self) -> dict[str, Skill]:
0043         found: dict[str, Skill] = {}
0044         for root in self.roots:
0045             if not root.exists():
0046                 continue
0047             for skill_file in sorted(root.glob("**/SKILL.md")):
0048                 text = skill_file.read_text(encoding="utf-8", errors="replace")
0049                 meta, body = self._split_frontmatter(text)
0050                 name = str(meta.get("name") or skill_file.parent.name)
0051                 description = str(meta.get("description") or "No description provided")
0052                 found[name] = Skill(name, description, skill_file, body.strip(), meta)
0053         return found
0054
0055     def catalog(self) -> str:
0056         skills = self.discover()
0057         if not skills:
0058             return "(no skills installed)"
0059         return "\n".join(f"- {name}: {skill.description}" for name, skill in skills.items())
0060
0061     def load(self, name: str) -> str:
0062         skill = self.discover().get(name)
0063         if skill is None:
0064             return f"ERROR: unknown skill {name!r}. Available:\n{self.catalog()}"
0065         # A skill can reference support files by relative path; normal file tools can read them later.
0066         return (
0067             f"# Skill: {skill.name}\n"
0068             f"Source: {skill.path.relative_to(self.workspace)}\n\n"
0069             f"{skill.body[:30000]}"
0070         )
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

team.py (260 lines)

```
0001 """Parallel multi-agent runtime with Git worktree isolation and a tiny mailbox."""
0002 from __future__ import annotations
0003
0004 import json
```

```

0005 import os
0006 import re
0007 import shutil
0008 import subprocess
0009 import sys
0010 from concurrent.futures import ThreadPoolExecutor, as_completed
0011 from dataclasses import dataclass
0012 from datetime import datetime, timezone
0013 from pathlib import Path
0014 from typing import Any
0015
0016
0017 @dataclass(frozen=True)
0018 class WorkerSpec:
0019     name: str
0020     agent: str
0021     prompt: str
0022
0023
0024 class TeamBus:
0025     """Very small file-backed mailbox shared by worker processes."""
0026
0027     def __init__(self):
0028         raw = os.getenv("MINICLAUDE_TEAM_DIR")
0029         self.root = Path(raw).resolve() if raw else None
0030         self.me = os.getenv("MINICLAUDE_WORKER_NAME")
0031         try:
0032             self.members = json.loads(os.getenv("MINICLAUDE_TEAM_MEMBERS", "[]"))
0033         except json.JSONDecodeError:
0034             self.members = []
0035
0036     @property
0037     def enabled(self) -> bool:
0038         return bool(self.root and self.me)
0039
0040     def context(self) -> str:
0041         if not self.enabled:
0042             return ""
0043         return (
0044             f"You are teammate {self.me}. Team members: {' '.join(self.members)}."
0045             "Use team_send for concise discoveries that unblock another teammate. "
0046             "Use team_inbox when you expect a reply."
0047         )
0048
0049     def send(self, to: str, message: str) -> str:
0050         if not self.enabled or self.root is None or self.me is None:
0051             return "ERROR: not running inside a team"
0052         if to not in self.members:
0053             return f"ERROR: unknown teammate {to!r}"
0054         mailbox = self.root / "mailboxes" / f"{to}.jsonl"
0055         mailbox.parent.mkdir(parents=True, exist_ok=True)
0056         record = {
0057             "from": self.me,
0058             "to": to,
0059             "time": datetime.now(timezone.utc).isoformat(),
0060             "message": message,
0061         }
0062         with mailbox.open("a", encoding="utf-8") as fh:
0063             fh.write(json.dumps(record, ensure_ascii=False) + "\n")
0064         return f"OK: sent message to {to}"
0065
0066     def inbox(self) -> str:
0067         if not self.enabled or self.root is None or self.me is None:
0068             return "ERROR: not running inside a team"
0069         mailbox = self.root / "mailboxes" / f"{self.me}.jsonl"
0070         if not mailbox.exists():
0071             return "(inbox empty)"
0072         lines = mailbox.read_text(encoding="utf-8", errors="replace").splitlines()
0073         return "\n".join(lines[-50:]) if lines else "(inbox empty)"
0074
0075
0076 class WorktreeManager:
0077     def __init__(self, repo: Path):
0078         self.repo = repo.resolve()
0079         self.root = self.repo / ".mini_claude" / "worktrees"
0080
0081     def _git(self, *args: str, check: bool = True) -> subprocess.CompletedProcess[str]:
0082         return subprocess.run(
0083             ["git", "-C", str(self.repo), *args],
0084             text=True,

```

```

0085         capture_output=True,
0086         check=check,
0087     )
0088
0089     def is_git_repo(self) -> bool:
0090         proc = self.git("rev-parse", "--is-inside-work-tree", check=False)
0091         return proc.returncode == 0 and proc.stdout.strip() == "true"
0092
0093     @staticmethod
0094     def slug(text: str) -> str:
0095         value = re.sub(r"[^a-zA-Z0-9._-]+", "-", text).strip("-.")
0096         return value[:60] or "worker"
0097
0098     def create(self, team_name: str, worker_name: str) -> tuple[Path, str]:
0099         if not self.is_git_repo():
0100             raise RuntimeError("multi-agent worktree mode requires a Git repository")
0101         team_slug = self.slug(team_name)
0102         worker_slug = self.slug(worker_name)
0103         branch = f"mini/{team_slug}/{worker_slug}"
0104         path = (self.root / team_slug / worker_slug).resolve()
0105         path.parent.mkdir(parents=True, exist_ok=True)
0106         if path.exists():
0107             shutil.rmtree(path)
0108         proc = subprocess.run(
0109             ["git", "-C", str(self.repo), "worktree", "add", "-b", branch, str(path), "HEAD"],
0110             text=True,
0111             capture_output=True,
0112         )
0113         if proc.returncode != 0:
0114             raise RuntimeError(proc.stderr.strip() or proc.stdout.strip())
0115         return path, branch
0116
0117     @staticmethod
0118     def status(path: Path) -> tuple[str, str]:
0119         status = subprocess.run(
0120             ["git", "-C", str(path), "status", "--short"],
0121             text=True,
0122             capture_output=True,
0123         ).stdout.strip()
0124         diff = subprocess.run(
0125             ["git", "-C", str(path), "diff", "--stat"],
0126             text=True,
0127             capture_output=True,
0128         ).stdout.strip()
0129         return status, diff
0130
0131     def remove_if_clean(self, path: Path, branch: str) -> bool:
0132         status, _ = self.status(path)
0133         if status:
0134             return False
0135         subprocess.run(
0136             ["git", "-C", str(self.repo), "worktree", "remove", "--force", str(path)],
0137             text=True,
0138             capture_output=True,
0139         )
0140         subprocess.run(
0141             ["git", "-C", str(self.repo), "branch", "-D", branch],
0142             text=True,
0143             capture_output=True,
0144         )
0145         return True
0146
0147
0148 class TeamRuntime:
0149     def __init__(self, workspace: Path, entry_script: Path):
0150         self.workspace = workspace.resolve()
0151         self.entry_script = entry_script.resolve()
0152         self.worktrees = WorktreeManager(self.workspace)
0153
0154     def run(self, team_name: str, tasks: list[dict[str, Any]]) -> str:
0155         if not tasks:
0156             return "ERROR: tasks must not be empty"
0157         if len(tasks) > 8:
0158             return "ERROR: tutorial runtime limits a team to 8 workers"
0159
0160         specs: list[WorkerSpec] = []
0161         names: set[str] = set()
0162         for i, item in enumerate(tasks, start=1):
0163             name = WorktreeManager.slug(str(item.get("name") or f"worker-{i}"))
0164             if name in names:

```

```

0165         return f"ERROR: duplicate worker name {name!r}"
0166     names.add(name)
0167     specs.append(
0168         WorkerSpec(
0169             name=name,
0170             agent=str(item.get("agent") or "implementer"),
0171             prompt=str(item.get("prompt") or "").strip(),
0172         )
0173     )
0174     if any(not spec.prompt for spec in specs):
0175         return "ERROR: every worker needs a non-empty prompt"
0176
0177     team_slug = WorktreeManager.slug(team_name)
0178     team_dir = (self.workspace / ".mini_claude" / "teams" / team_slug).resolve()
0179     team_dir.mkdir(parents=True, exist_ok=True)
0180     members = [s.name for s in specs]
0181     (team_dir / "team.json").write_text(
0182         json.dumps({"team": team_name, "members": members}, ensure_ascii=False, indent=2),
0183         encoding="utf-8",
0184     )
0185
0186     assignments: List[tuple[WorkerSpec, Path, str]] = []
0187     try:
0188         for spec in specs:
0189             path, branch = self.worktrees.create(team_name, spec.name)
0190             assignments.append((spec, path, branch))
0191     except Exception as exc:
0192         return f"ERROR creating worktrees: {type(exc).__name__}: {exc}"
0193
0194     def worker_job(spec: WorkerSpec, path: Path, branch: str) -> dict[str, Any]:
0195         env = os.environ.copy()
0196         env.update(
0197             {
0198                 "MINICLAUDE_NONINTERACTIVE": "1",
0199                 "MINICLAUDE_WORKER_ALLOW_EDITS": "1",
0200                 "MINICLAUDE_TEAM_DIR": str(team_dir),
0201                 "MINICLAUDE_WORKER_NAME": spec.name,
0202                 "MINICLAUDE_TEAM_MEMBERS": json.dumps(members),
0203             }
0204         )
0205         proc = subprocess.run(
0206             [
0207                 sys.executable,
0208                 str(self.entry_script),
0209                 "--worker",
0210                 "--agent",
0211                 spec.agent,
0212                 "--prompt",
0213                 spec.prompt,
0214             ],
0215             cwd=path,
0216             env=env,
0217             text=True,
0218             capture_output=True,
0219             timeout=900,
0220         )
0221         status, diff = self.worktrees.status(path)
0222         cleaned = False
0223         if proc.returncode == 0 and not status:
0224             cleaned = self.worktrees.remove_if_clean(path, branch)
0225         return {
0226             "name": spec.name,
0227             "agent": spec.agent,
0228             "branch": branch,
0229             "worktree": str(path),
0230             "exit_code": proc.returncode,
0231             "stdout": proc.stdout[-12000:],
0232             "stderr": proc.stderr[-4000:],
0233             "git_status": status,
0234             "diff_stat": diff,
0235             "cleaned_up": cleaned,
0236         }
0237
0238     results: List[dict[str, Any]] = []
0239     with ThreadPoolExecutor(max_workers=len(assignments)) as pool:
0240         futures = {
0241             pool.submit(worker_job, spec, path, branch): spec.name
0242             for spec, path, branch in assignments
0243         }
0244         for future in as_completed(futures):

```

```

0245         name = futures[future]
0246         try:
0247             results.append(future.result())
0248         except Exception as exc:
0249             results.append({"name": name, "error": f"{type(exc).__name__}: {exc}"})
0250
0251     results.sort(key=lambda x: str(x.get("name", "")))
0252     return json.dumps(
0253         {
0254             "team": team_name,
0255             "note": "Changed worktrees are intentionally left for review; this runtime never auto-merges branches.",
0256             "results": results,
0257         },
0258         ensure_ascii=False,
0259         indent=2,
0260     )

```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

12. 从教程版走向可维护 Runtime：重构路线

s01-s10 故意强调“每层能看懂”，因此 code.py 逐步变长。真正维护时应在理解完成后再做一次结构化重构，而不是一开始就把所有抽象拆成十几个目录。

12.1 推荐最终包结构

```
mini_claude/  
  app.py          # CLI / REPL / non-interactive entry  
  llm.py          # Messages API adapter  
  loop.py         # agent loop state machine  
  tools/          # schemas + handlers  
  policy/         # permissions  
  context.py      # budget + compaction  
  memory.py       # instructions + durable memory  
  skills.py       # discovery + loading  
  hooks.py        # lifecycle bus  
  agents.py       # subagent registry/runtime  
  sandbox.py      # process isolation adapters  
  team.py         # worktrees + worker orchestration  
  telemetry.py    # events / costs / traces  
.claude or .mini_claude/  
  skills/  
  agents/  
  settings / hooks / memory
```

12.2 生产化还缺什么

- 模型感知的精确 token counting、prompt caching、streaming 与 retry/backoff。
- 结构化事件/trace：每轮模型输入、工具调用、权限决定、Hook 决定、成本与延迟。
- 更严格的 Tool Schema validation、输出类型、并行 tool call 调度和幂等性。
- 强 OS sandbox/容器/VM、网络 egress proxy、secret broker、资源 quota。
- 非交互权限策略文件、组织级 policy、审计签名与策略版本。
- 可靠的 session persistence、JSONL/event sourcing、checkpoint/replay。
- MCP client/server 接入与 Tool Search；本教程故意把它留在 s10 之后作为下一阶段。
- Agent eval：任务成功率、工具选择准确率、permission friction、context efficiency、回归集。
- Team scheduler：依赖图、取消、超时、重试、任务窃取、消息可靠性和 merge/rebase 策略。

13. 调试手册：Agent 为什么“看起来很笨”

症状	根因	优先修法
工具老选错	Schema/description 含糊或相似工具重叠	缩窄工具边界；用 eval 验证选择率；不要只改 system prompt。
不断重复读文件	Context 中没有稳定的状态摘要	增加明确 plan/state；在适当节点 compact；Subagent 只回结论。
写完就说完成	没有可执行 verifier	把 test/lint/build/fixture/screenshot 作为 stopping condition。
频繁弹权限	allow policy 过窄或工具粒度太通用	把重复低风险操作变成明确 rule；高风险保留 ask。
CLAUDE.md 越写越长	把程序和临时知识都塞进启动 Context	事实放 CLAUDE.md，程序放 Skill，经验放 Memory，强制动作放 Hook。
Subagent 没省 Context	把主历史整段复制给 worker，或 worker 返回完整轨迹	只给 task + 必要 project context；返回 structured summary。
多 Agent 越多越慢	任务存在强依赖或 worker 争用同一文件	先做 dependency graph；只有真正可分割任务才并行。

症状	根因	优先修法
Sandbox 开了仍不放心	使用 soft 模式或网络/secret 仍共享	把 sandbox status 可见化；生产用 OS/VM containment + egress/secret policy。

14. 练习：把 Mini Claude Code 继续演进到 s11-s15

s11 Session Persistence

用 append-only JSONL 保存 user/assistant/tool events；实现 --resume 与 fork。

s12 MCP

实现 MCP client，动态注册 external tools；比较“schema 常驻”与 Tool Search。

s13 Verification Runtime

把 tests/lint/typecheck 变成 first-class verifier，支持失败反馈和 stopping policy。

s14 Eval Harness

为工具选择、权限、修复成功率建立离线任务集和可重复评测。

s15 Agent Team Scheduler

加入 DAG task board、worker heartbeat、可靠 mailbox、取消/超时、worktree merge policy。

15. 参考资料与公开机制映射

下列资料用于校准公开协议和产品概念。教程代码是原创教学实现；参考资料不意味着内部源码一一对应。

[R1] Claude Platform Docs - How tool use works

客户端工具的规范循环：tool_use -> Host execution -> tool_result -> continue。

<https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>

[R2] Claude Platform Docs - Handle tool calls

tool_use block、tool_result、is_error 与 tool_use_id 的协议语义。

<https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls>

[R3] Claude Code Docs - How Claude Code works

Claude Code 的 Agentic loop、可访问资源、上下文与 compaction 概览。

<https://code.claude.com/docs/en/how-claude-code-works>

[R4] Claude Code Docs - How Claude remembers your project

CLAUDE.md、auto memory、加载范围和跨会话知识。

<https://code.claude.com/docs/en/memory>

[R5] Claude Code Docs - Extend Claude with skills

Skill 的按需加载、SKILL.md 与可复用程序。

<https://code.claude.com/docs/en/slash-commands>

[R6] Claude Code Docs - Hooks reference / guide

PreToolUse/PostToolUse、Session、Compact、Subagent、Worktree 等生命周期事件。

<https://code.claude.com/docs/en/hooks>

[R7] Claude Code Docs - Create custom subagents

独立 context、tool allowlist、max turns、isolation 等 Subagent 机制。

<https://code.claude.com/docs/en/sub-agents>

[R8] Claude Code Docs - Run parallel sessions with worktrees

使用 Git worktree 隔离并行会话和 Subagent 的文件修改。

<https://code.claude.com/docs/en/worktrees>

[R9] Claude Code Docs - Orchestrate teams of Claude Code sessions

Agent Teams 的独立 context、共享任务和消息协调；当前文档仍标注 experimental。

<https://code.claude.com/docs/en/agent-teams>

[R10] Claude Code Docs - Extend Claude Code / Tools reference

CLAUDE.md、Skills、Subagents、Hooks、MCP/Tools 在 Agent loop 中的扩展位置。

<https://code.claude.com/docs/en/features-overview>

16. 最终心智模型

一句话

真正的 Coding Agent 不是“LLM + 一个 Prompt”，而是 Model x Context x Tools x Policy x Feedback x Isolation x Lifecycle。s01-s10 的目的，就是让这些乘数项一层层显形。

- Agent Loop 决定“能否继续行动”。
- Tools 决定“能做什么”。
- Permissions 决定“这次能不能做”。
- Context 决定“当前能想起什么”。
- Memory 决定“跨会话保留什么”。
- Skills 决定“程序性经验如何按需注入”。
- Hooks 决定“哪些动作必须确定发生”。
- Subagents 决定“哪些认知工作应隔离”。
- Sandbox 决定“最坏情况下能影响多大范围”。
- Multi-Agent Runtime 决定“如何把多个独立 Agent 变成一个可协调的软件工程系统”。